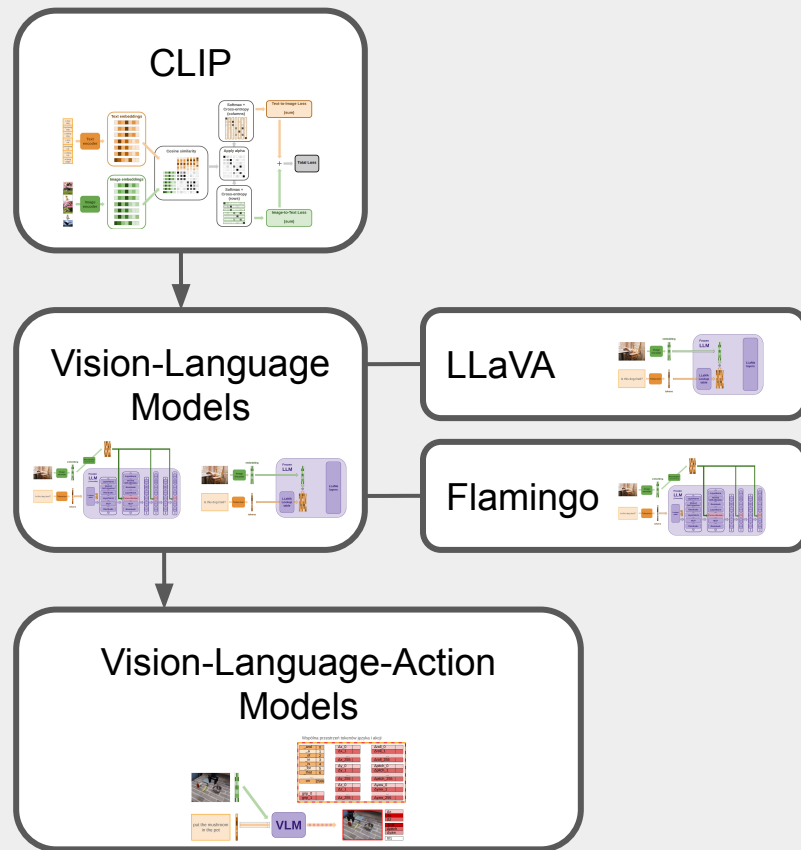


Modele multimodalne

Od łączenia faktów do działania

Mapa wykładu



Czy człowiek jest multimodalny?



Czy człowiek jest multimodalny?

Co możemy wywnioskować z tych obrazków?





Czy człowiek jest multimodalny?

Co możemy wywnioskować z tych wypowiedzi?

Zdecydowałem – chcę różowego!



Mogłem wyjść wcześniej – wtedy bym zdążył.



Super, że kupiłeś kota





Czy człowiek jest multimodalny?

Co możemy wywnioskować łącząc informacje?

Zdecydowałem – chcę różowego!



Mogłem wyjść wcześniej – wtedy bym zdążył.



Super, że kupiłeś kota

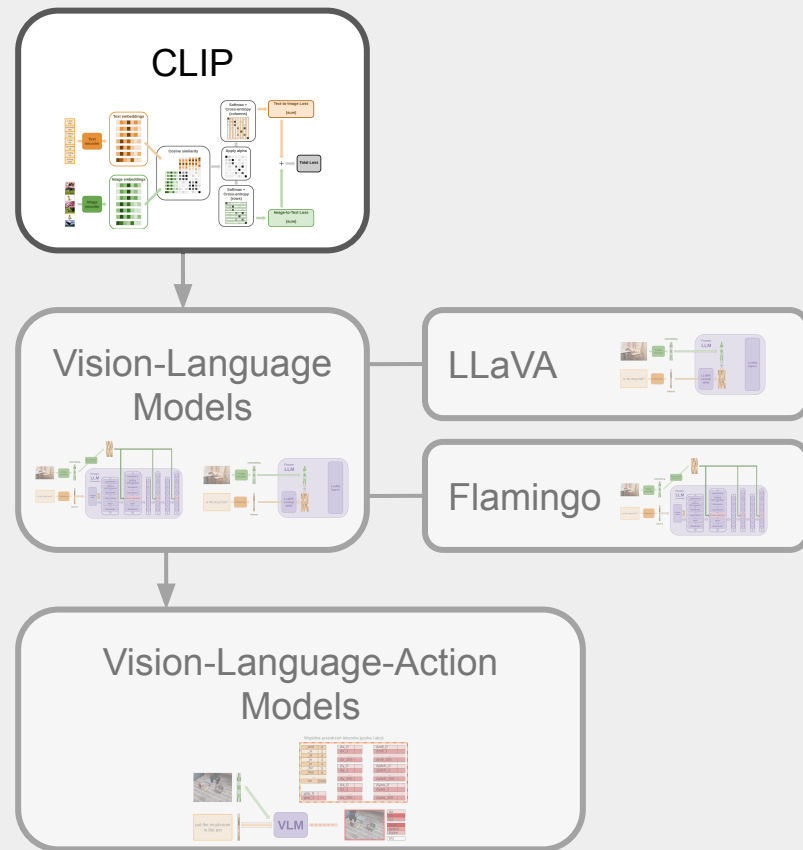


Multimodalność

Multimodalność to ***integracja** wielu źródeł informacji*
w jednym procesie reprezentacji i wnioskowania.

CLIP

Czyli łączenie faktów



Integracja obrazu i tekstu

*Komentarz:
Zarówno obraz jak i tekst możemy reprezentować
w postaci jednego wektora, korzystając ze
zagregowanego embeddingu.*



a flying eagle



a jumping cat



a lazy dog

Integracja obrazu i tekstu

Komentarz:

Chcemy mieć takie reprezentacje obrazkowe i tekstowe, że odpowiadające sobie znaczeniowo pary (obraz, tekst) mają tę samą reprezentację.



a flying eagle

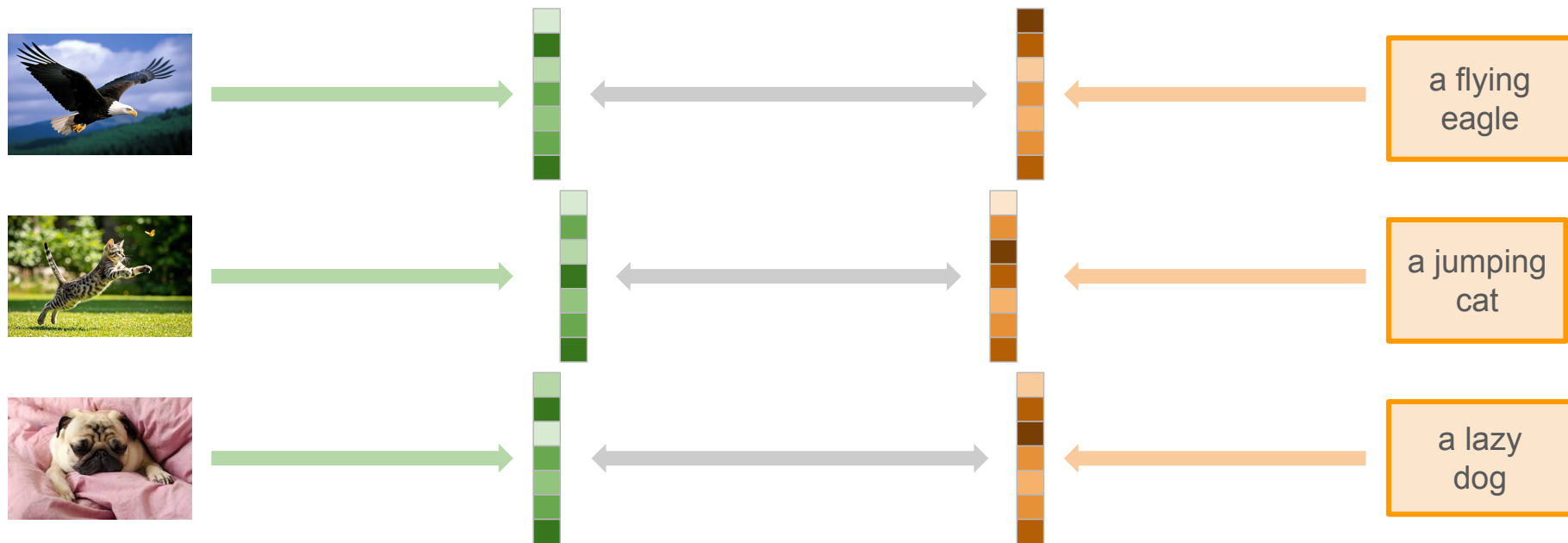


a jumping cat



a lazy dog

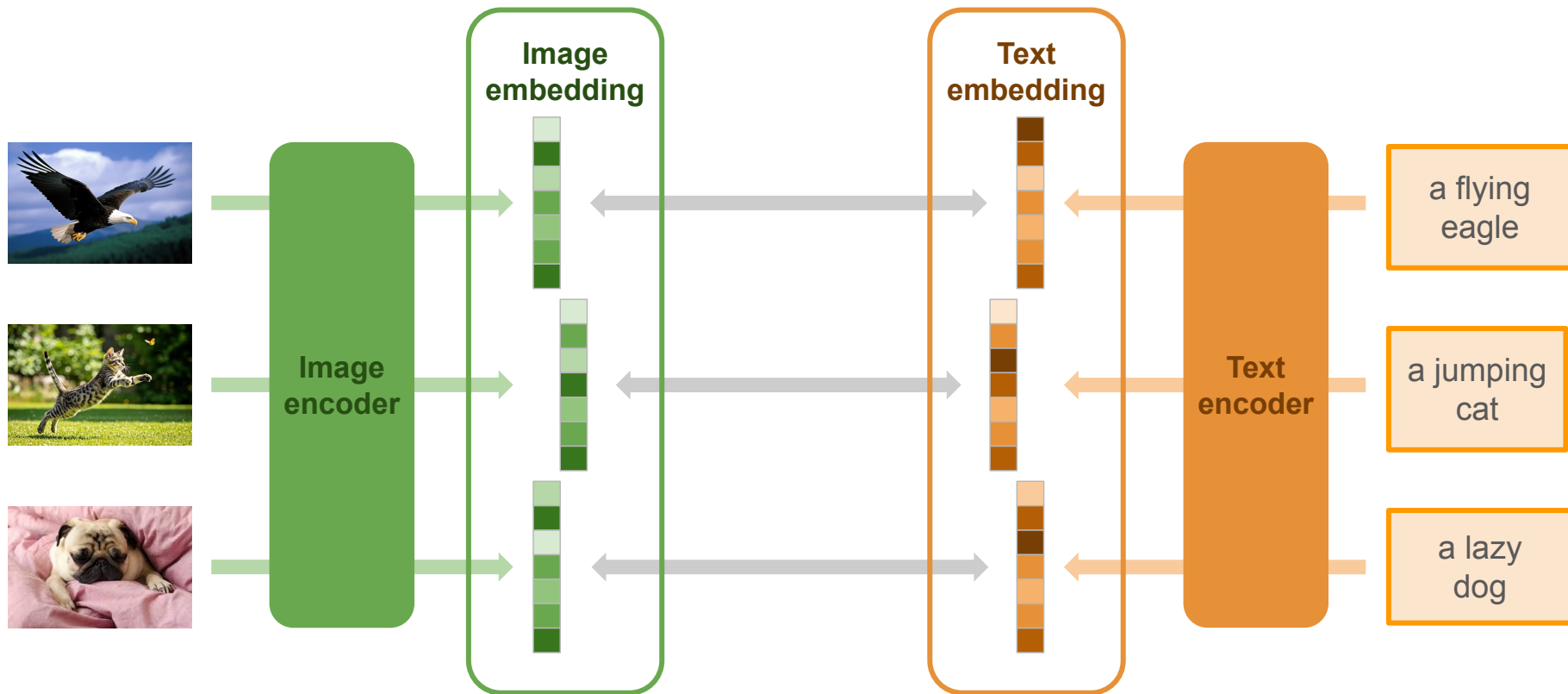
Integracja obrazu i tekstu



Integracja obrazu i tekstu

Komentarz:

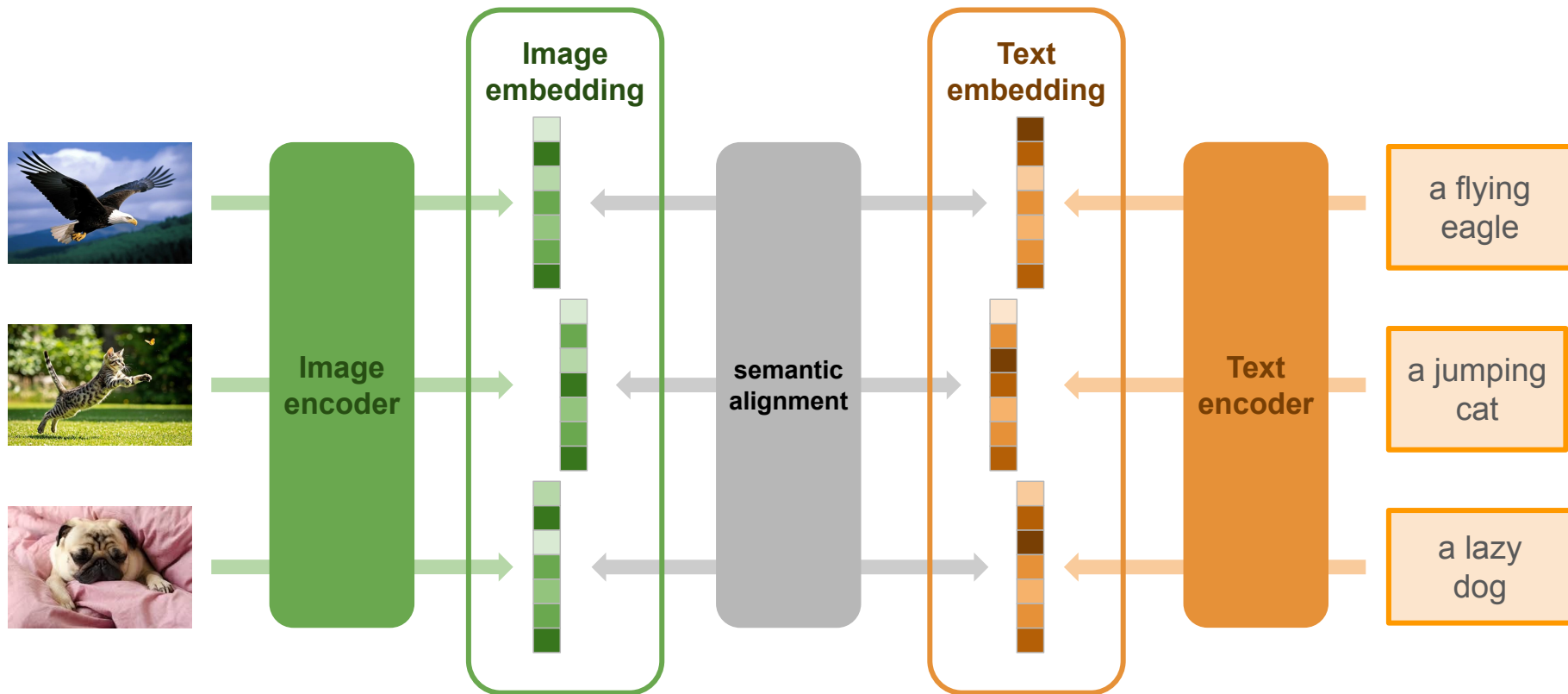
Reprezentacje pochodzą z enkodera obrazkowego i enkodera tekstowego. Te dwa enkodery będziemy chcieli wytrenować, tak aby zwracały odpowiadające sobie dopasowania



Integracja obrazu i tekstu

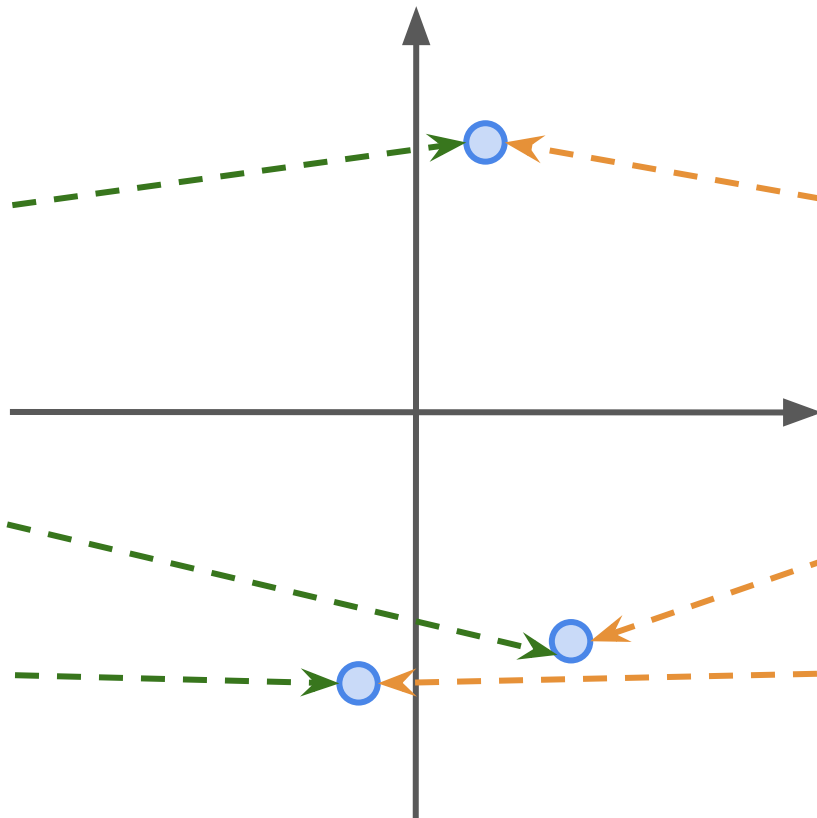
Komentarz:

W celu wytrenowania enkoderów będziemy potrzebowali funkcji definiującej semantyczne dopasowanie embeddingów pochodzących z dwóch różnych przestrzeni.



Integracja obrazu i tekstu

Wspólna przestrzeń



a flying
eagle

a jumping
cat

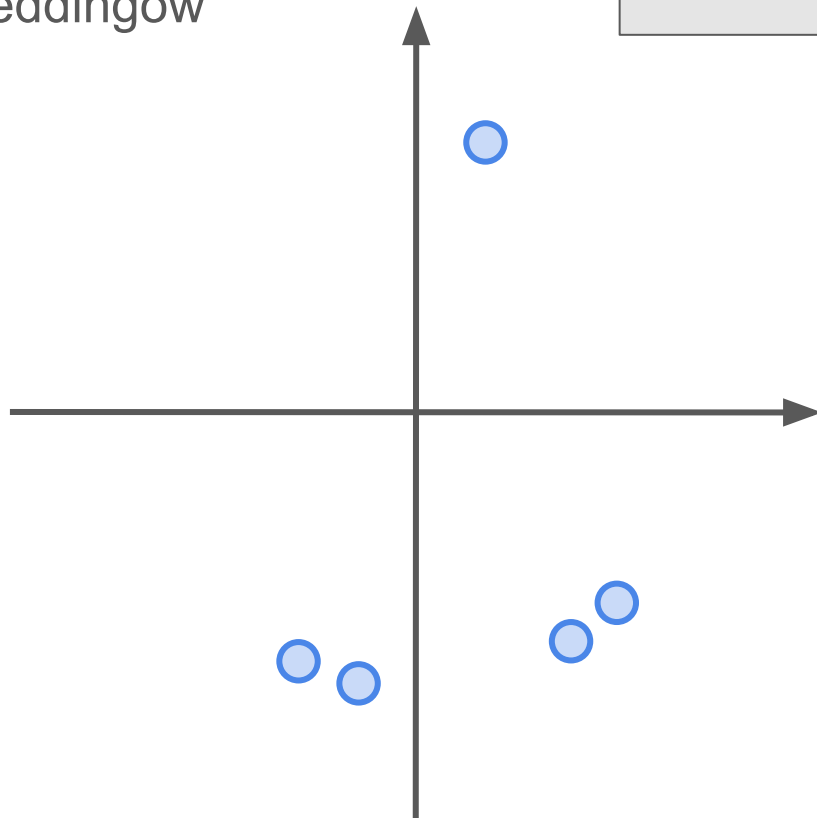
a lazy
dog

Komentarz:

Aby móc porównywać embeddingi tekstowe i obrazkowe, umieścimy je we wspólnej przestrzeni n -wymiarowej. Odpowiadające sobie znaczeniowo pary powinny trafić w ten sam punkt.

Integracja obrazu i tekstu

Porównywanie embeddingów



Komentarz:

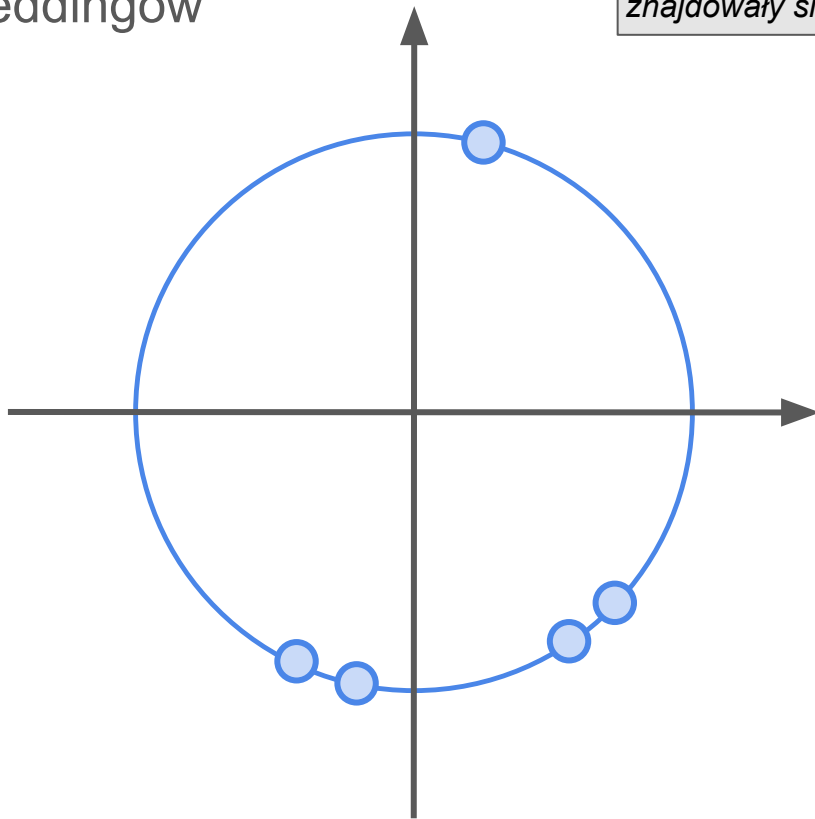
Potrzebujemy metryki, która dla pary wektorów (embeddingów) będzie zwracała informację o tym, jak daleko siebie są te dwa wektory (czyli ich warstwy znaczeniowe).

Integracja obrazu i tekstu

Porównywanie embeddingów

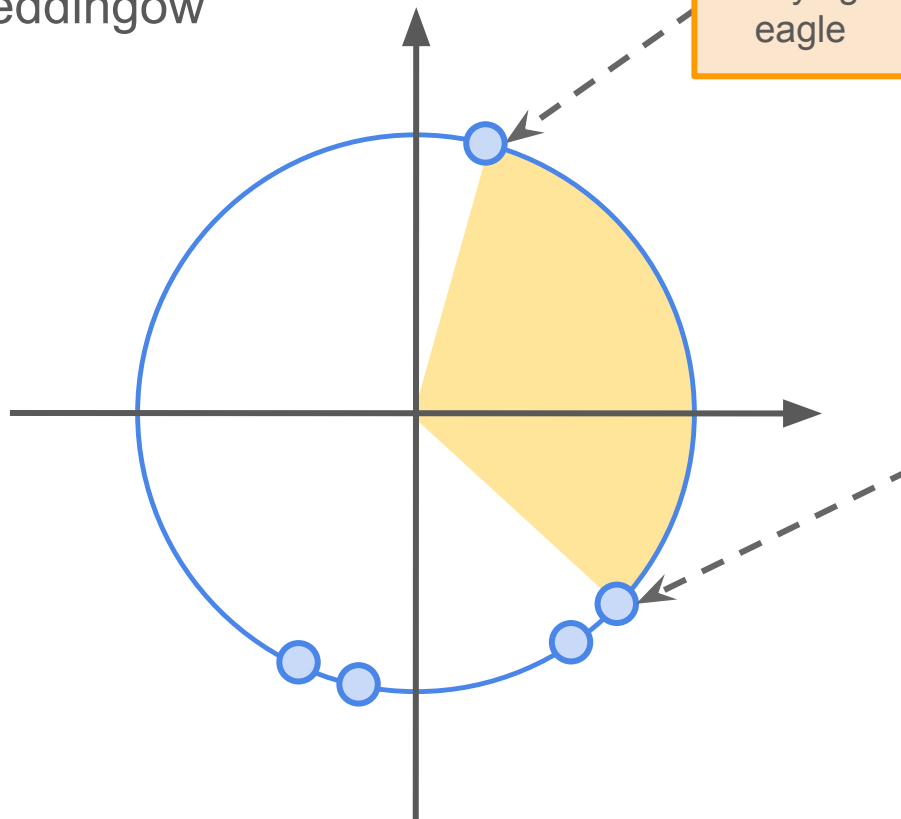
Komentarz:

Dla ułatwienia, będziemy zwracać uwagę tylko na kierunek wektorów. W tym celu będziemy normalizować embeddingi, tak aby miały długość jeden. Dzięki temu wszystkie wektory będą znajdowały się na hypersferze o promieniu 1.

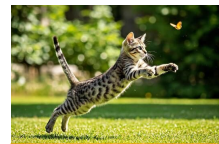


Integracja obrazu i tekstu

Porównywanie embeddingów



a flying
eagle



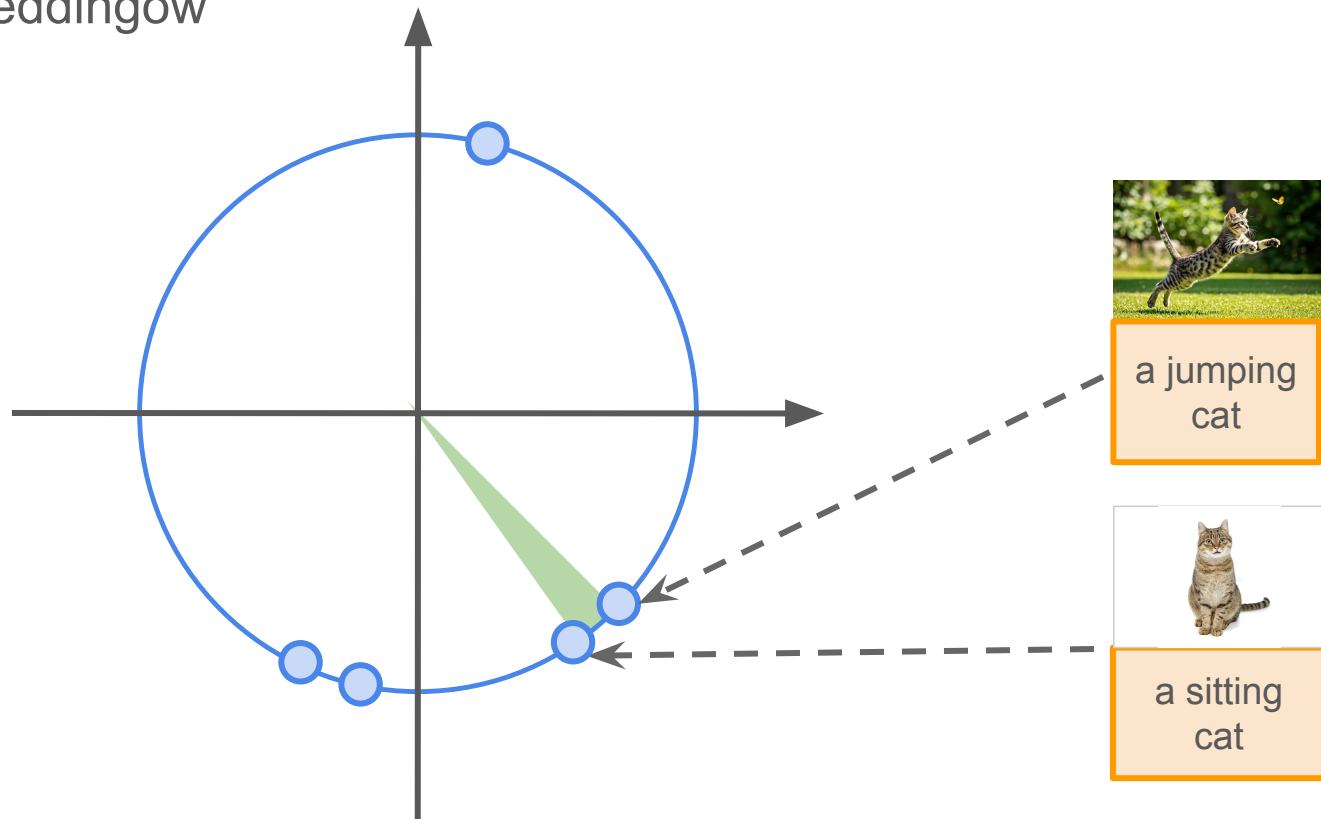
a jumping
cat

*Komentarz:
Wektory stosunkowo
daleko siebie (duży kąt)*

Integracja obrazu i tekstu

Porównywanie embeddingów

*Komentarz:
Wektory stosunkowo
blisko siebie (mały kąt)*



Integracja obrazu i tekstu

Porównywanie embeddingów

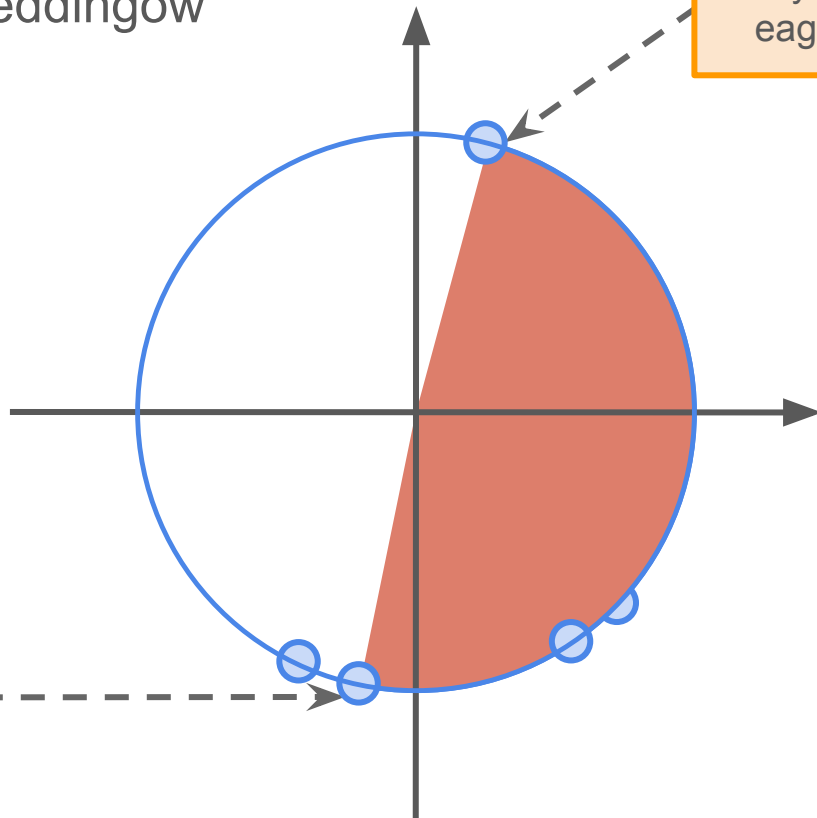


a flying
eagle

*Komentarz:
Wektory bardzo daleko
siebie (b. duży kąt)*



a lazy
dog



Integracja obrazu i tekstu

Porównywanie embeddingów



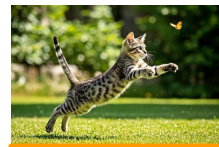
a sitting
dog



a lazy
dog



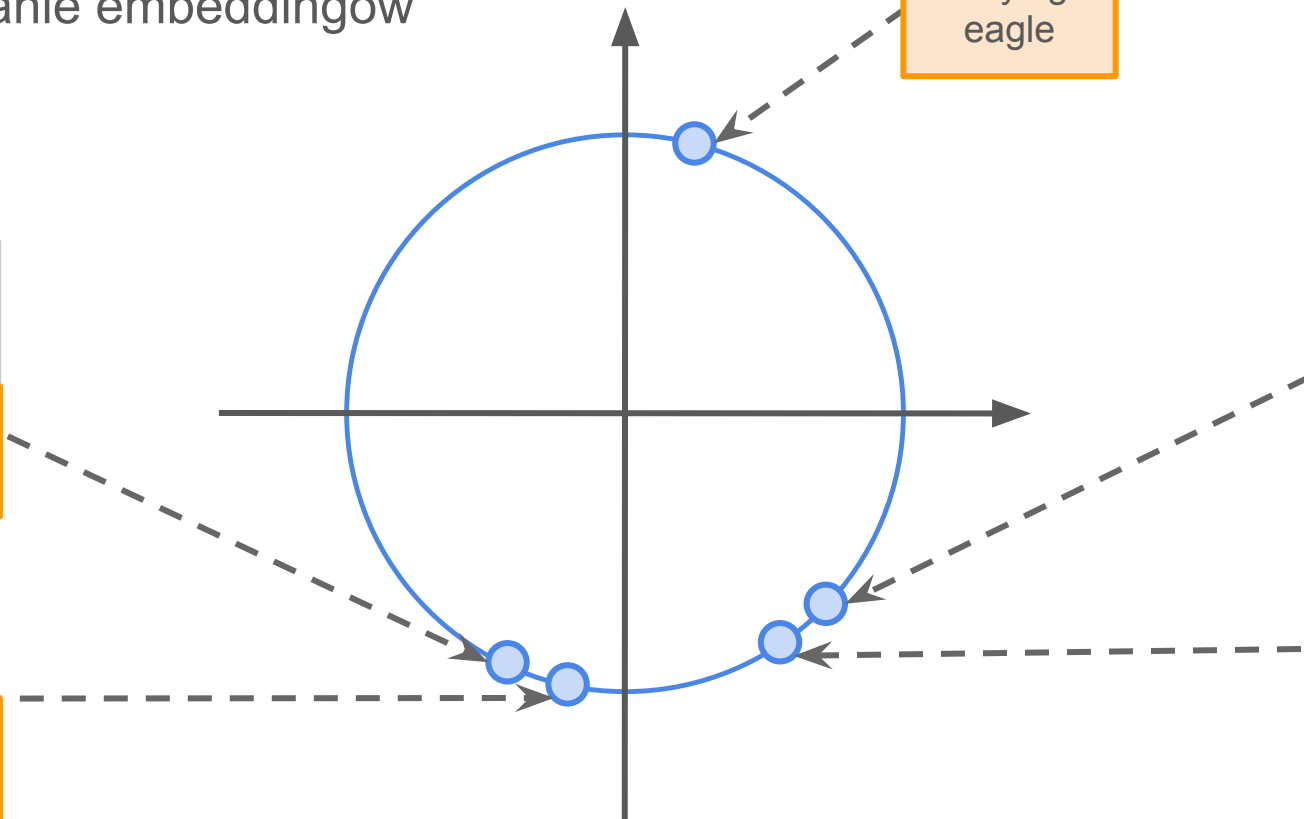
a flying
eagle



a jumping
cat



a sitting
cat



Komentarz:

*W ogólności:
czym mniejszy kąt, tym
bliższe znaczeniowo są
reprezentacje.*

Integracja obrazu i tekstu

Porównywanie embeddingów

Iloczyn skalarny wektorów

$$\mathbf{v} = (v_1, v_2, v_3, v_4)$$

$$\mathbf{w} = (w_1, w_2, w_3, w_4),$$

$$\mathbf{v} \cdot \mathbf{w} = v_1 w_1 + v_2 w_2 + v_3 w_3 + v_4 w_4$$

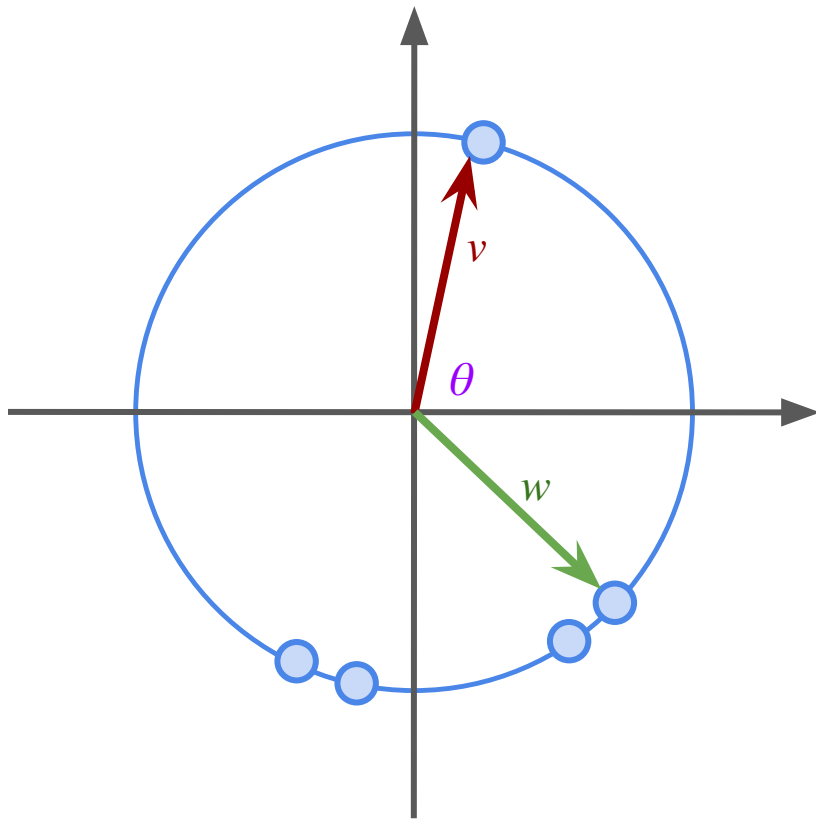
Własność iloczynu skalarnego

$$\mathbf{v} \cdot \mathbf{w} = \|\mathbf{v}\| \|\mathbf{w}\| \cos(\theta)$$

gdzie:

$\|\mathbf{v}\|$, $\|\mathbf{w}\|$ – długości wektorów

θ – kąt między nimi



Integracja obrazu i tekstu

Porównywanie embeddingów

Iloczyn skalarny wektorów

$$\mathbf{v} = (v_1, v_2, v_3, v_4)$$

$$\mathbf{w} = (w_1, w_2, w_3, w_4),$$

$$\mathbf{v} \cdot \mathbf{w} = v_1 w_1 + v_2 w_2 + v_3 w_3 + v_4 w_4$$

Własność iloczynu skalarnego znormalizowanych wektorów

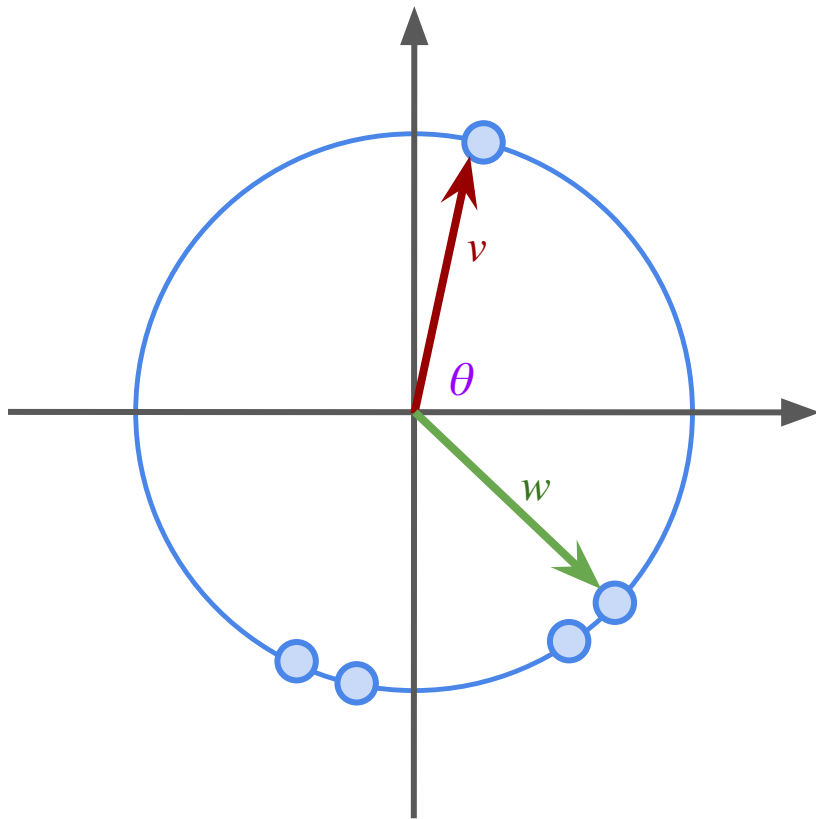
$$\mathbf{v} \cdot \mathbf{w} = \|\mathbf{v}\| \|\mathbf{w}\| \cos(\theta)$$

gdzie:

$\|\mathbf{v}\|$, $\|\mathbf{w}\|$ – długości wektorów

θ – kąt między nimi

*Komentarz:
Iloczyn skalarny dla znormalizowanych
wektorów to cosinus kąta między nimi.
Taki iloczyn skalarny to zwykle mnożenie
odpowiednich wektorów.*



Integracja obrazu i tekstu

Porównywanie embeddingów

Iloczyn skalarny wektorów

$$\mathbf{v} = (v_1, v_2, v_3, v_4)$$

$$\mathbf{w} = (w_1, w_2, w_3, w_4),$$

$$\mathbf{v} \cdot \mathbf{w} = v_1 w_1 + v_2 w_2 + v_3 w_3 + v_4 w_4$$

Własność iloczynu skalarnego
znormalizowanych wektorów

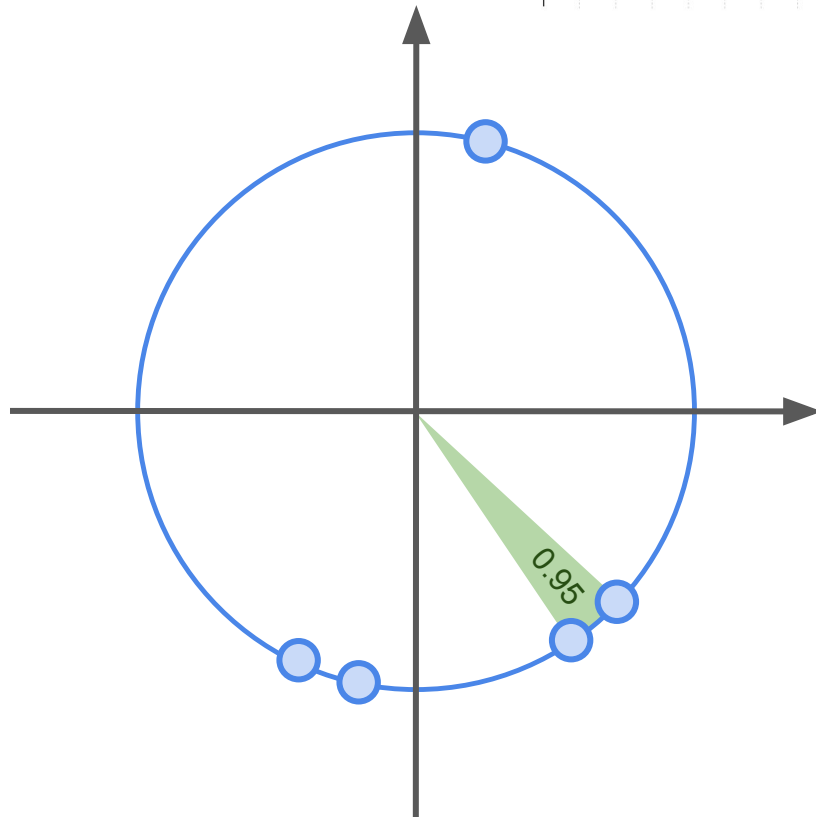
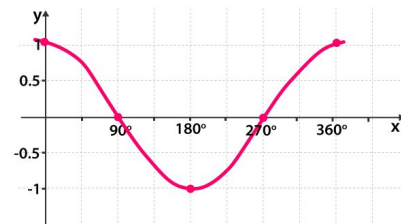
$$\mathbf{v} \cdot \mathbf{w} = \|\mathbf{v}\| \|\mathbf{w}\| \cos(\theta)$$

gdzie:

$\|\mathbf{v}\|$, $\|\mathbf{w}\|$ – długości wektorów

θ – kąt między nimi

*Komentarz:
Dla kąta bliskiemu zero
cosinus jest bliski
wartości 1.*



Integracja obrazu i tekstu

Porównywanie embeddingów

Iloczyn skalarny wektorów

$$\mathbf{v} = (v_1, v_2, v_3, v_4)$$

$$\mathbf{w} = (w_1, w_2, w_3, w_4),$$

$$\mathbf{v} \cdot \mathbf{w} = v_1 w_1 + v_2 w_2 + v_3 w_3 + v_4 w_4$$

Własność iloczynu skalarnego
znormalizowanych wektorów

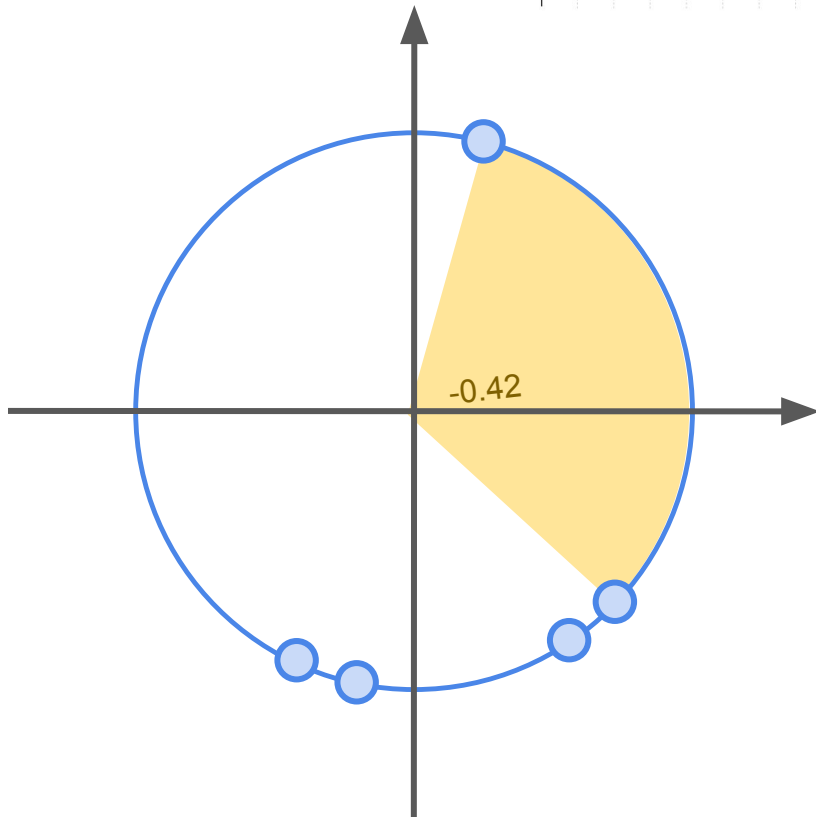
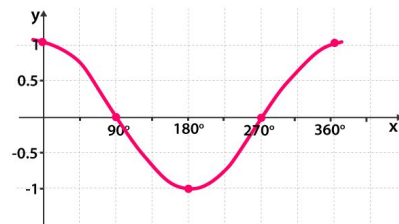
$$\mathbf{v} \cdot \mathbf{w} = \|\mathbf{v}\| \|\mathbf{w}\| \cos(\theta)$$

gdzie:

$\|\mathbf{v}\|$, $\|\mathbf{w}\|$ – długości wektorów

θ – kąt między nimi

Komentarz:
Dla kąta rozwartego
cosinus jest ujemny.



Integracja obrazu i tekstu

Porównywanie embeddingów

Iloczyn skalarny wektorów

$$\mathbf{v} = (v_1, v_2, v_3, v_4)$$

$$\mathbf{w} = (w_1, w_2, w_3, w_4),$$

$$\mathbf{v} \cdot \mathbf{w} = v_1 w_1 + v_2 w_2 + v_3 w_3 + v_4 w_4$$

Własność iloczynu skalarnego
znormalizowanych wektorów

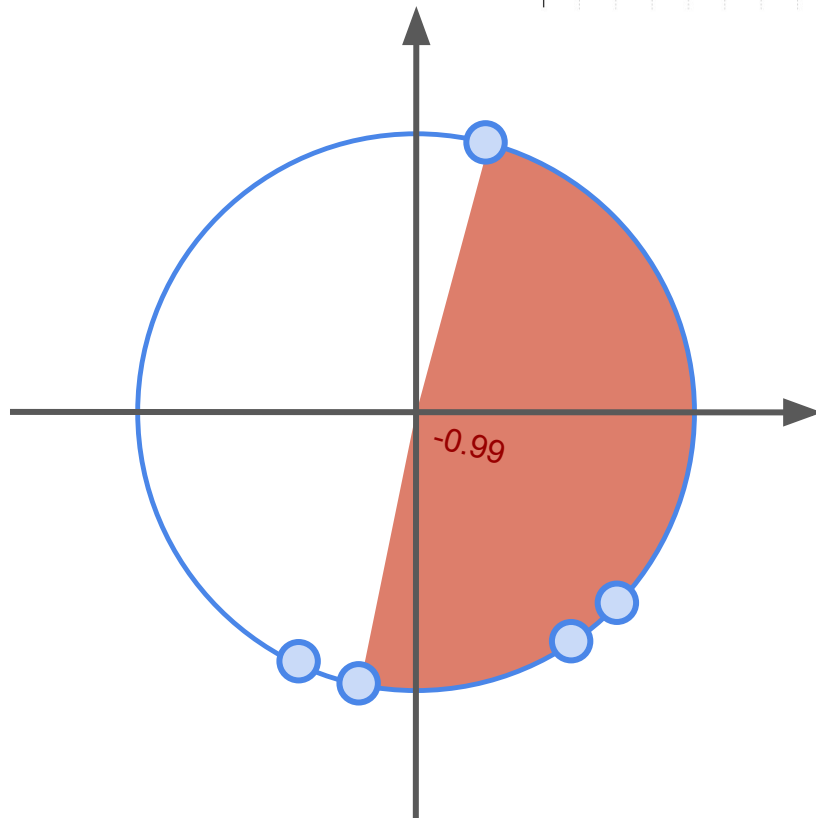
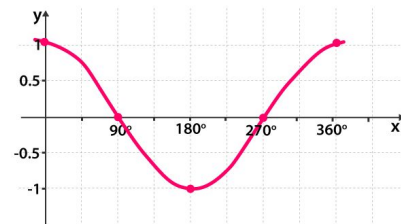
$$\mathbf{v} \cdot \mathbf{w} = \|\mathbf{v}\| \|\mathbf{w}\| \cos(\theta)$$

gdzie:

$\|\mathbf{v}\|$, $\|\mathbf{w}\|$ – długości wektorów

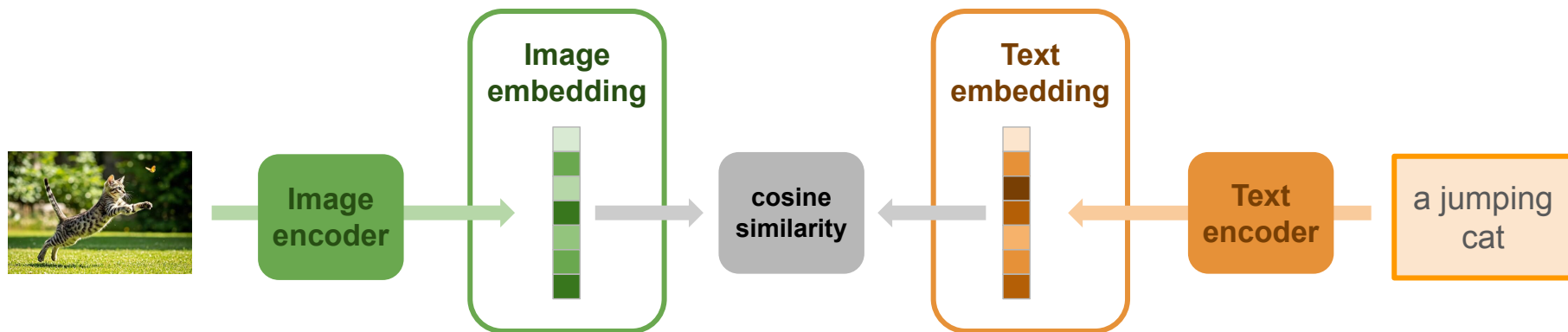
θ – kąt między nimi

Komentarz:
Dla kąta bliskiemu
półpełnemu cosinus jest
bliski wartości -1.



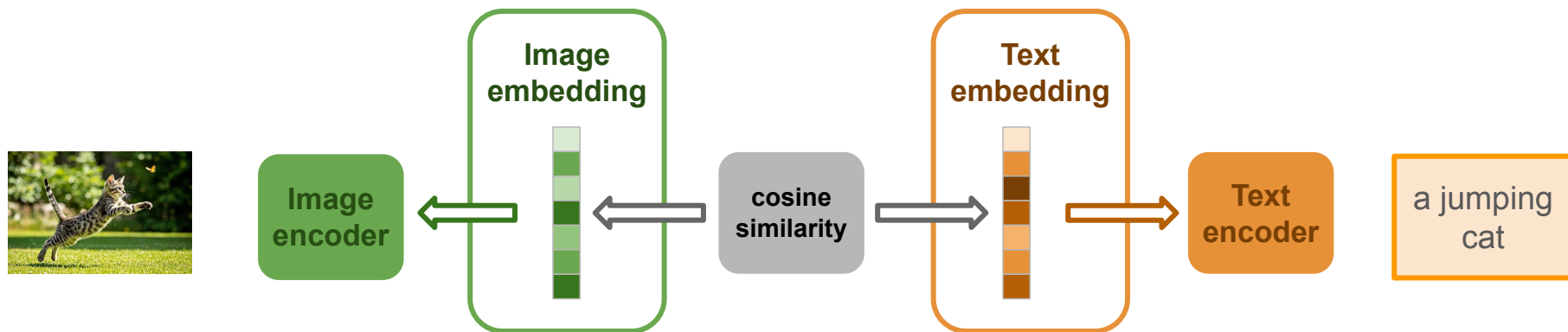
Integracja obrazu i tekstu

*Komentarz:
Moglibyśmy trenować
enkodery na pozytywnych
parach.*



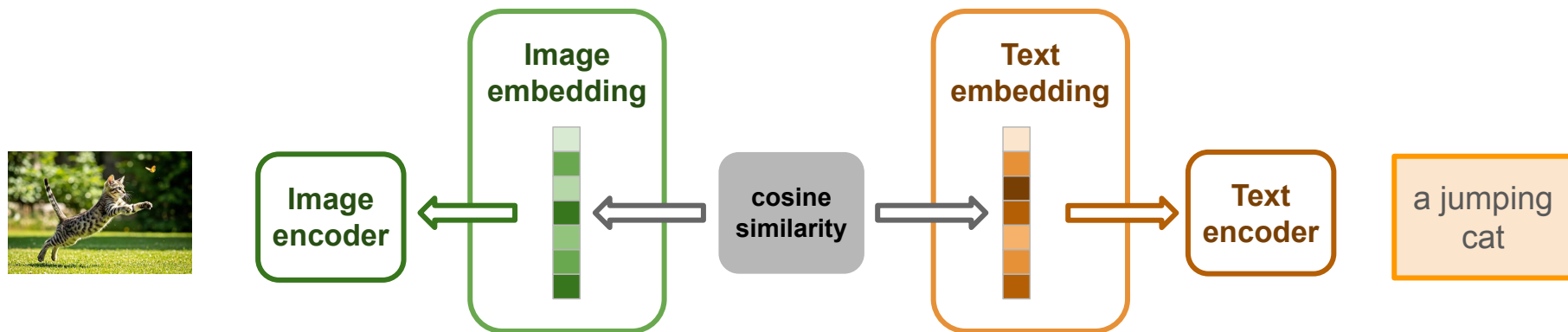
Integracja obrazu i tekstu

*Komentarz:
Propagacja wsteczna
działałaby na parametry
enkoderów.*

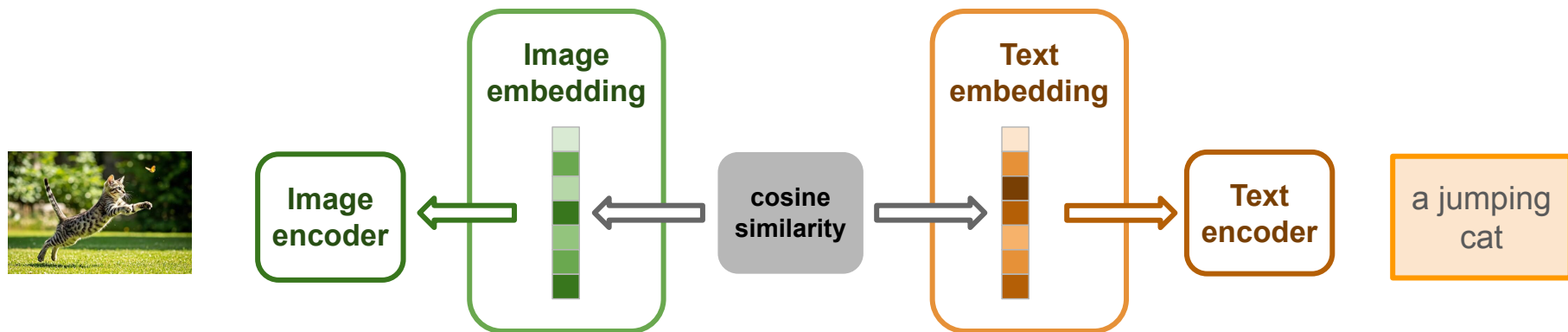
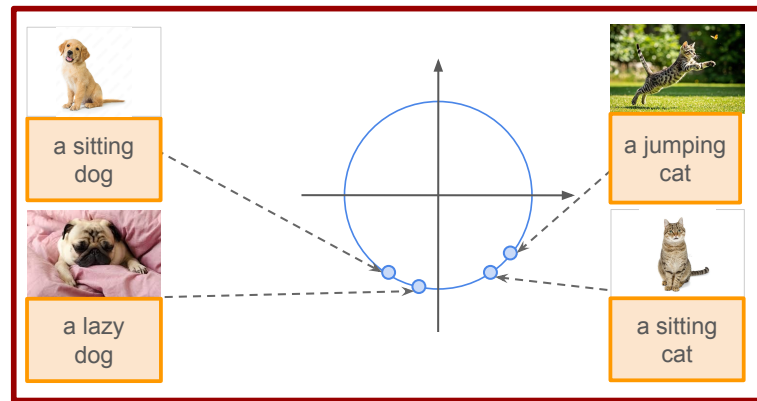


Integracja obrazu i tekstu

*Komentarz:
Propagacja wsteczna
działałaby na parametry
enkoderów. W ten sposób
moglibyśmy je trenować.*

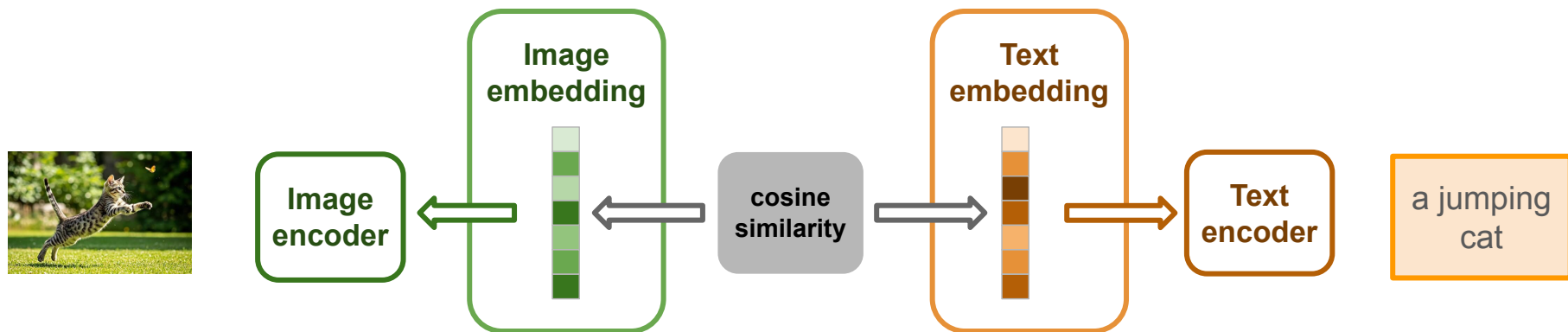
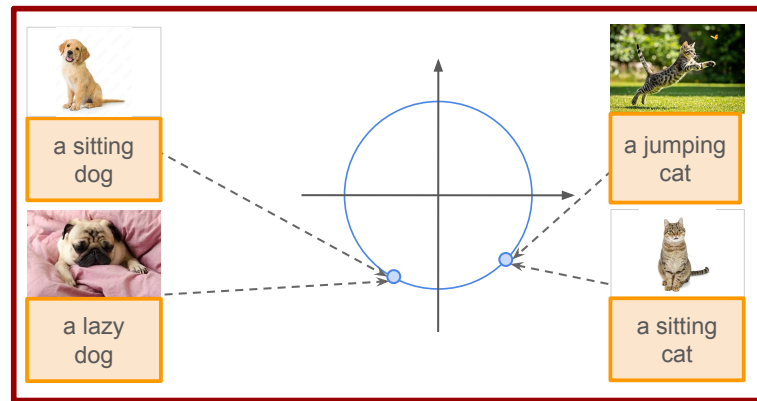


Integracja obrazu i tekstu



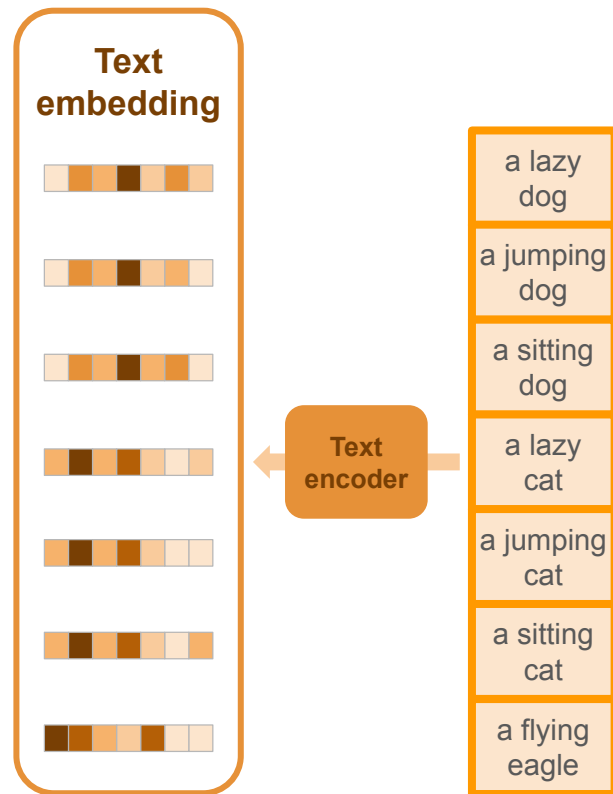
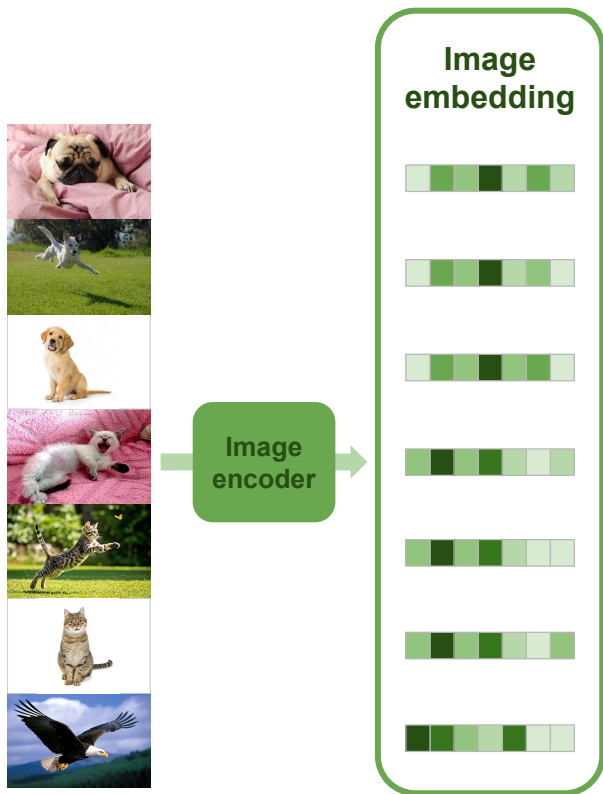
Integracja obrazu i tekstu

Tylko pozytywne pary → Brak separacji semantycznej
Potrzebujemy też negatywnych par



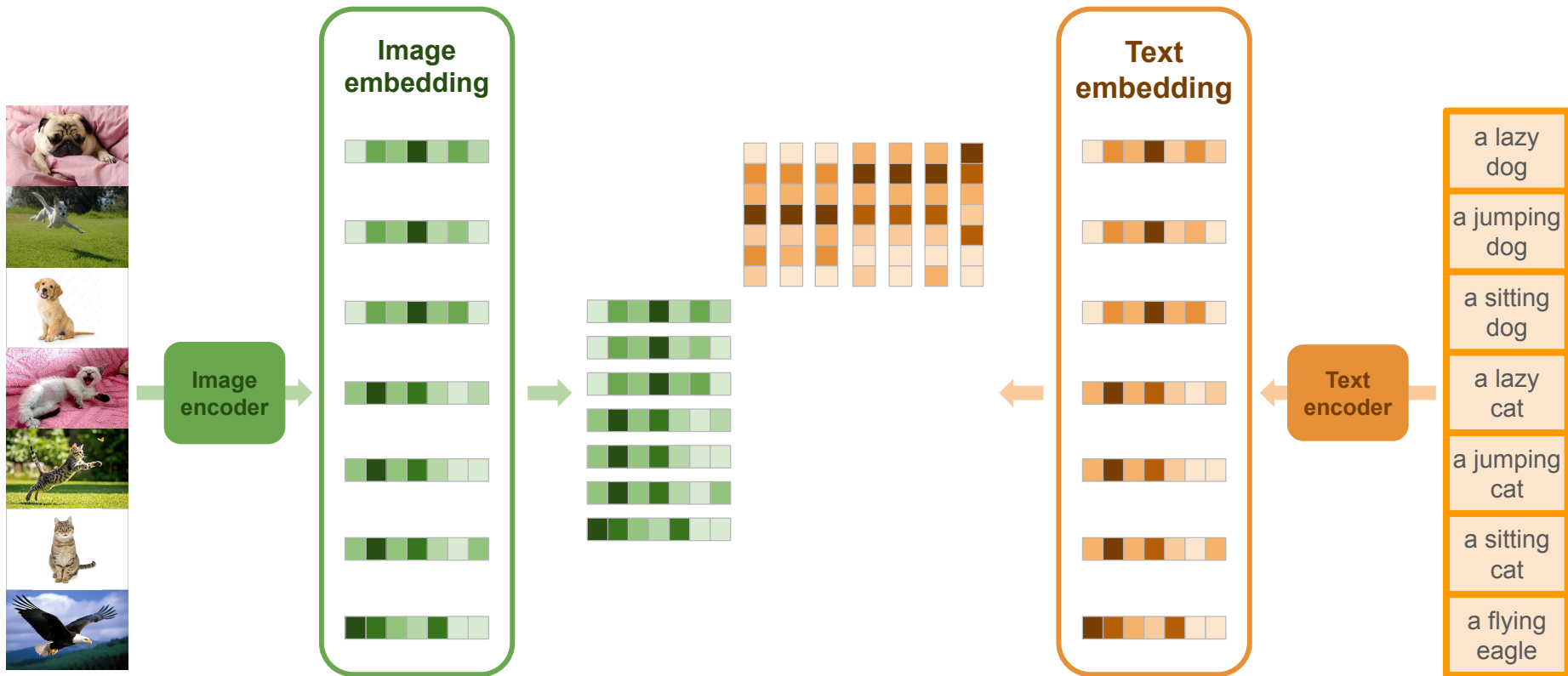
Integracja obrazu i tekstu

*Komentarz:
Weźmy serie obrazów i tekstów, tak
aby móc trenować także na
negatywnych parach.*



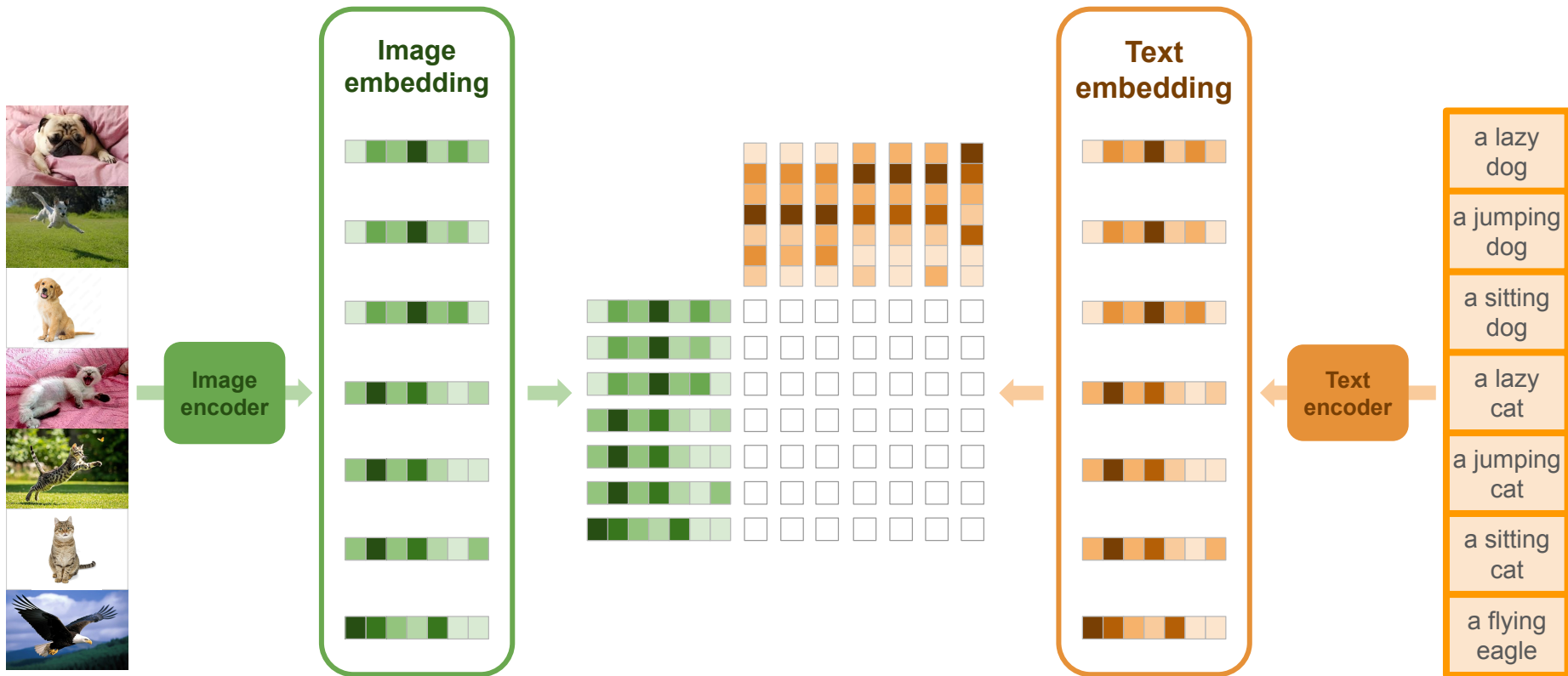
Integracja obrazu i tekstu

*Komentarz:
Dla każdego obrazu i dla każdego tekstu mamy embedding. Embeddingi obrazkowe tworzą jedną macierz, a embeddingi tekstowe - drugą.*



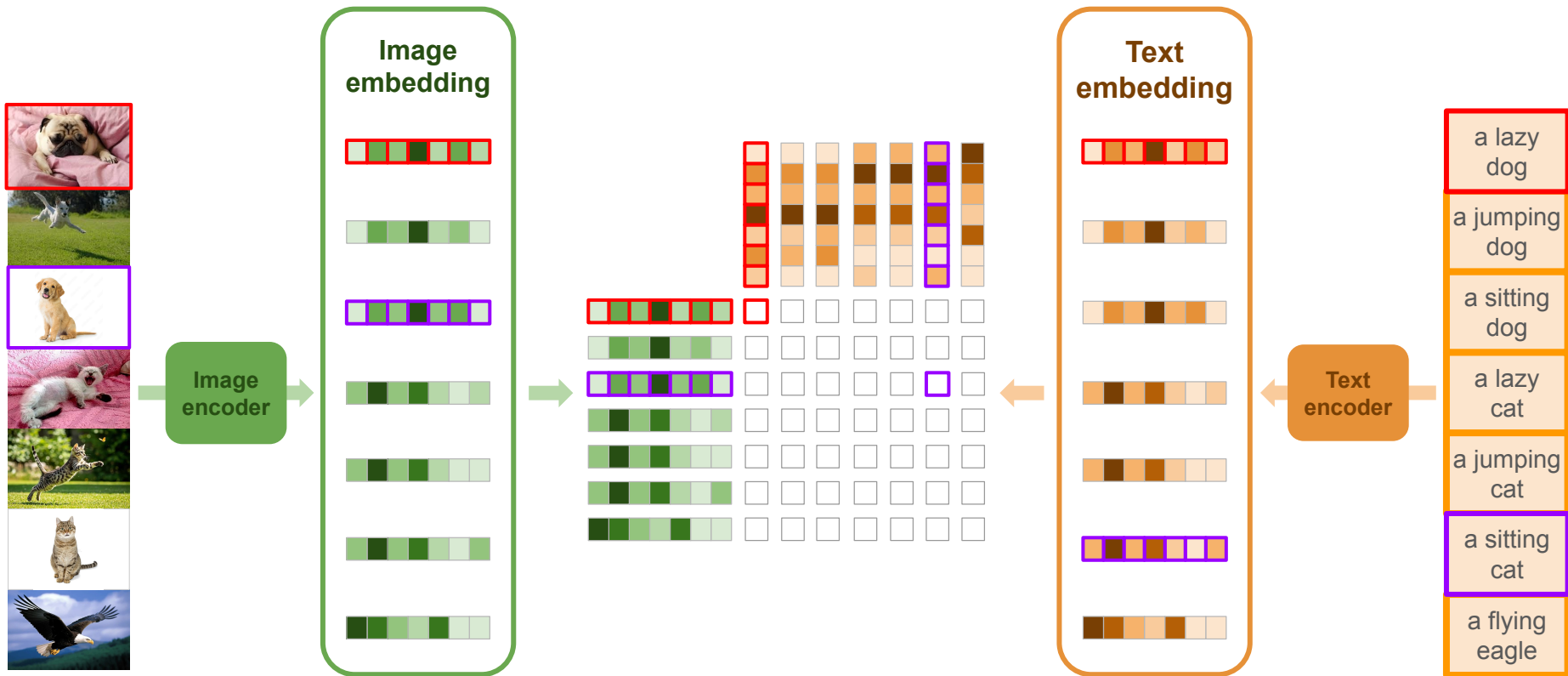
Integracja obrazu i tekstu

Komentarz:
Dla każdej pary embeddingów obrazkowego i tekstowego liczymy iloczyn skalarny na odpowiadającym im znormalizowanych wektorach.

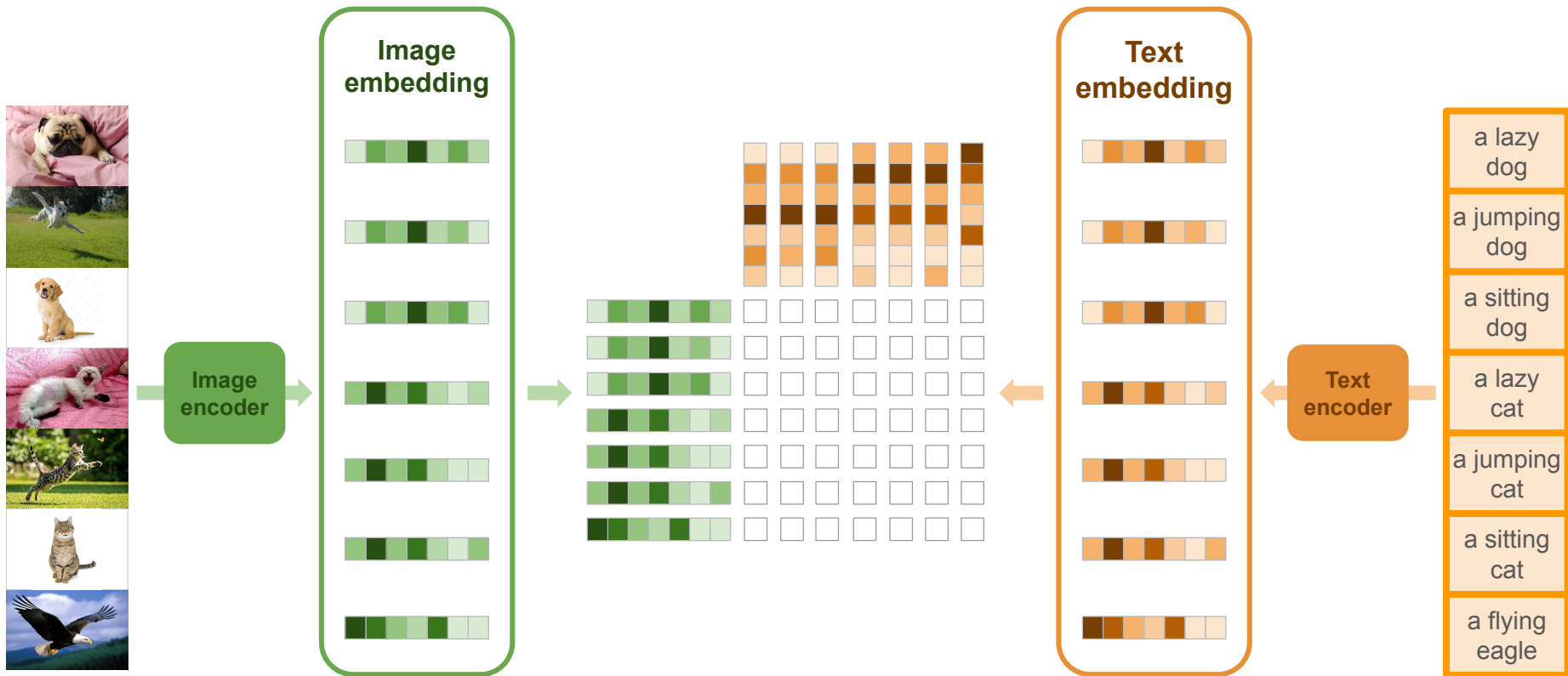


Integracja obrazu i tekstu

Komentarz:
Mamy zarówno pary pozytywne (czerwony), jak i negatywne (fioletowy).

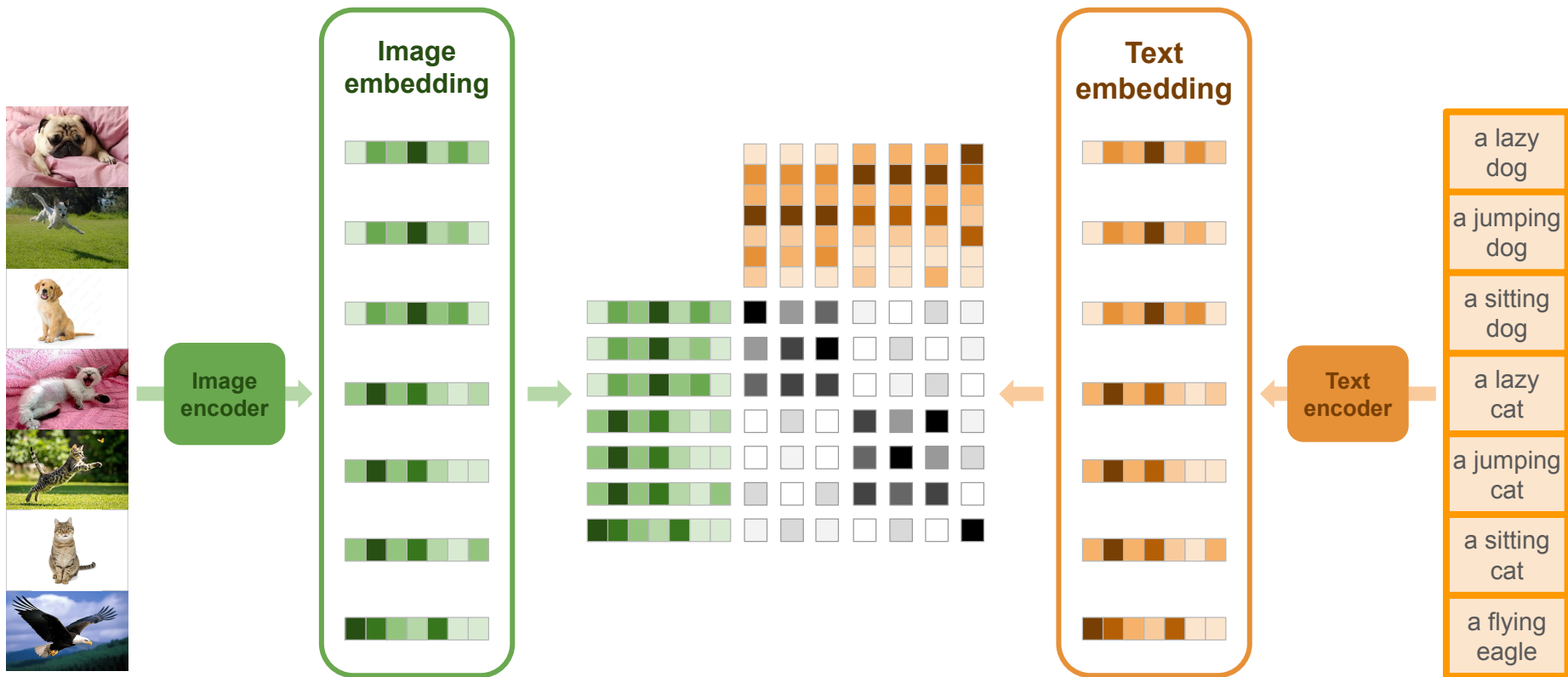


Integracja obrazu i tekstu



Integracja obrazu i tekstu

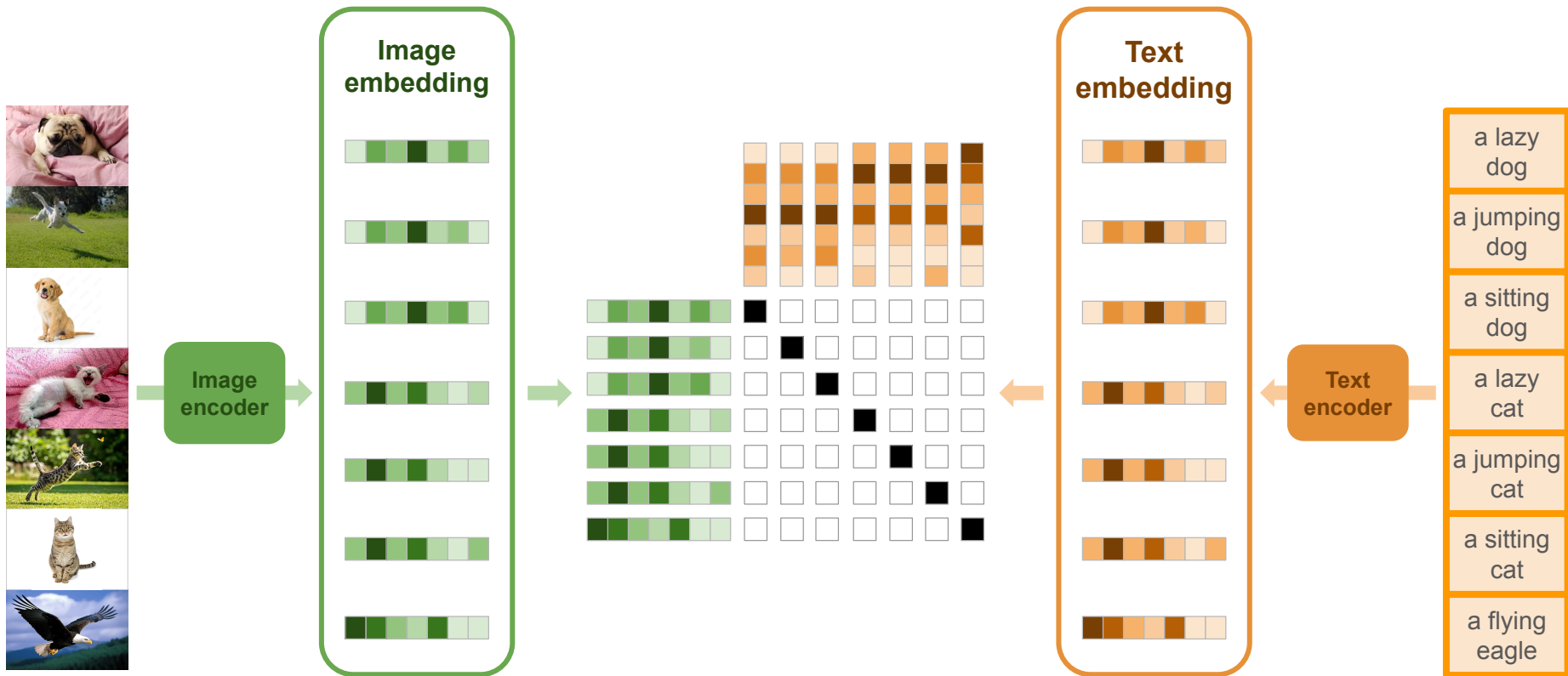
*Komentarz:
Otrzymujemy iloczyn skalarny dla
każdej pary.*



Integracja obrazu i tekstu

Komentarz:

Chcielibyśmy, aby na przekątnej były 1 (idealne dopasowanie), a w innych miejscach -1 (niedopasowanie). Wystarczy policzyć loss między tą macierzą, a macierzą iloczynów skalarnych.



Integracja obrazu i tekstu

*Komentarz:
Loss będzie składał się z dwóch
elementów:
Image-to-Text oraz Text-to-Image.*

a lazy dog	a jumping dog	a sitting dog	a lazy cat	a jumping cat	a sitting cat	a flying eagle
---------------	------------------	------------------	---------------	------------------	------------------	-------------------

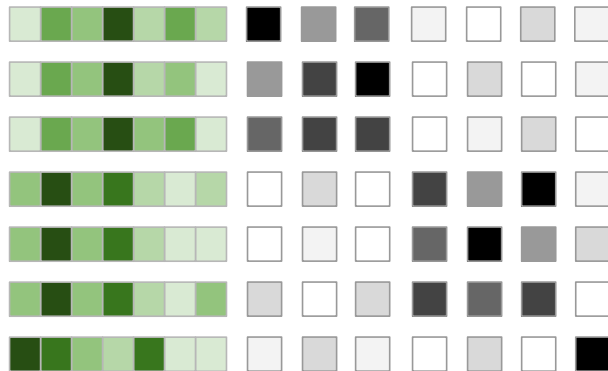
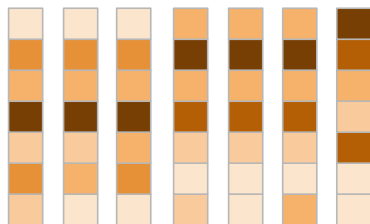


Image-to-Text Loss

Integracja obrazu i tekstu

*Komentarz:
logits to wartości iloczynu skalarnego
dla każdej pary.*

a lazy
dog
a jumping
dog
a sitting
dog
a lazy
cat
a jumping
cat
a sitting
cat
a flying
eagle

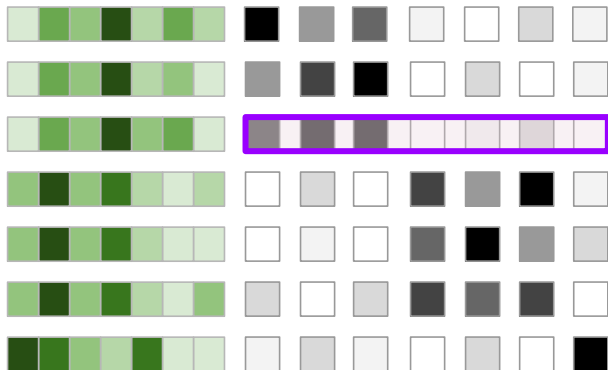
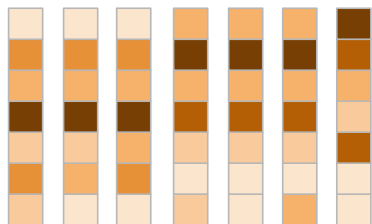


Image-to-Text Loss

- Softmax wierszami

$$p_{ij} = \frac{\exp(\text{logits}_{ij})}{\sum_{k=1}^N \exp(\text{logits}_{ik})}$$

Integracja obrazu i tekstu

a lazy dog	a jumping dog	a sitting dog	a lazy cat	a jumping cat	a sitting cat	a flying eagle
------------	---------------	---------------	------------	---------------	---------------	----------------

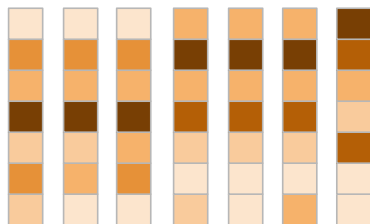


Image-to-Text Loss

- Softmax wierszami

$$p_{ij} = \frac{\exp(\text{logits}_{ij})}{\sum_{k=1}^N \exp(\text{logits}_{ik})}$$

Integracja obrazu i tekstu

a lazy dog
a jumping dog
a sitting dog
a lazy cat
a jumping cat
a sitting cat
a flying eagle

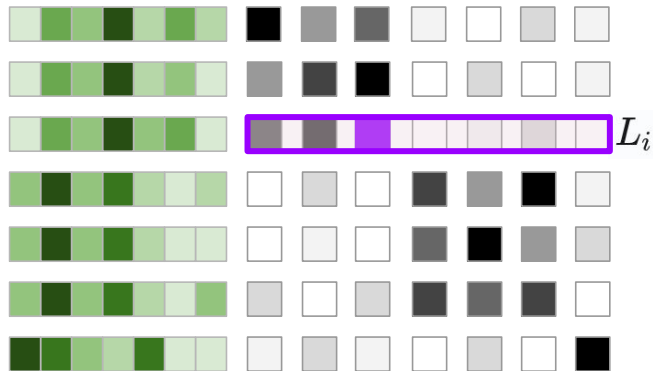
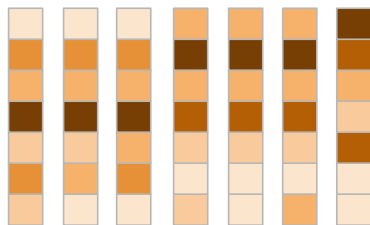


Image-to-Text Loss

- Softmax wierszami
$$p_{ij} = \frac{\exp(\text{logits}_{ij})}{\sum_{k=1}^N \exp(\text{logits}_{ik})}$$
- Cross-entropy względem poprawnego indeksu
$$L_i = -\log(p_{ii})$$

Integracja obrazu i tekstu

a lazy dog
a jumping dog
a sitting dog
a lazy cat
a jumping cat
a sitting cat
a flying eagle

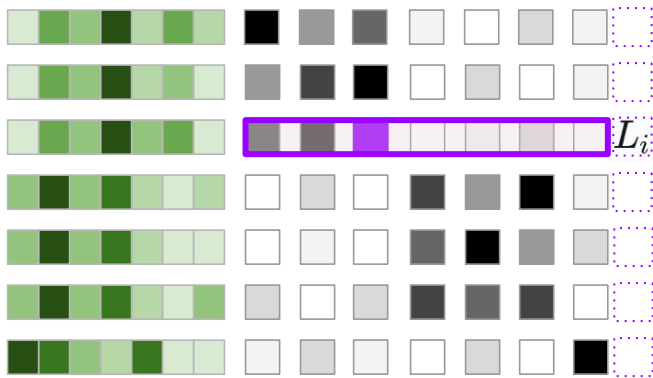
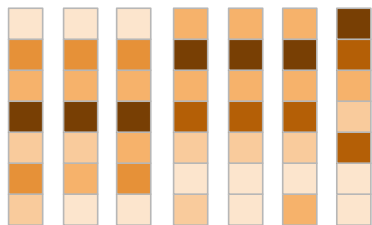
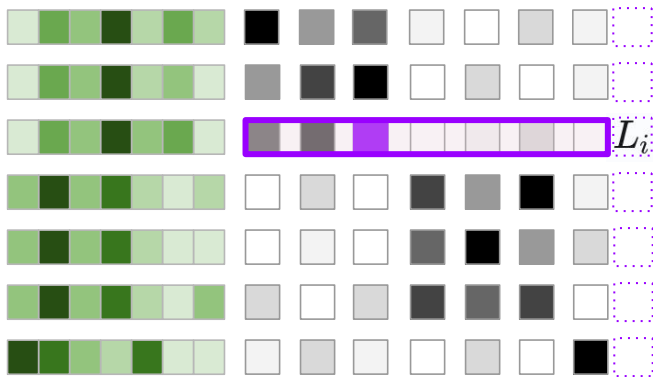
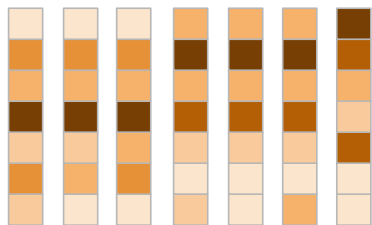


Image-to-Text Loss

- Softmax wierszami
$$p_{ij} = \frac{\exp(\text{logits}_{ij})}{\sum_{k=1}^N \exp(\text{logits}_{ik})}$$
- Cross-entropy względem poprawnego indeksu
$$L_i = -\log(p_{ii})$$
- Uśrednienie po batchu
$$\mathcal{L}_{i2t} = \frac{1}{N} \sum_{i=1}^N L_i$$

Integracja obrazu i tekstu

a lazy dog
a jumping dog
a sitting dog
a lazy cat
a jumping cat
a sitting cat
a flying eagle



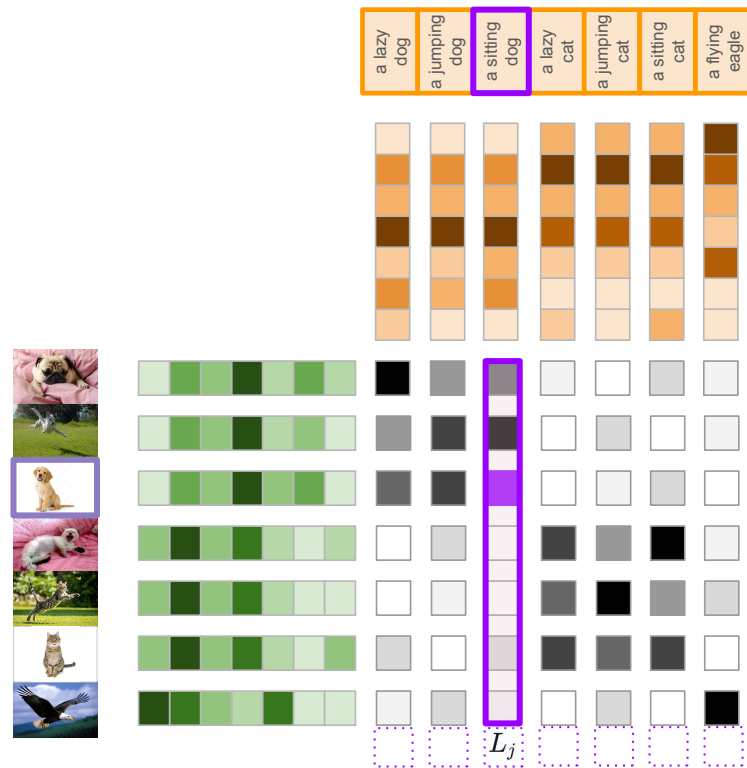
Komentarz:

Dodajemy wyuczalny parametr α . Parametr ten będzie zmieniał się w trakcie treningu. Na początku będzie mniejszy, dzięki czemu softmax będzie mniej ostry, a prawdopodobieństwo będzie rozkładało się bardziej równomiernie. Z czasem będzie rósł, a prawdopodobieństwo wyliczone za pomocą funkcji softmax będzie bardziej skupione, co odpowiada “większej pewności” dopasowań.

Image-to-Text Loss

- Skorzystanie z mnożnika $\text{logits}_{ij} = \alpha \cdot \text{cosine_similarity}_{ij}$
- Softmax wierszami
$$p_{ij} = \frac{\exp(\text{logits}_{ij})}{\sum_{k=1}^N \exp(\text{logits}_{ik})}$$
- Cross-entropy względem poprawnego indeksu
$$L_i = -\log(p_{ii})$$
- Uśrednienie po batchu
$$\mathcal{L}_{i2t} = \frac{1}{N} \sum_{i=1}^N L_i$$

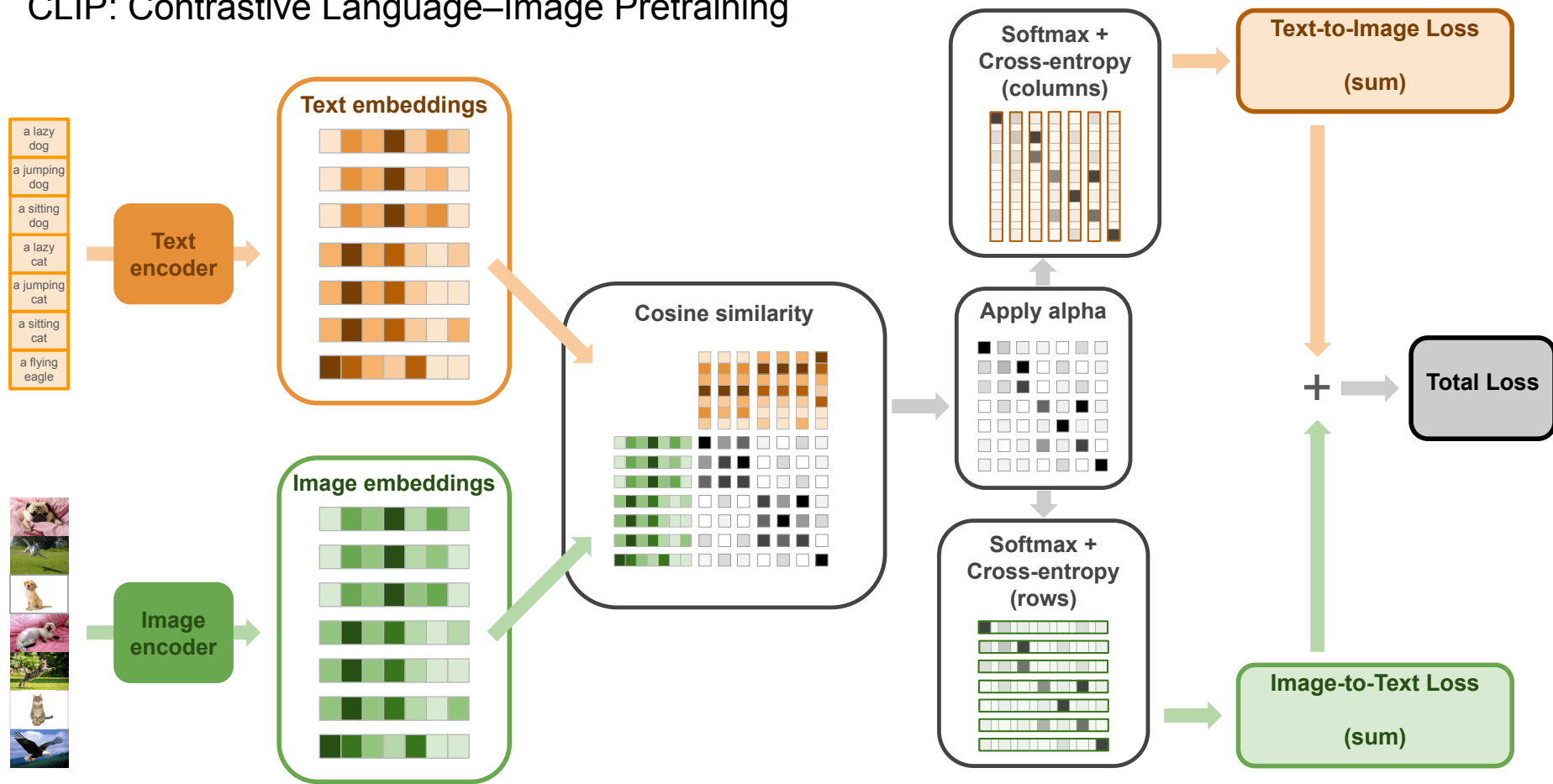
Integracja obrazu i tekstu



Text-to-Image Loss

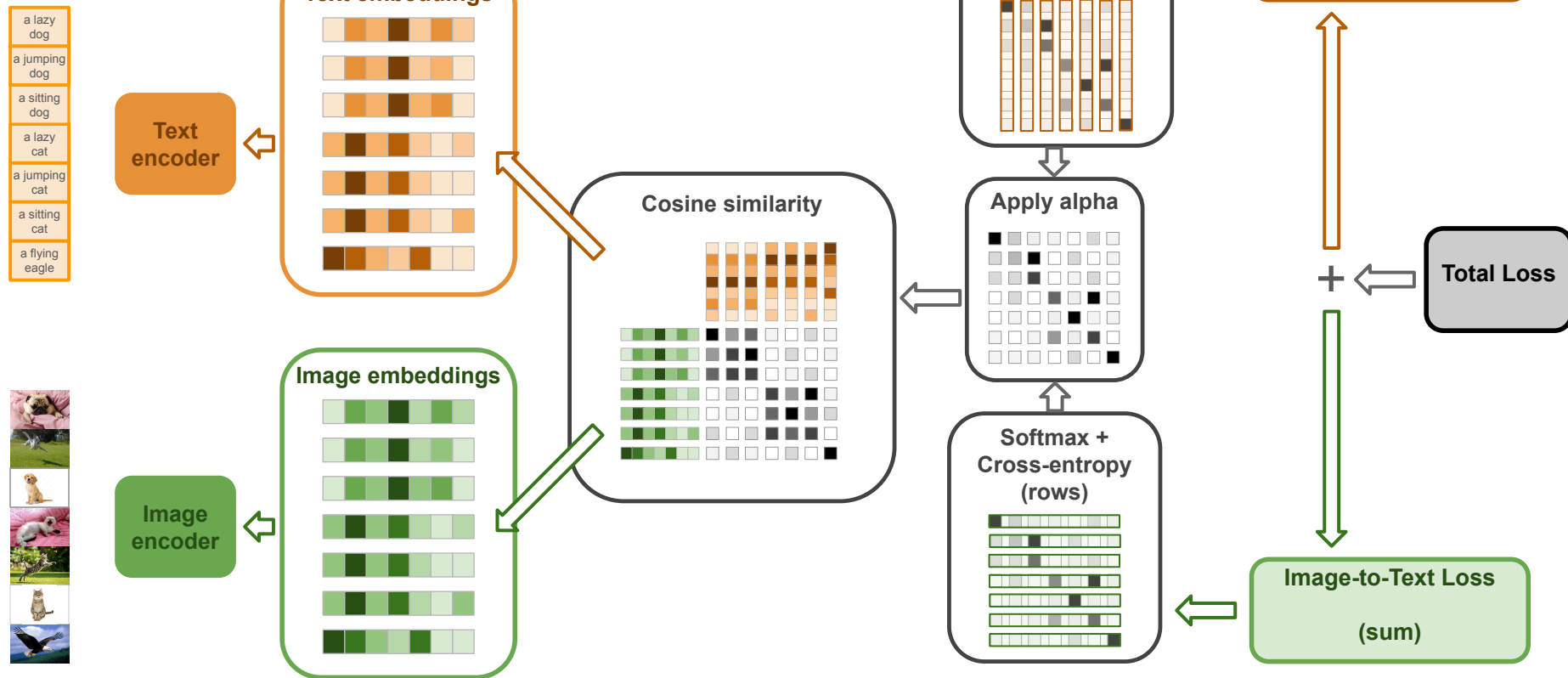
- Skorzystanie z mnożnika $\text{logits}_{ij} = \alpha \cdot \text{cosine_similarity}_{ij}$
- Softmax kolumnami $p_{ij} = \frac{\exp(\text{logits}_{ij})}{\sum_{k=1}^N \exp(\text{logits}_{kj})}$
- Cross-entropy względem poprawnego indeksu $L_j = -\log(p_{jj})$
- Uśrednienie po batchu $\mathcal{L}_{t2i} = \frac{1}{N} \sum_{j=1}^N L_j$

CLIP: Contrastive Language–Image Pretraining



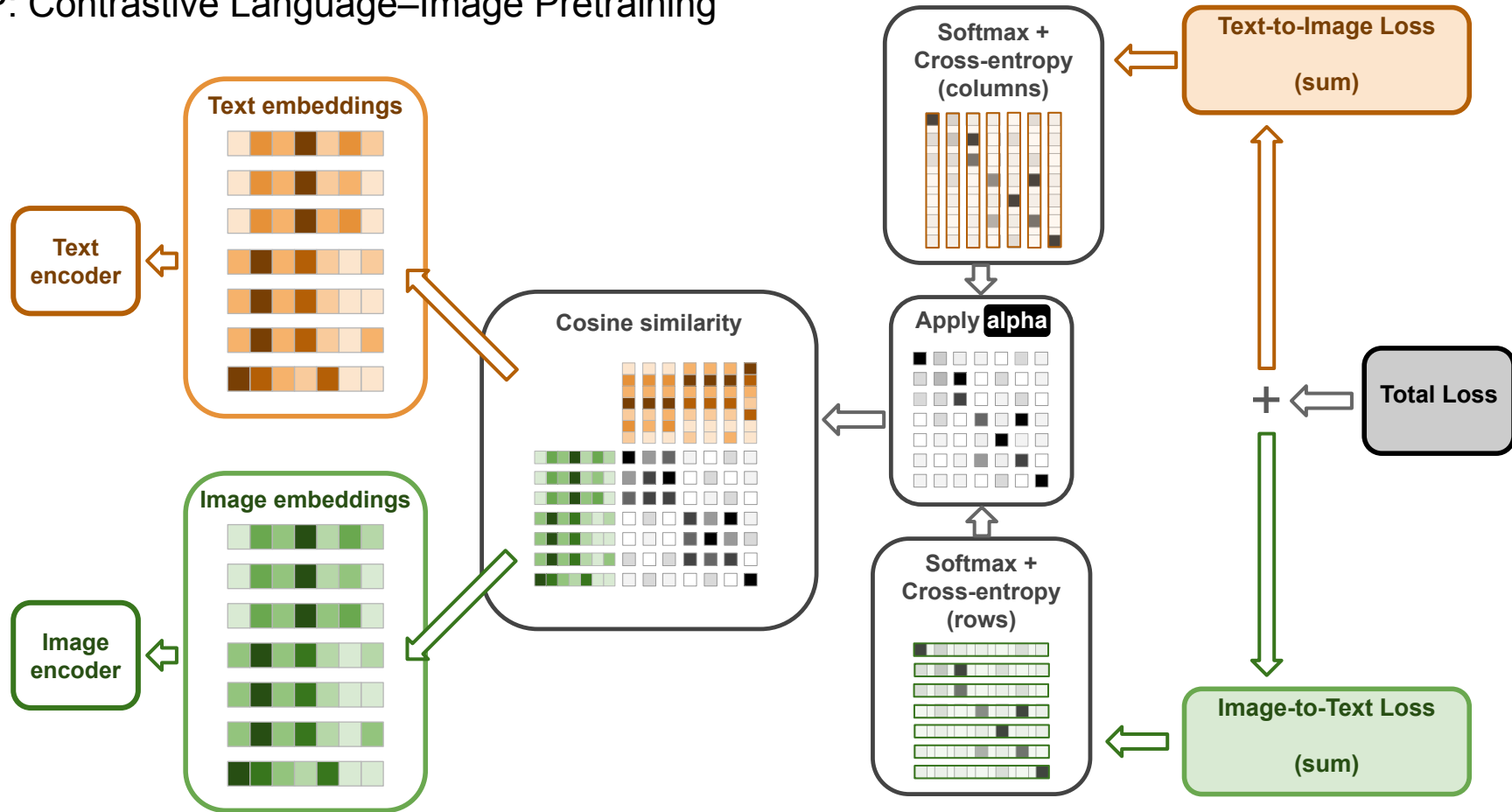
Komentarz: Propagacja wsteczna.

CLIP: Contrastive Language–Image Pretraining



CLIP: Contrastive Language–Image Pretraining

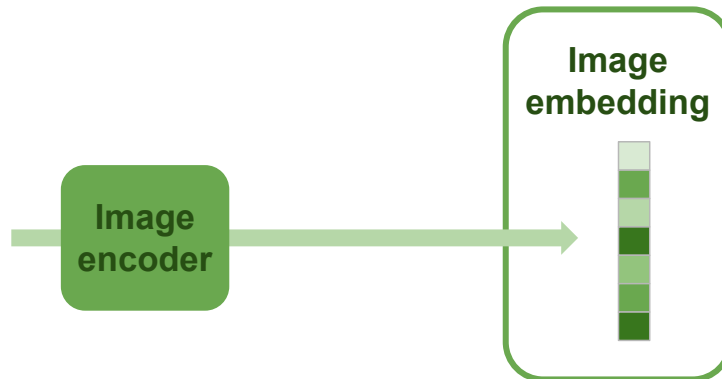
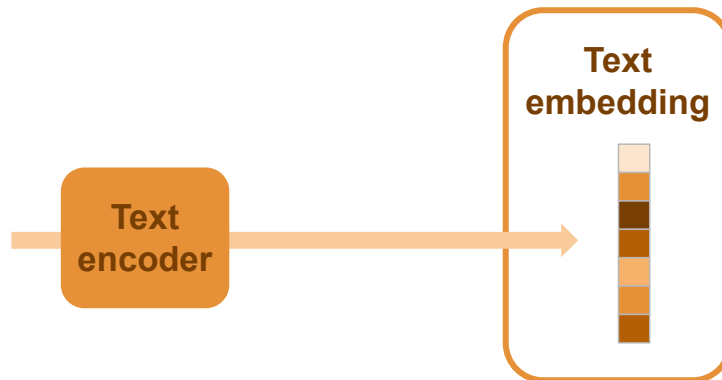
a lazy dog
a jumping dog
a sitting dog
a lazy cat
a jumping cat
a sitting cat
a flying eagle



Integracja obrazu i tekstu

*Komentarz:
Założyliśmy że embeddingi tekstowe i obrazkowe
mają ten sam wymiar. Tak nie musi być.*

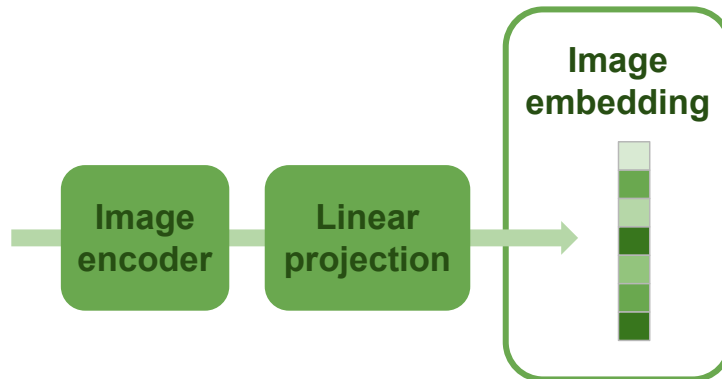
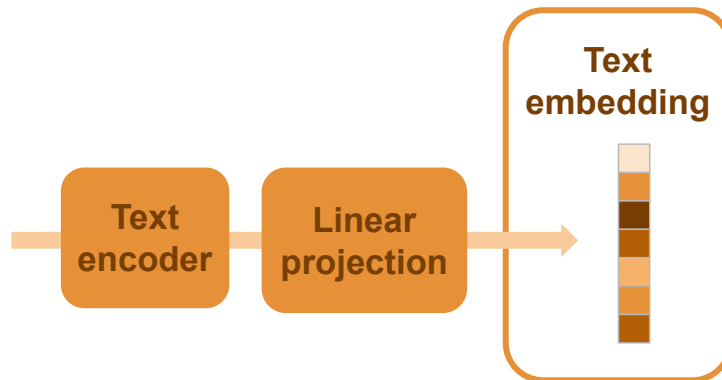
A photo of a brown tabby
cat jumping on grass.



Integracja obrazu i tekstu

*Komentarz:
Trzeba dołożyć projekcję liniową, która dopasuje
wymiarowość embeddingów.*

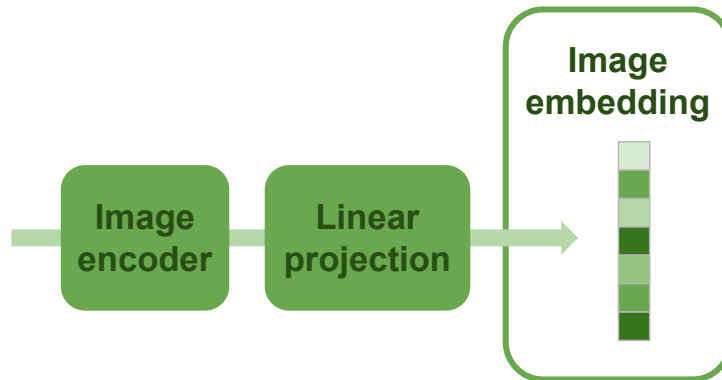
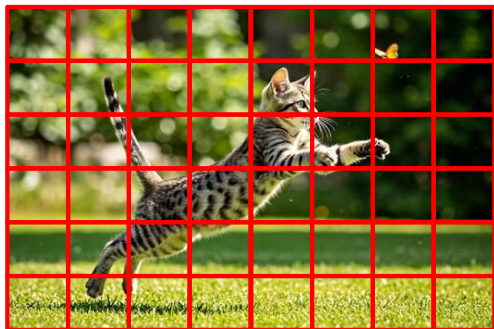
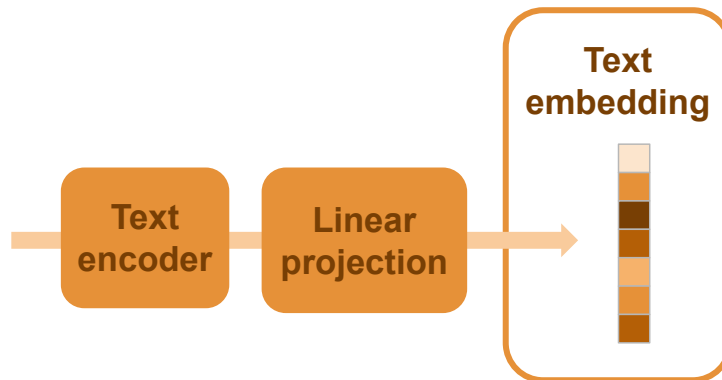
A photo of a brown tabby
cat jumping on grass.



Integracja obrazu i tekstu

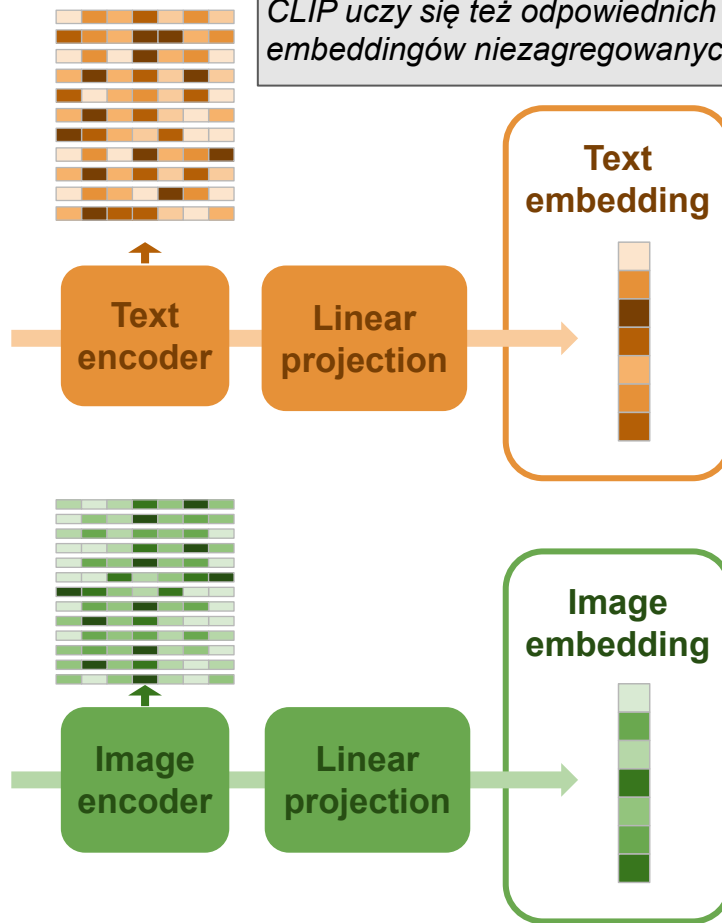
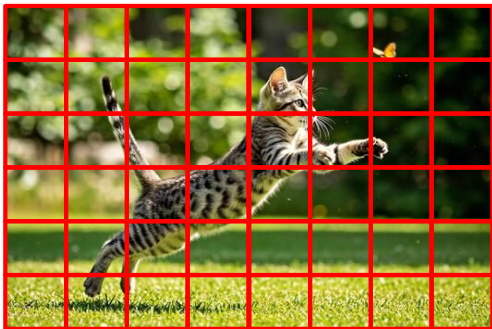
*Komentarz:
Przypomnienie skąd biorą się embeddingi
zagregowane.*

A photo of a brown tabby
cat jumping on grass.



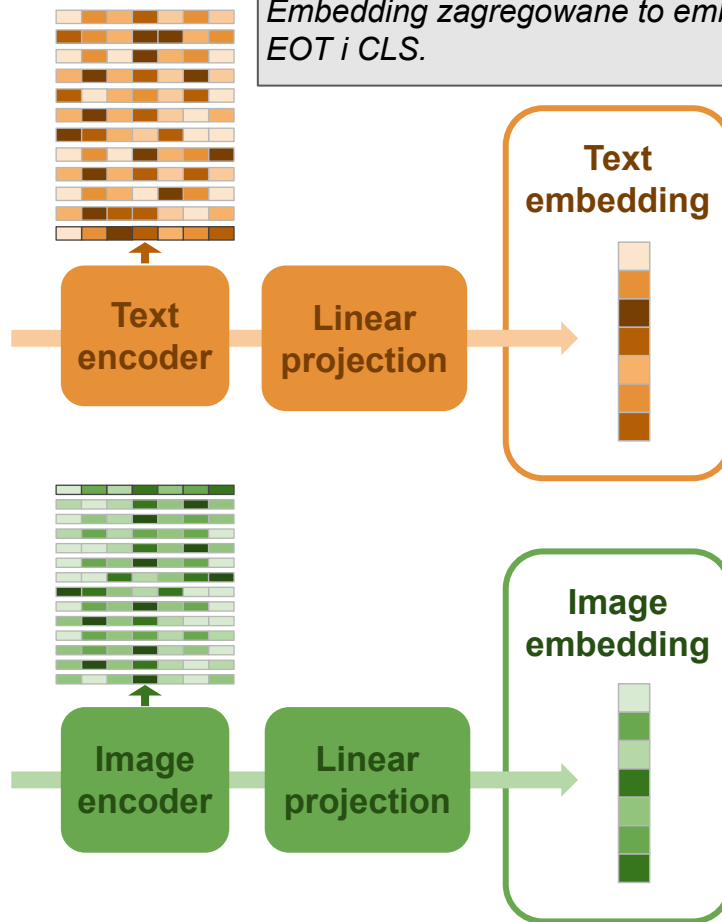
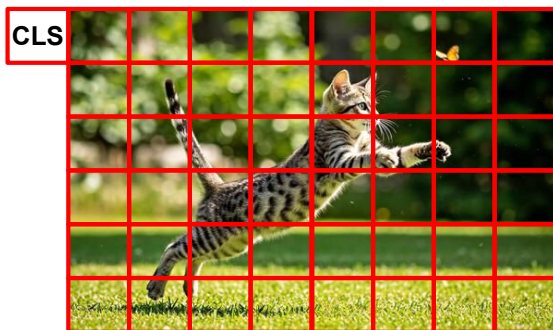
Integracja obrazu i tekstu

A photo of a brown tabby
cat jumping on grass.

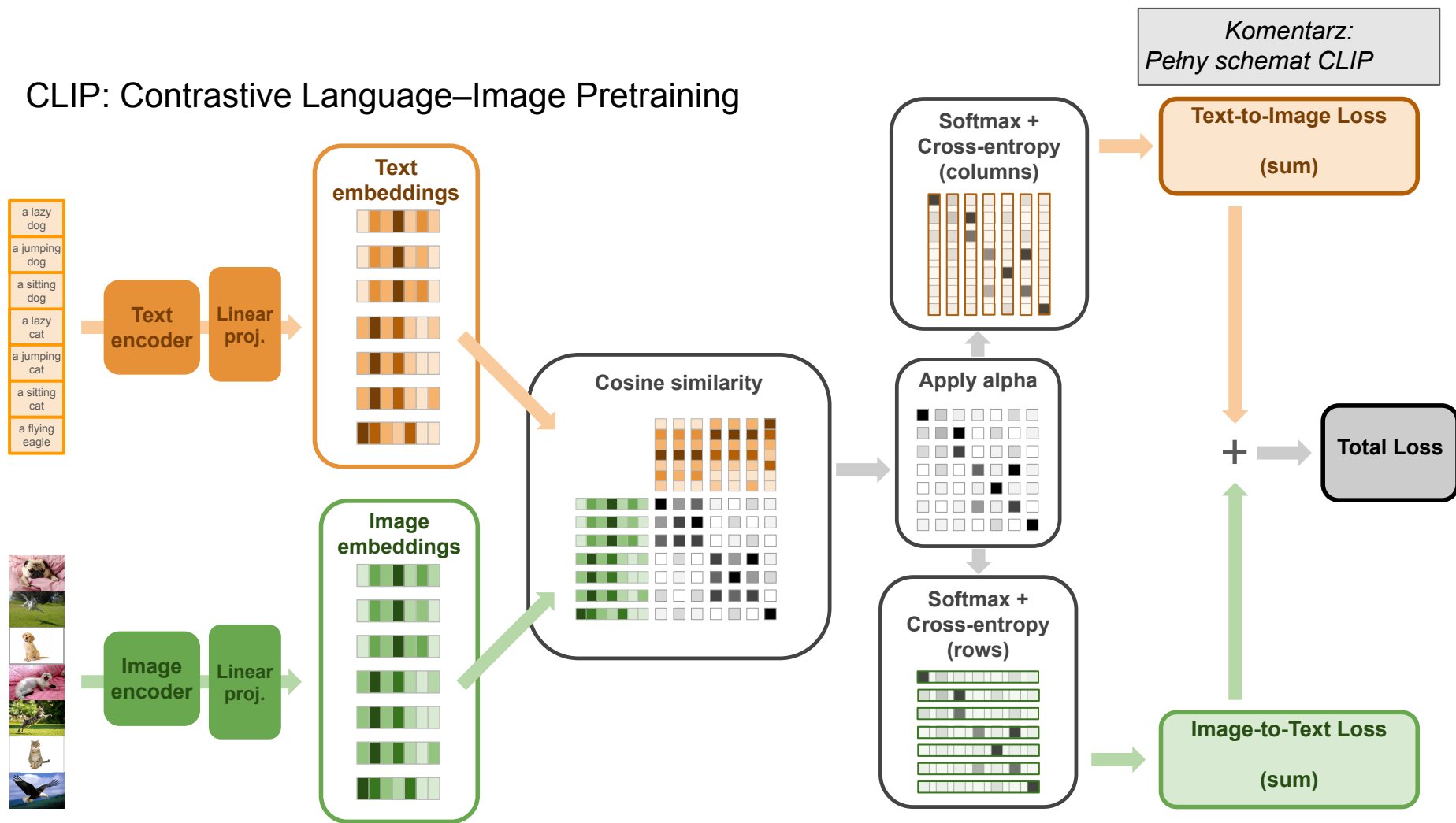


Integracja obrazu i tekstu

A photo of a brown tabby
cat jumping on grass. EOT



CLIP: Contrastive Language–Image Pretraining



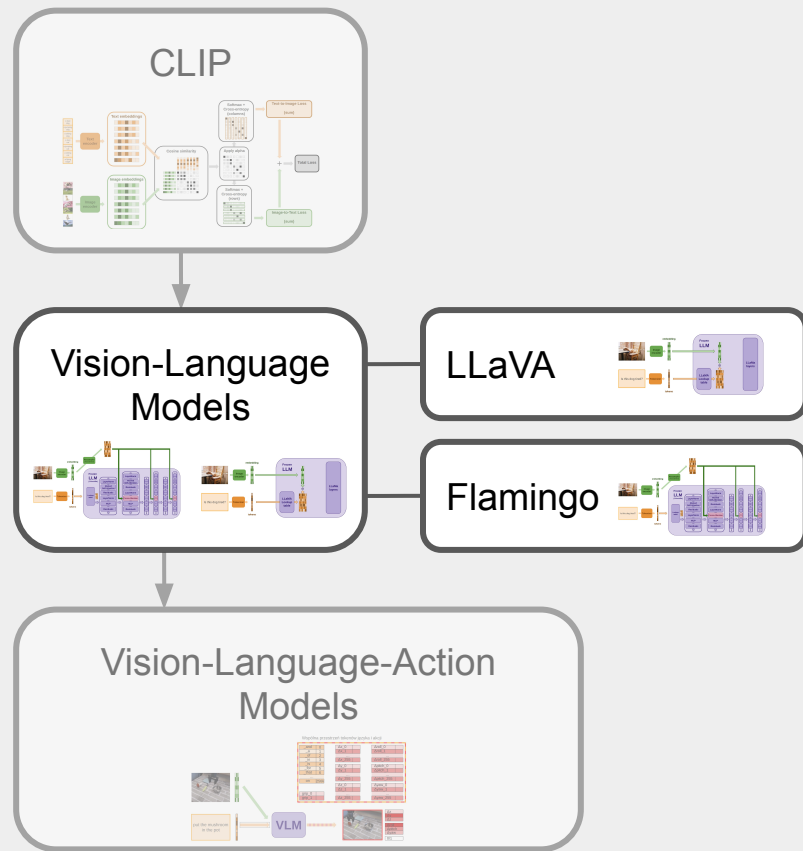
CLIP: Contrastive Language–Image Pretraining

Zastosowania

- Wyszukiwanie obrazów po opisie (Text-to-Image Search)
- Klasyfikacja obrazów bez trenowania (Zero-shot classification)
- Automatyczne tagowanie obrazów
- Wyszukiwanie podobnych obrazów (Image-to-Image Search)
- Moderacja treści
- Wsparcie generowania obrazów (np. DALL·E, Stable Diffusion)
- Klasteryzacja i analiza zbiorów obrazów

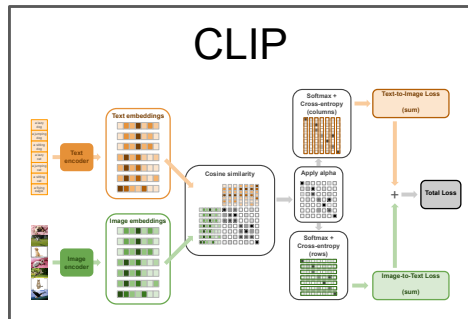
Vision-Language Models

Czyli rozumienie

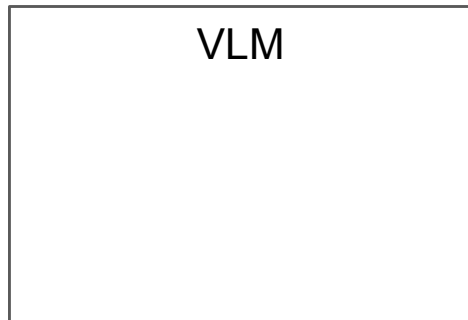


Vision-Language Models

Idea



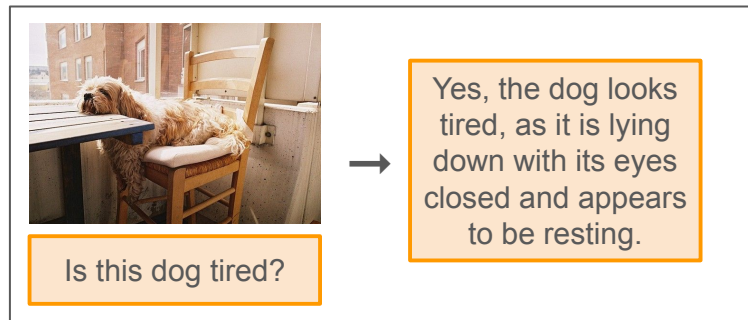
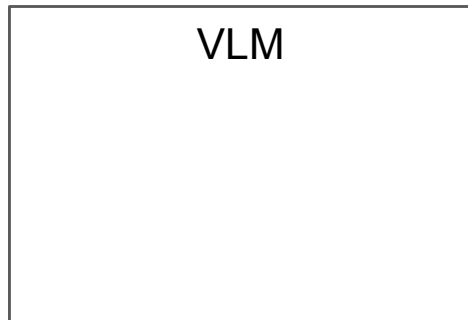
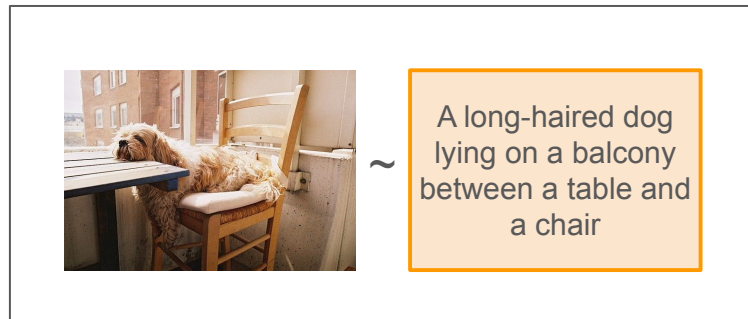
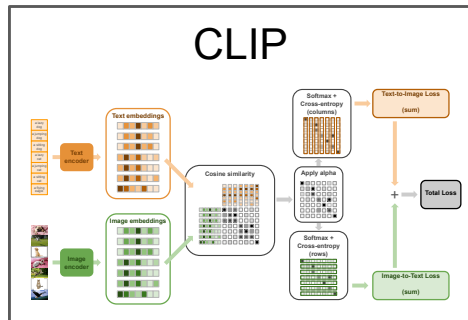
tekst ~ obraz



(tekst, obraz) → tekst

Vision-Language Models

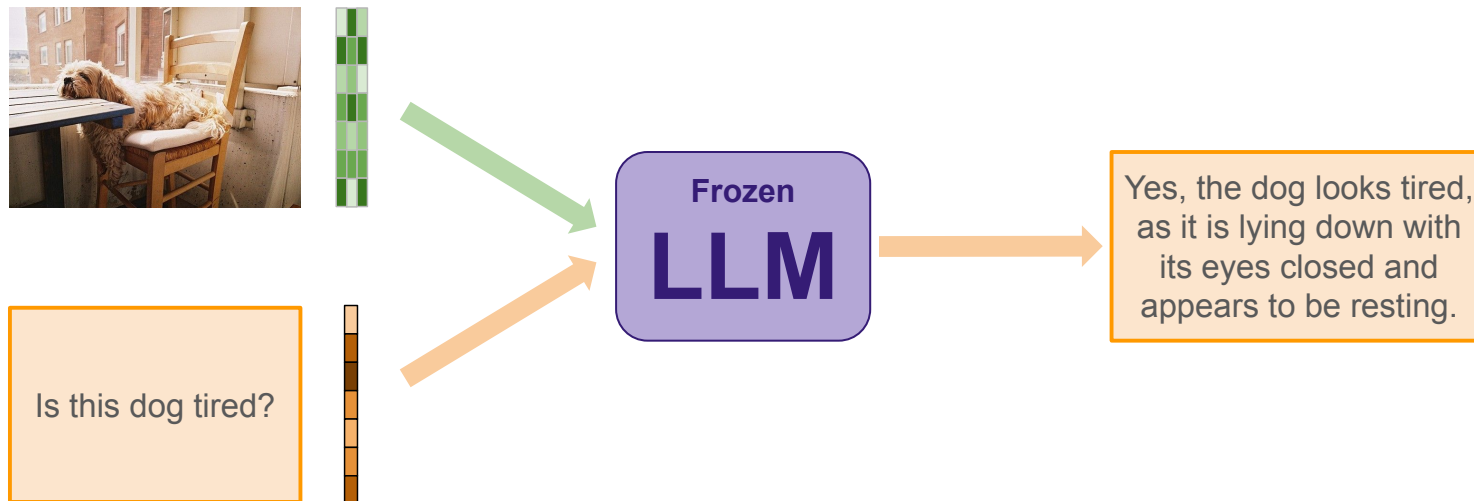
Idea



Vision-Language Models

Idea

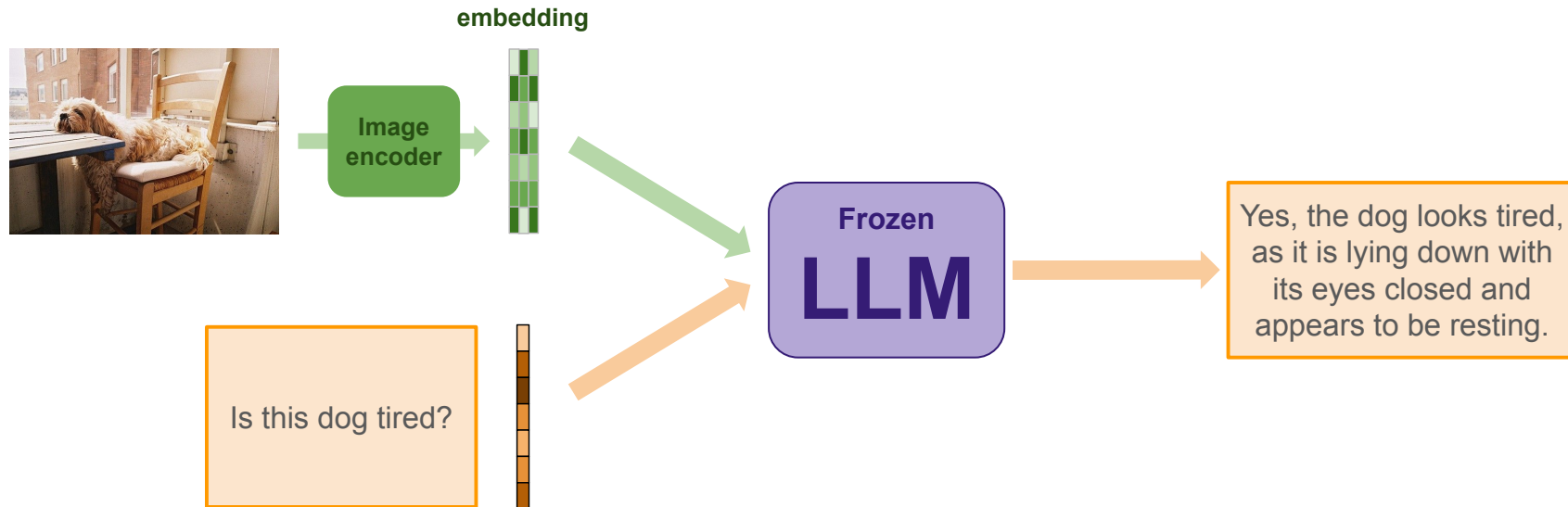
*Komentarz:
Idea najprostszych VLMów polega na
skorzystaniu z zamrożonego
wytrenowanego wcześniej LLM'a*



Vision-Language Models

Idea

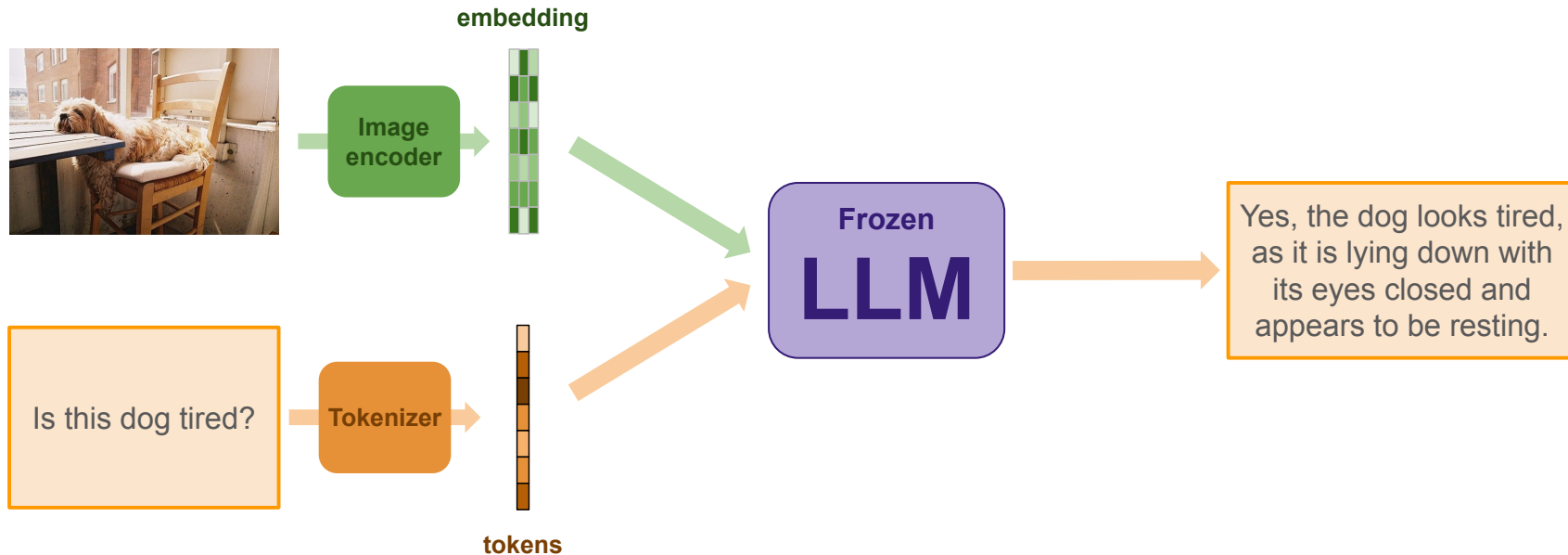
*Komentarz:
Reprezentacją obrazu są embeddingi
(niezagregowane).*



Vision-Language Models

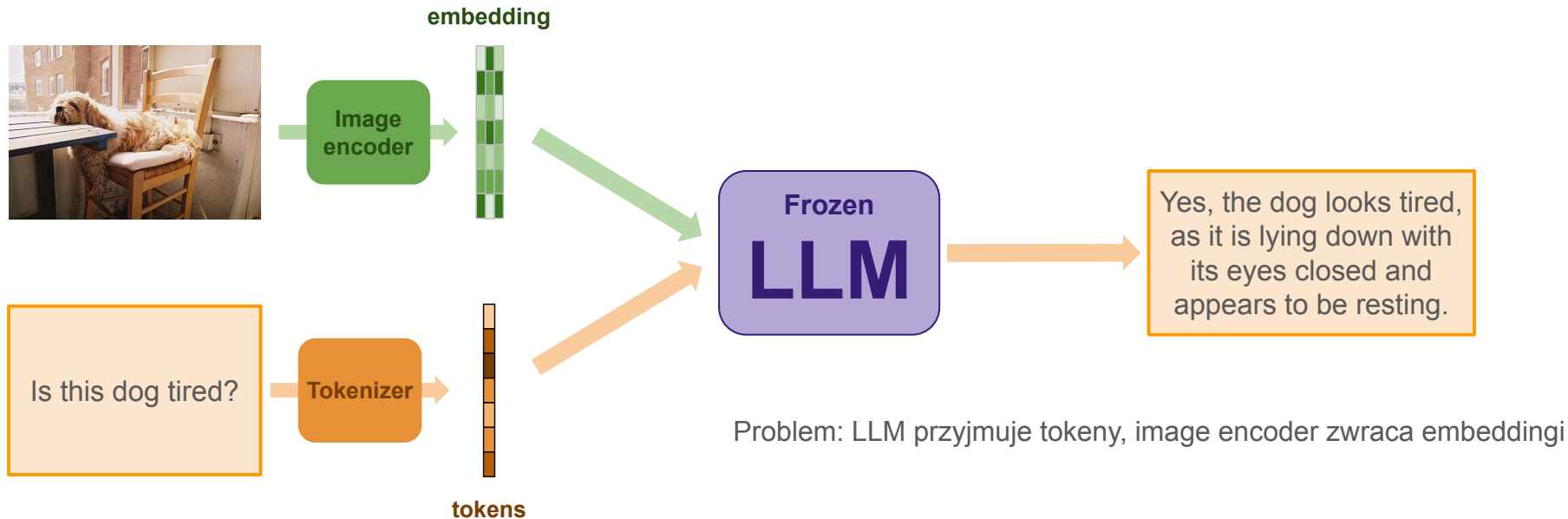
Idea

*Komentarz:
Reprezentację tekstu są tokeny.*



Vision-Language Models

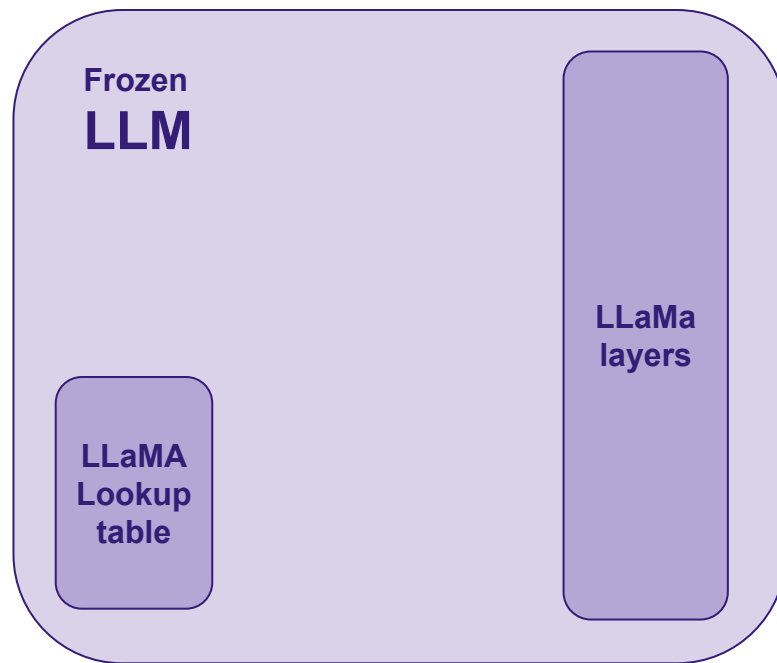
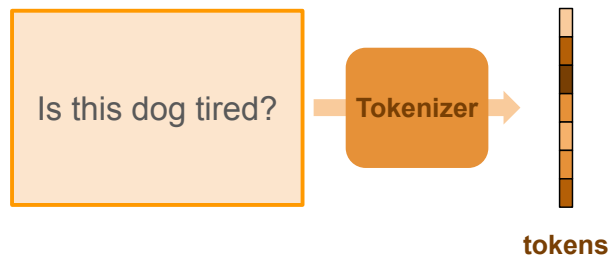
Idea



Vision-Language Models

LLaVA

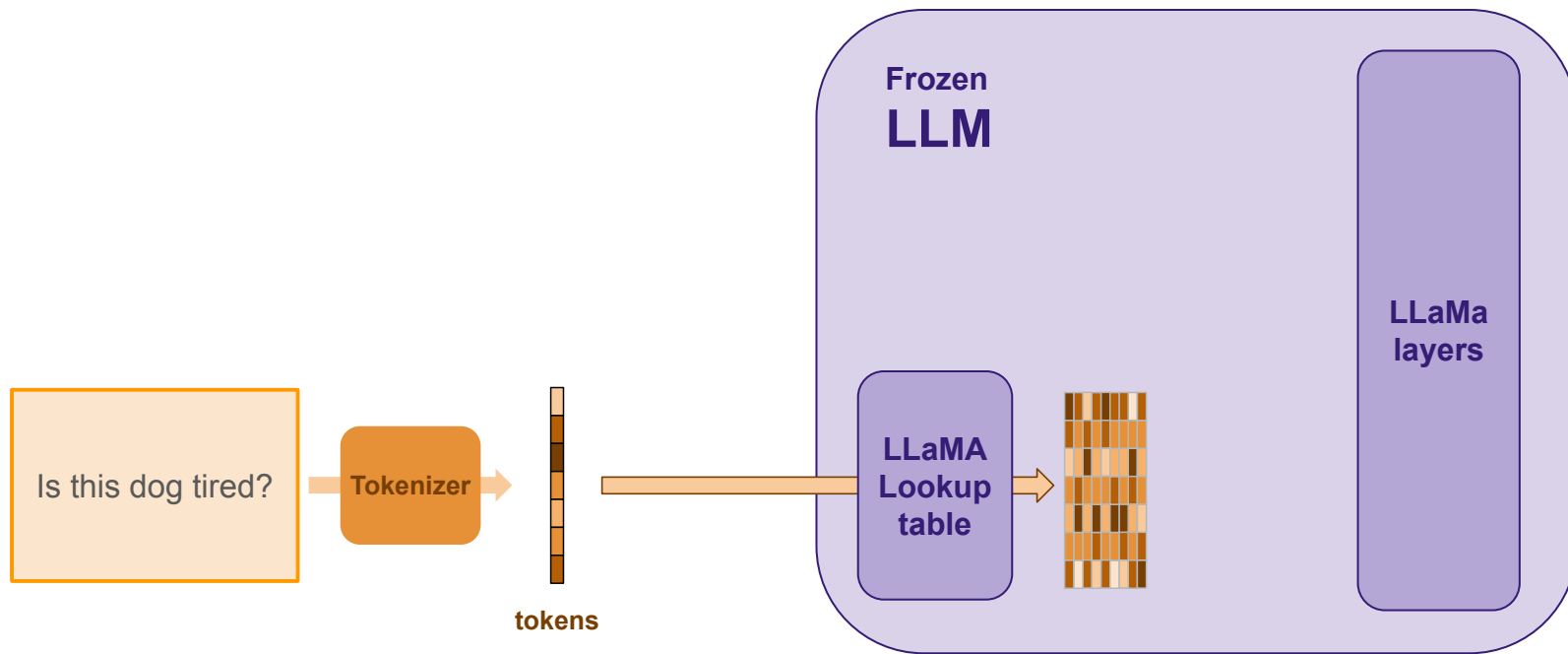
*Komentarz:
Standardowa inferencja w LLM.*



Vision-Language Models

LLaVA

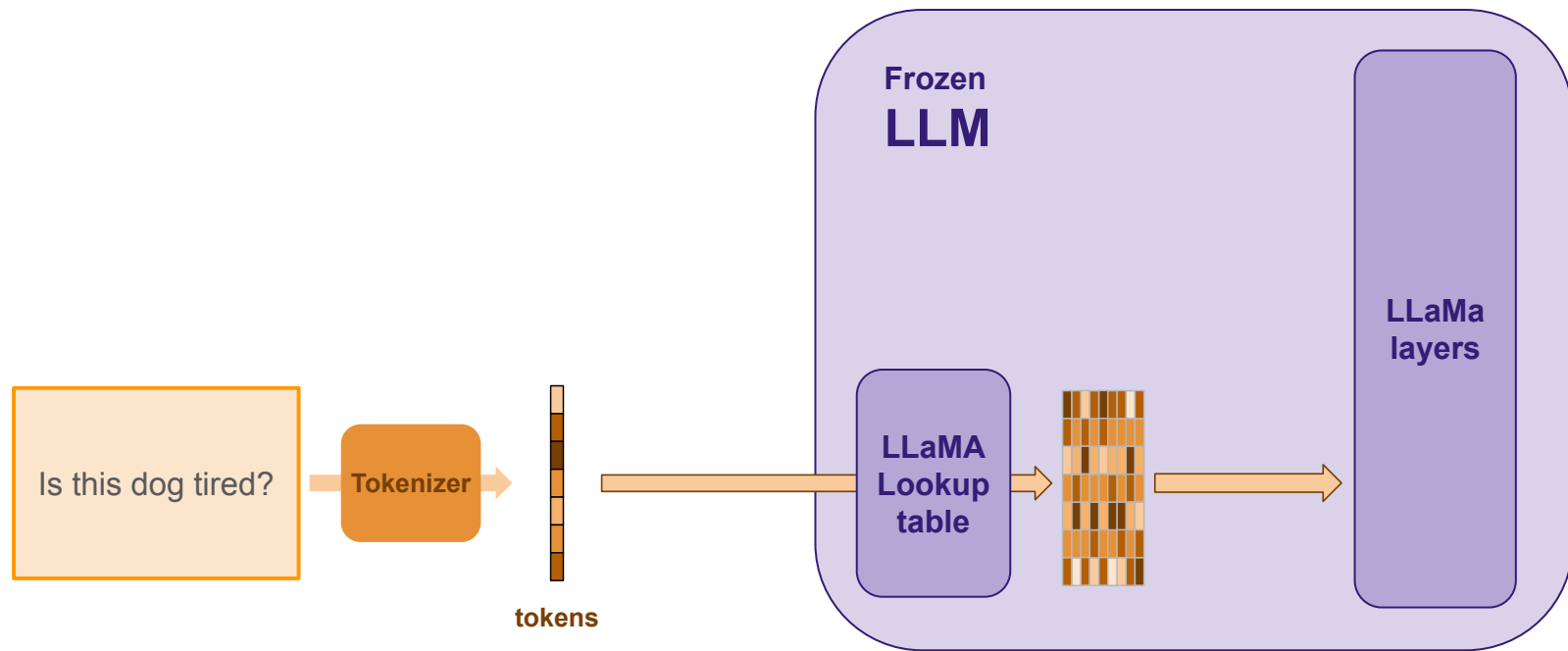
*Komentarz:
Standardowa inferencja w LLM.*



Vision-Language Models

LLaVA

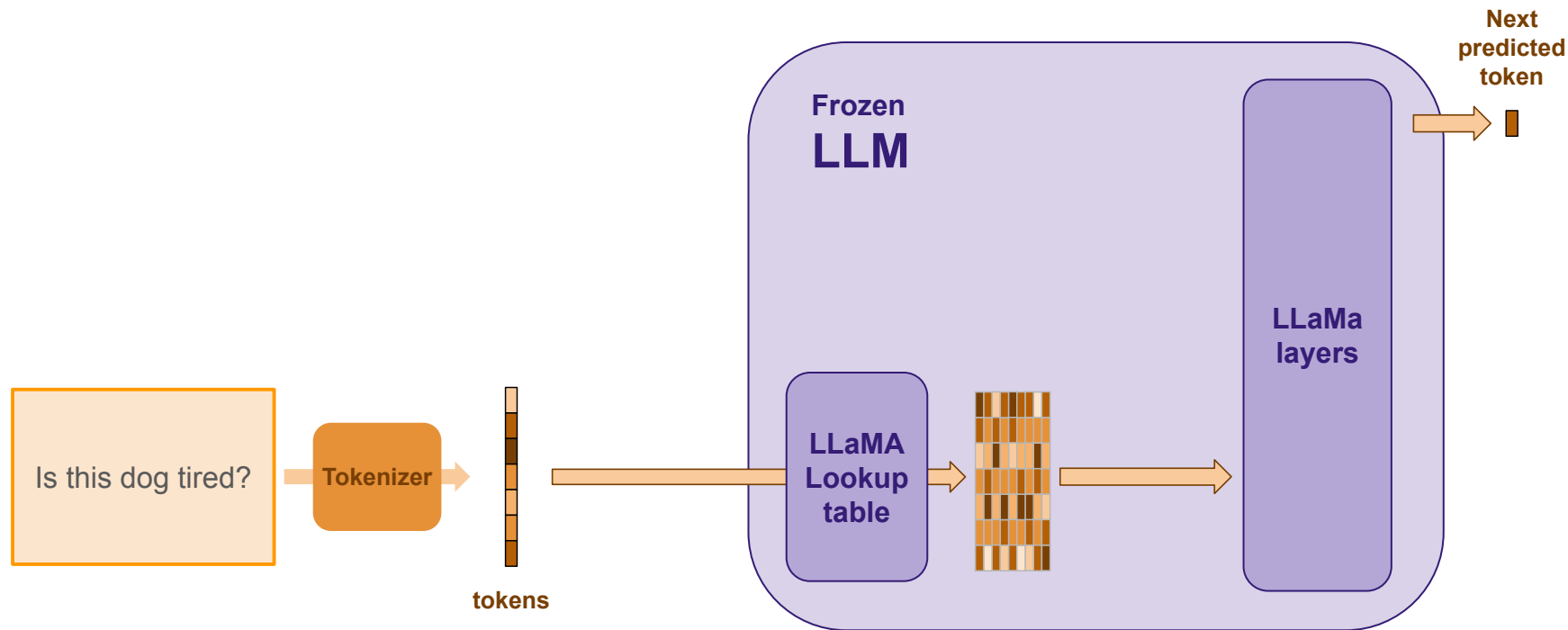
*Komentarz:
Standardowa inferencja w LLM.*



Vision-Language Models

LLaVA

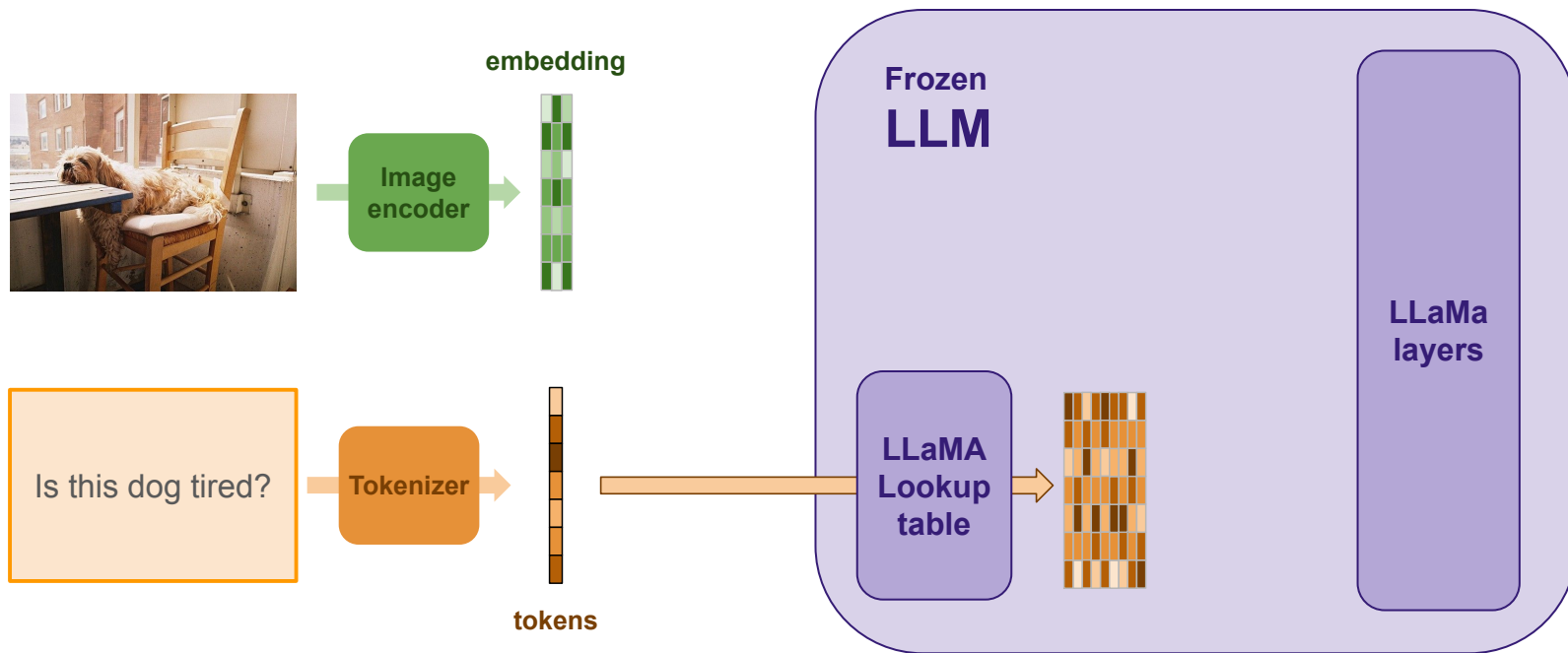
*Komentarz:
Standardowa inferencja w LLM.*



Vision-Language Models

LLaVA

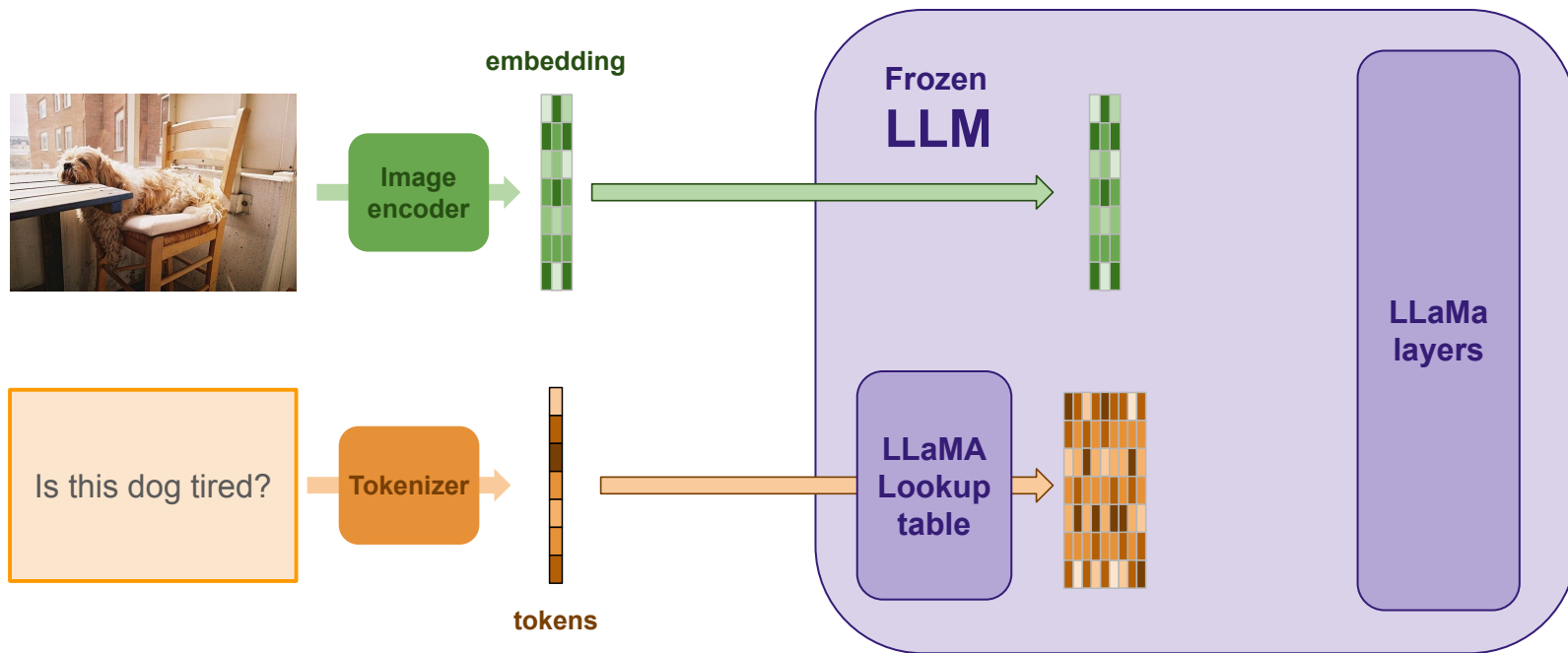
*Komentarz:
Chcemy dołożyć embeddingi z enkodera
obrazkowego (np z CLIP).*



Vision-Language Models

LLaVA

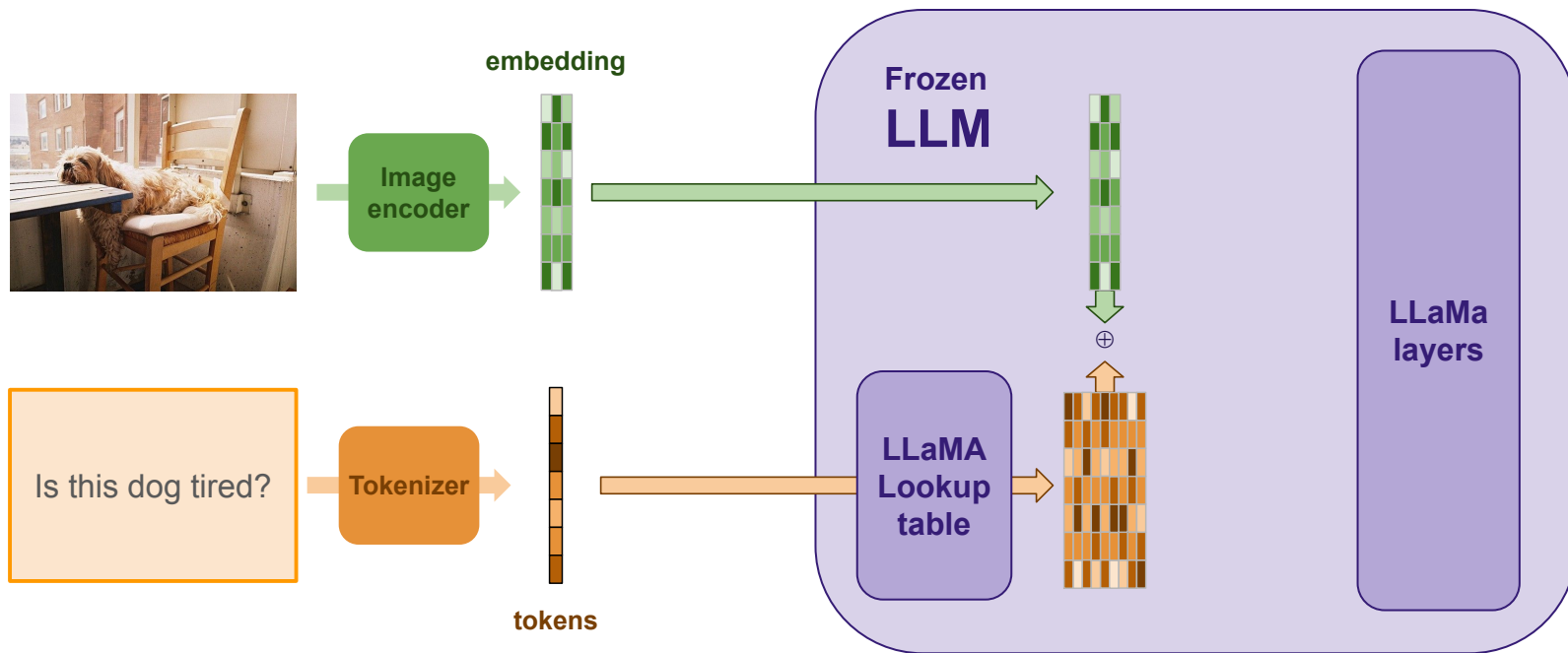
*Komentarz:
Chcemy dołożyć embeddingi z encodera
obrazkowego (np z CLIP).*



Vision-Language Models

LLaVA

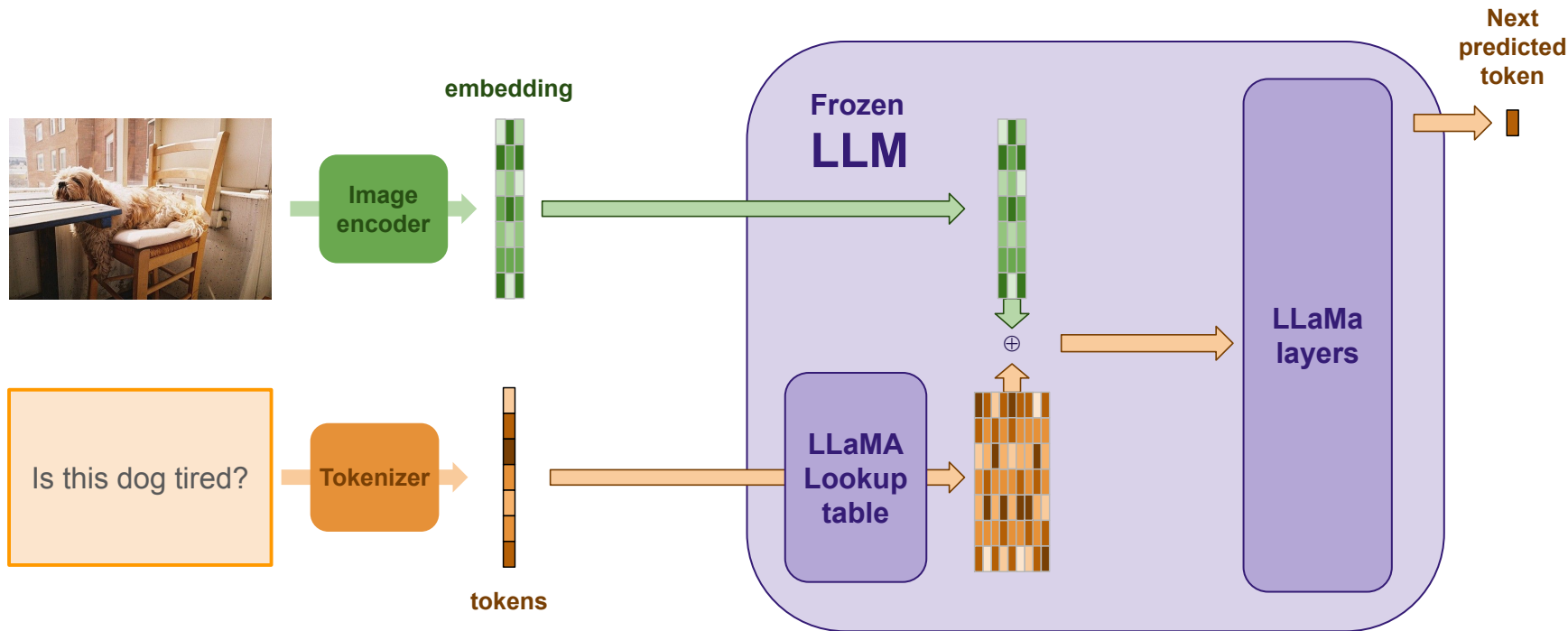
*Komentarz:
Wystarczy połączyć embeddingi
pochodzące z lookup table LLaMa, z tymi
pochodzącymi z enkodera obrazkowego.*



Vision-Language Models

LLaVA

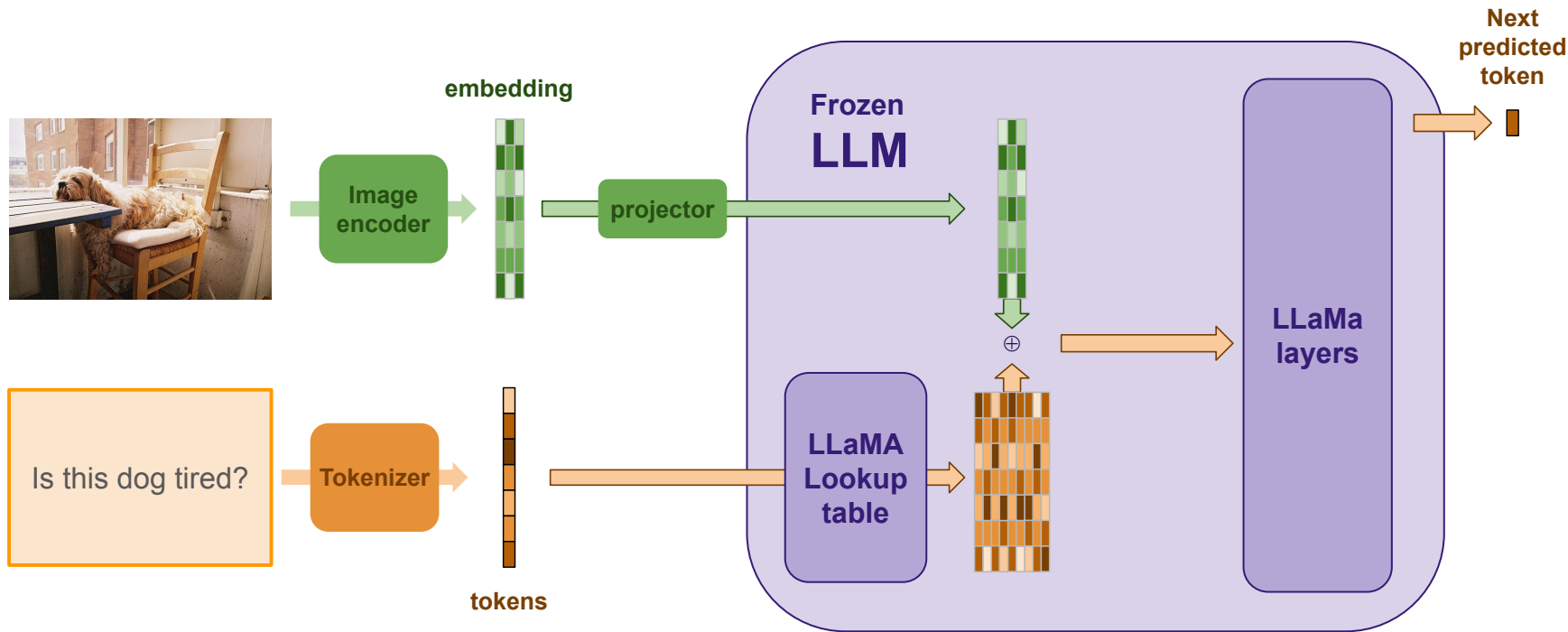
*Komentarz:
Problemem są różne wymiary
embeddingów pochodzących z różnych
źródeł.*



Vision-Language Models

LLaVA

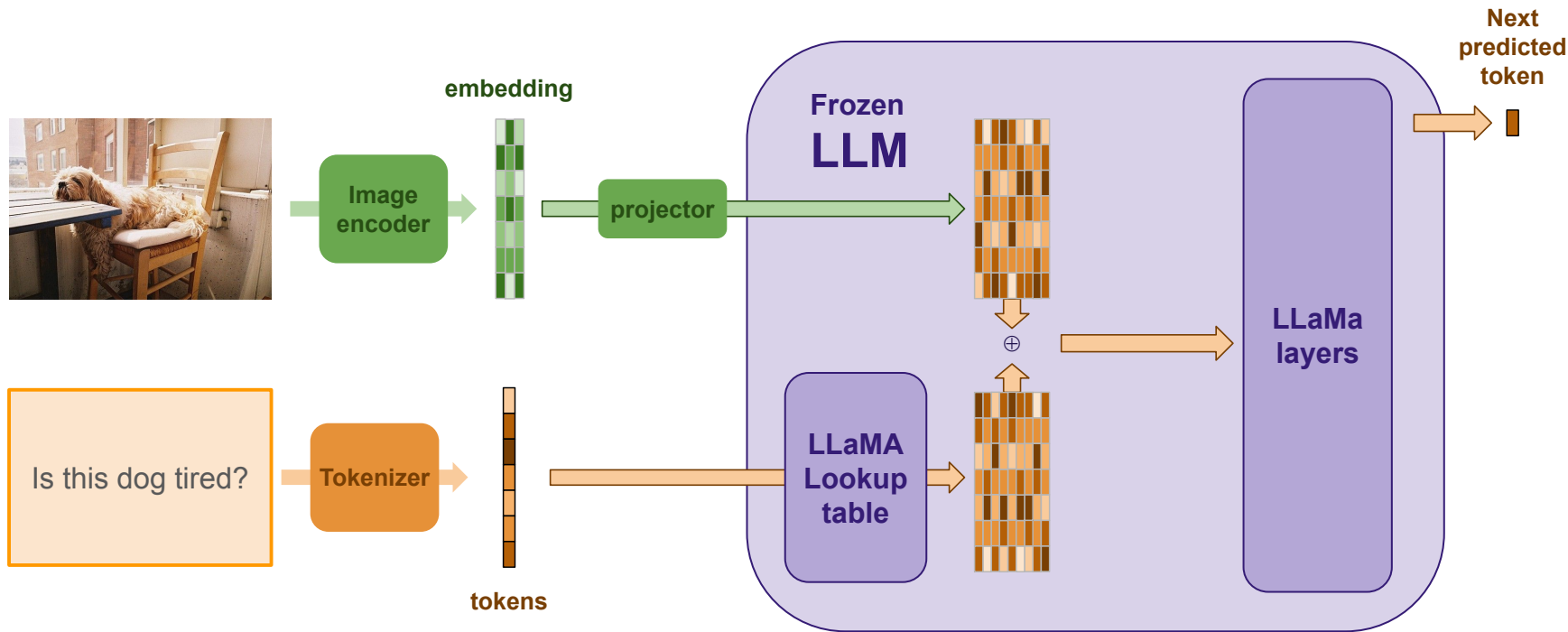
*Komentarz:
Dokładamy projektor, czyli prostą jedno-
lub dwuwarstwową sieć, której celem jest
dopasowanie wymiarowości.*



Vision-Language Models

LLaVA

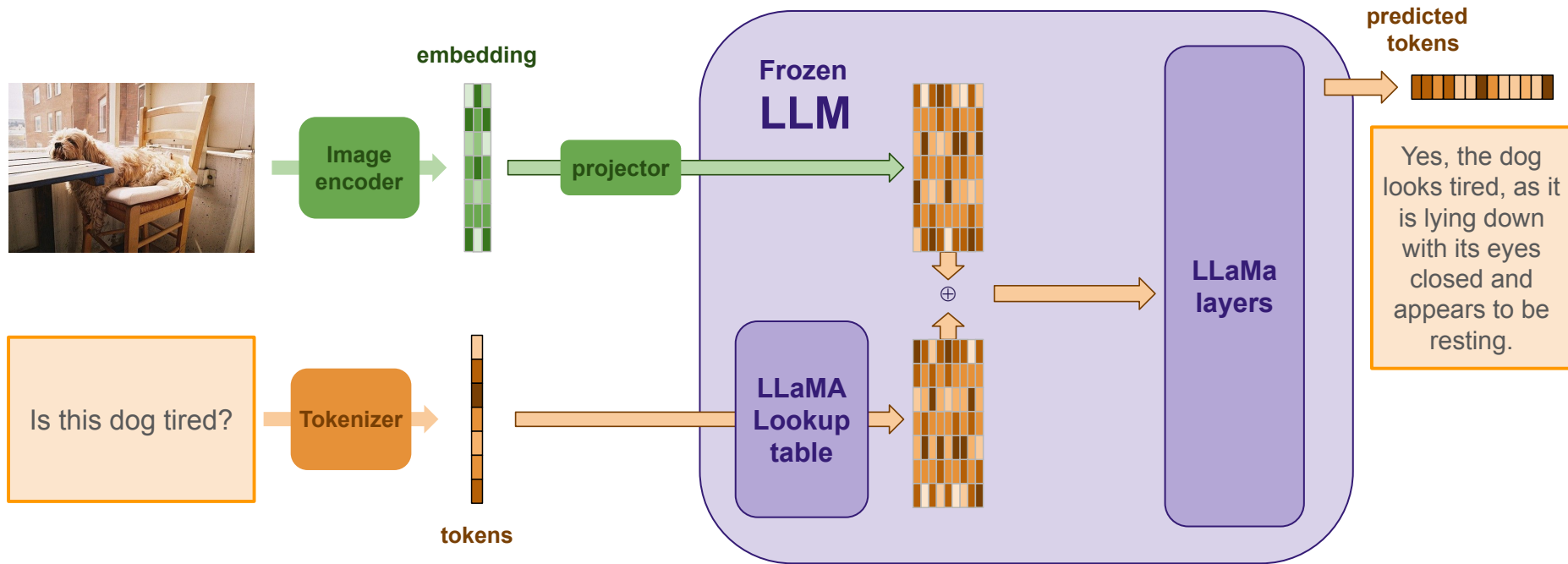
*Komentarz:
Dzięki projektorowi embeddingi
obrazkowe rzutujemy na przestrzeń
embeddingów tekstowych z lookup table.*



Vision-Language Models

LLaVA

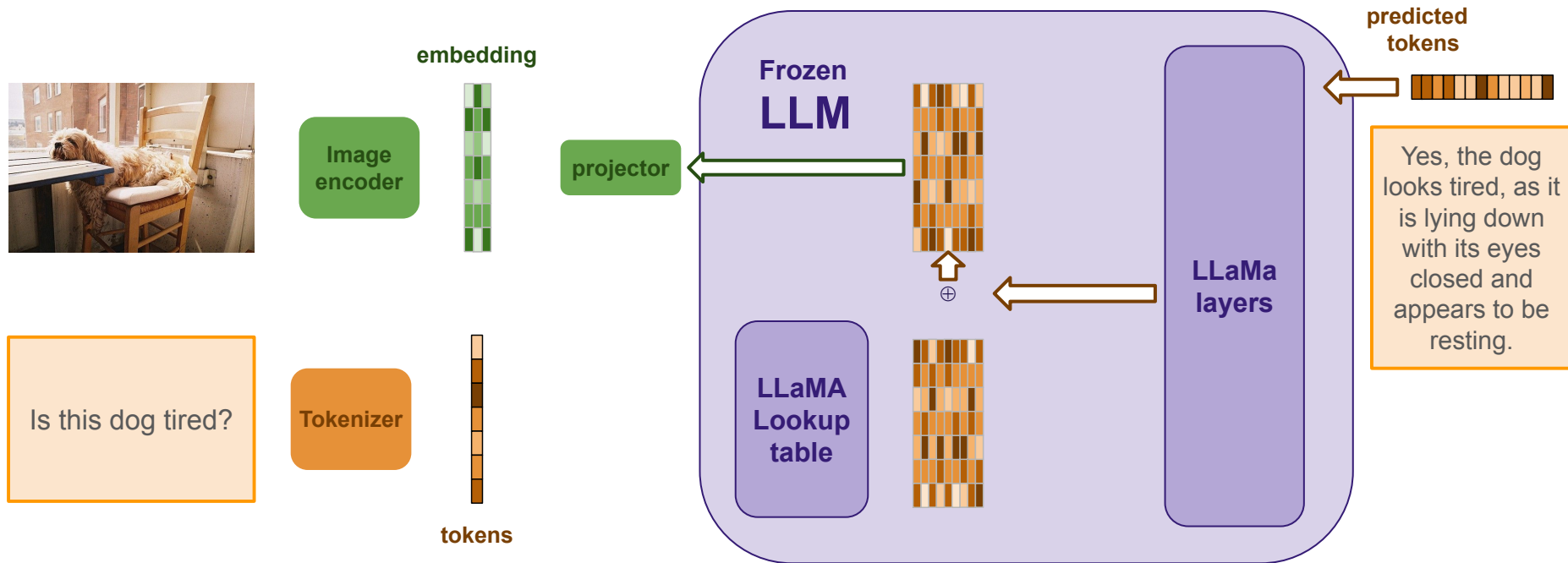
*Komentarz:
Podczas treningu przechodzimy
standardowo przez model i otrzymujemy
rozkład na następnych tokenach.*



Vision-Language Models

LLaVA

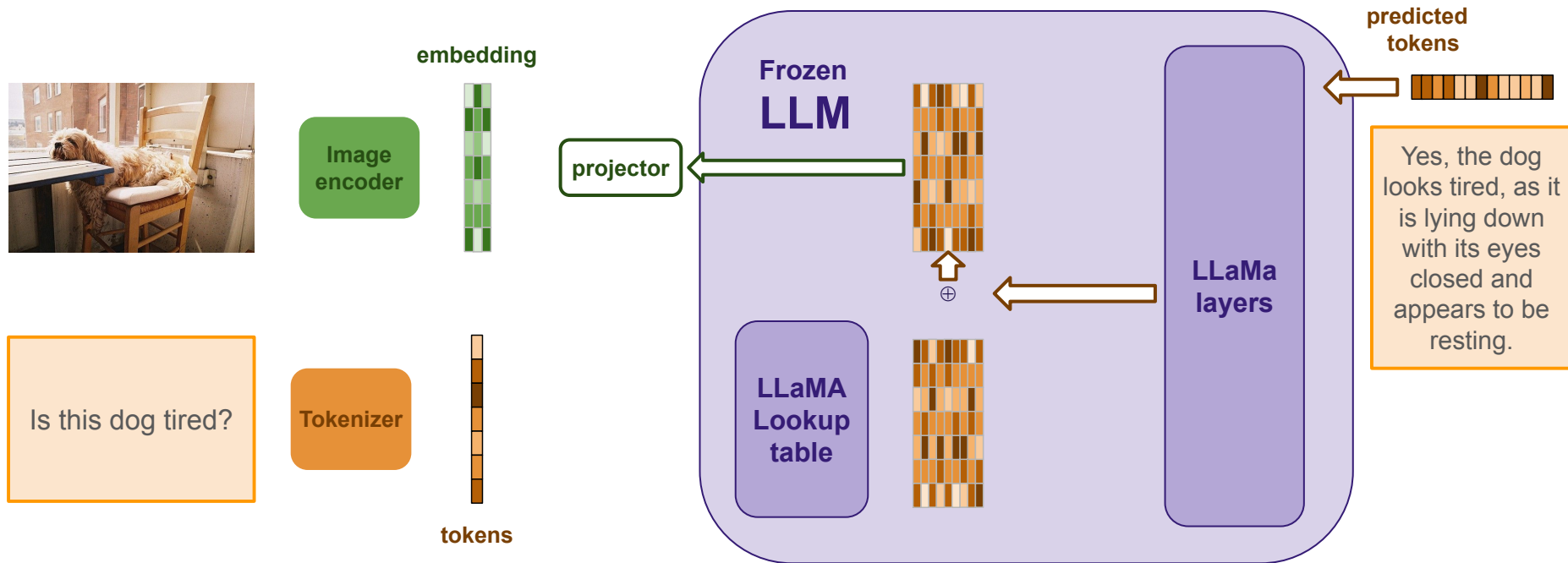
*Komentarz:
W propagacji wstecznej przechodzimy
przez wszystkie (zamrożone parametry)
LLMa.*



Vision-Language Models

LLaVA

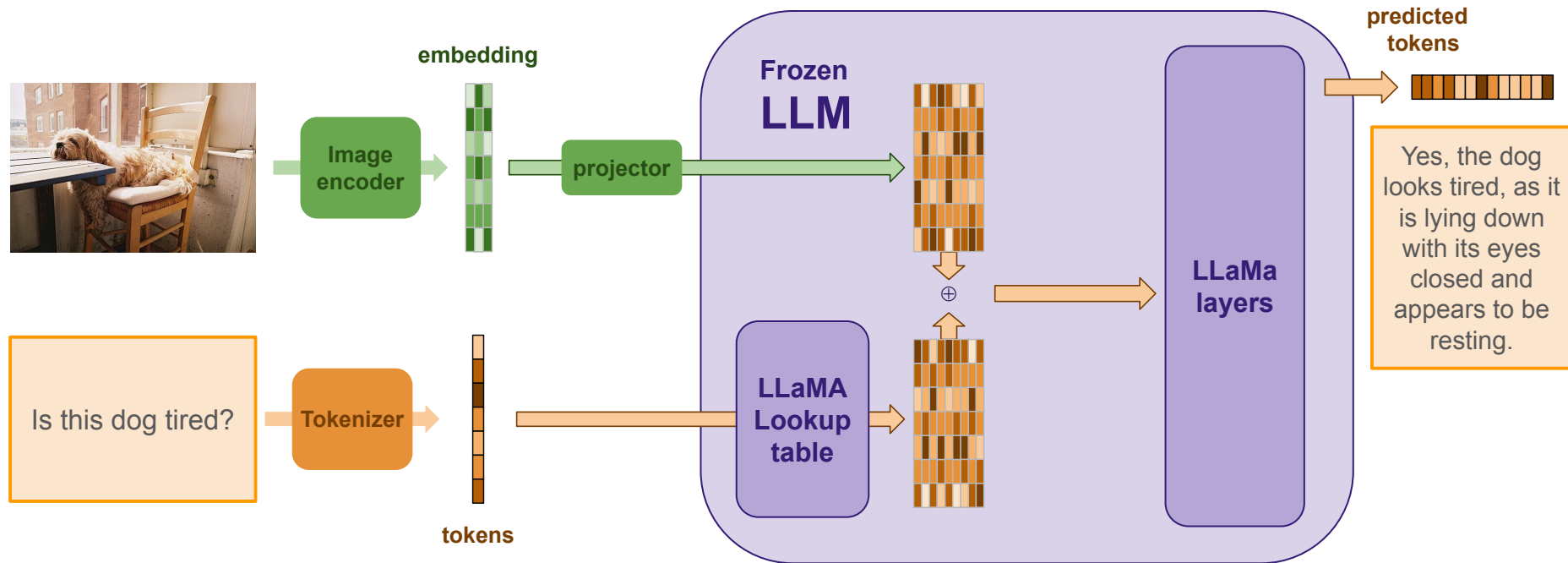
*Komentarz:
Jedyne wyuczalne parametry znajdują się w projektorze.*



Vision-Language Models

LLaVA

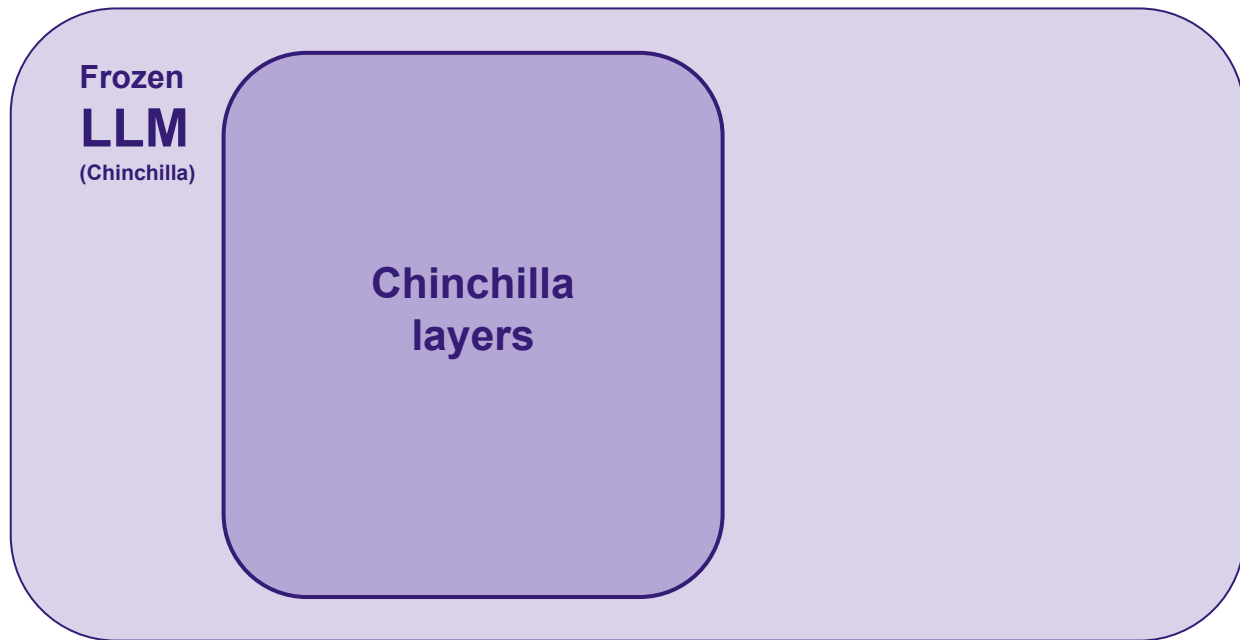
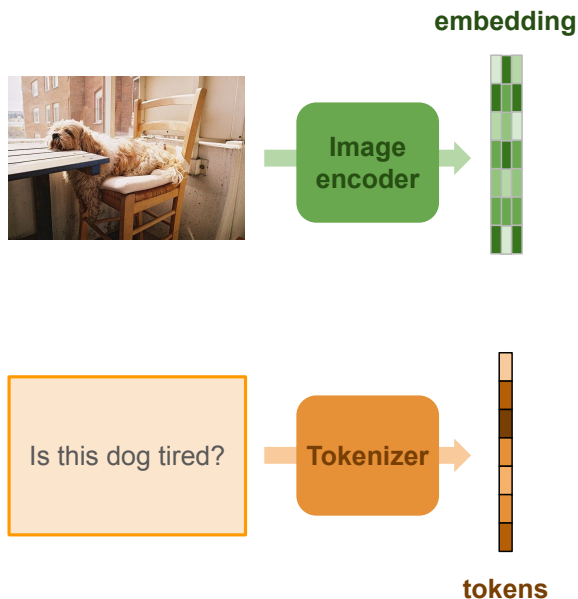
*Komentarz:
Cały schemat LLaVa.*



Vision-Language Models

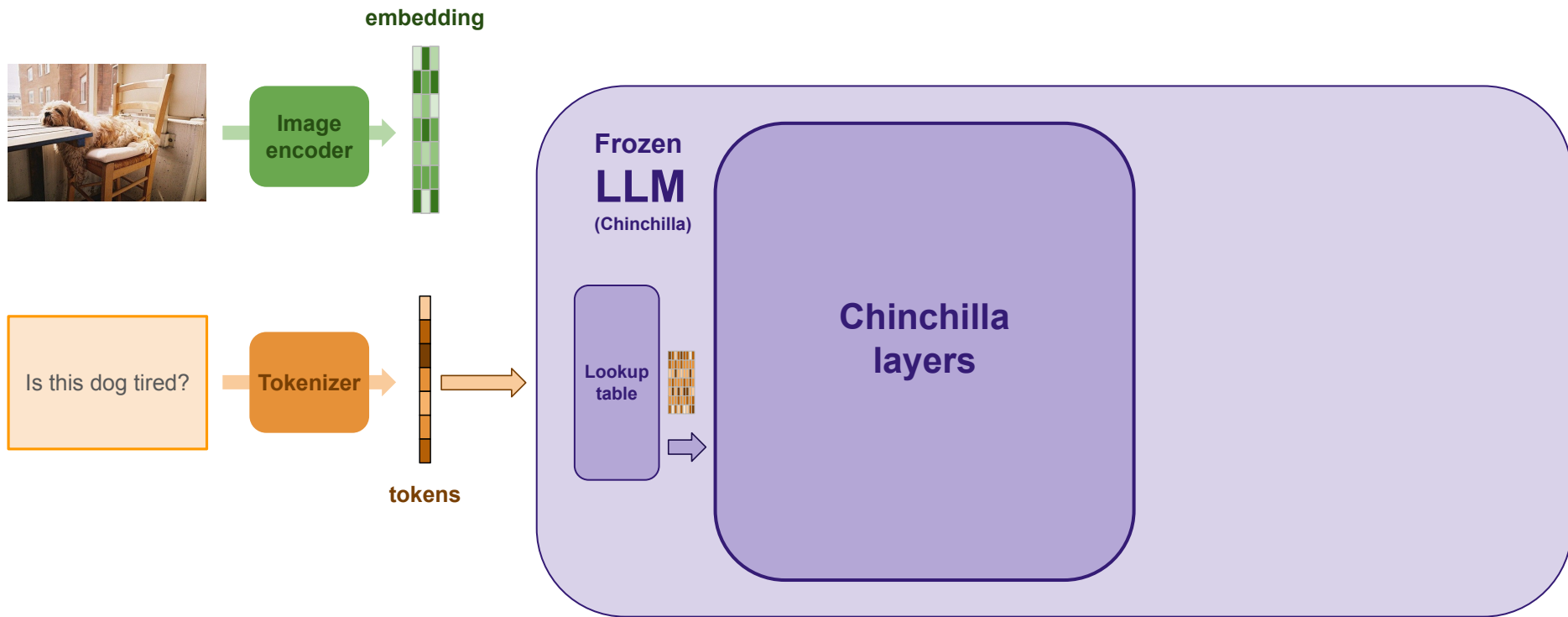
Flamingo

*Komentarz:
Flamingo - inne podejście do tworzenia
VLM z zamrożonego LLMa.*



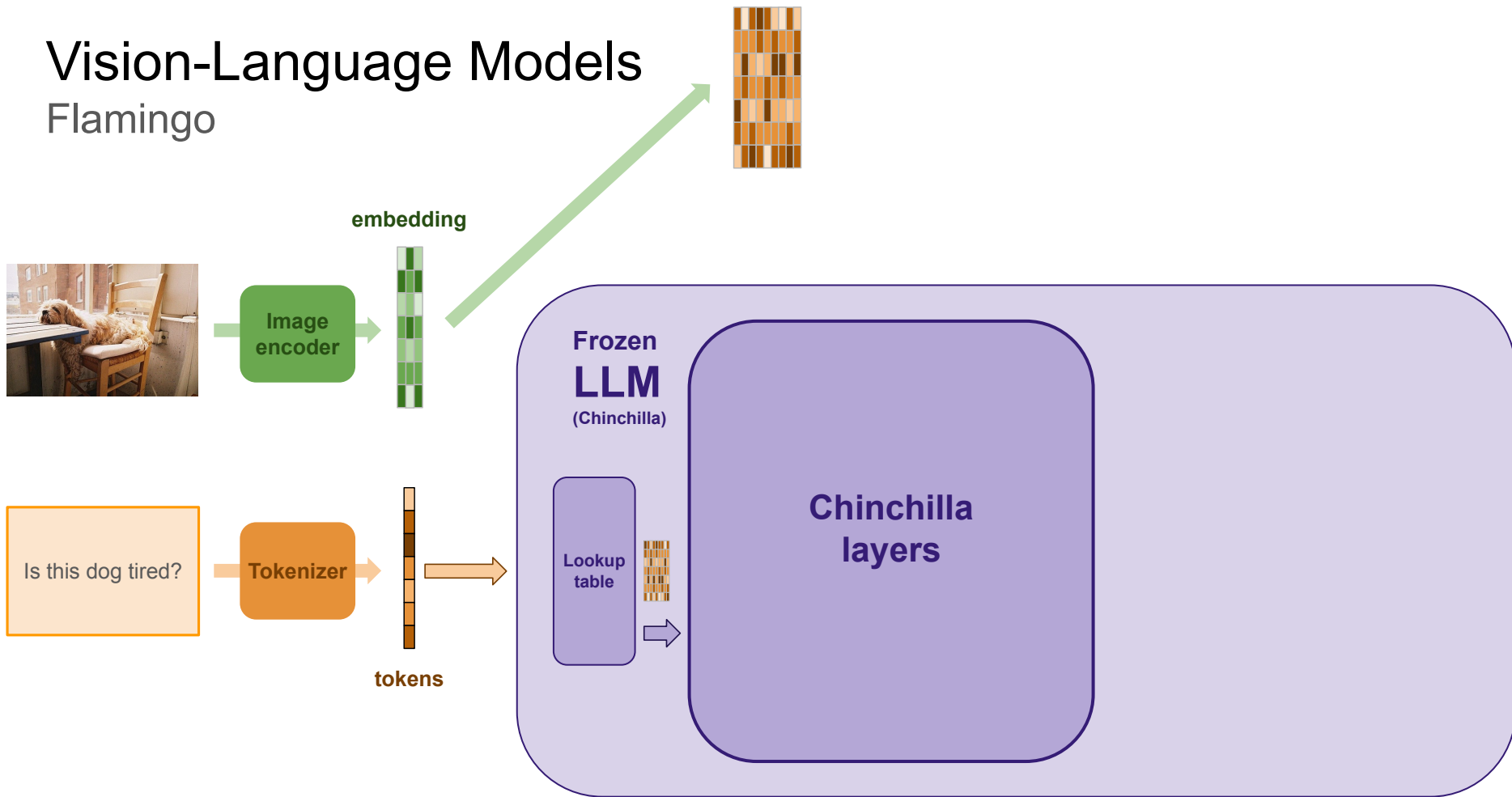
Vision-Language Models

Flamingo



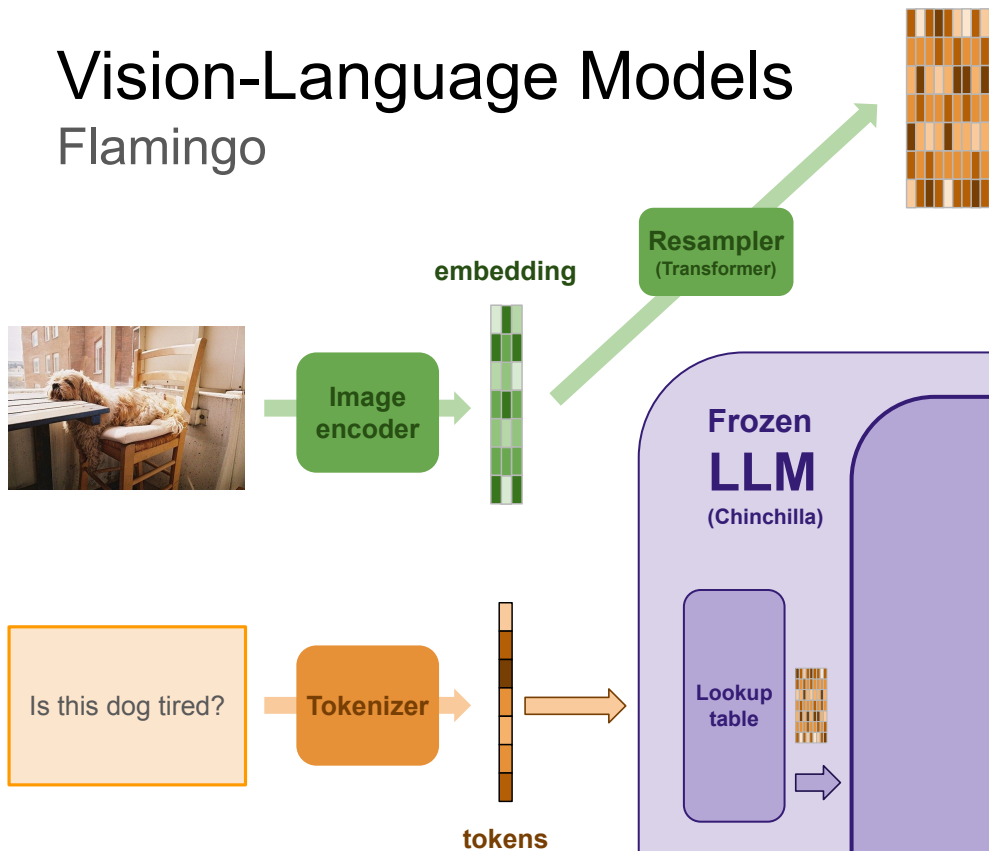
Vision-Language Models

Flamingo



Vision-Language Models

Flamingo

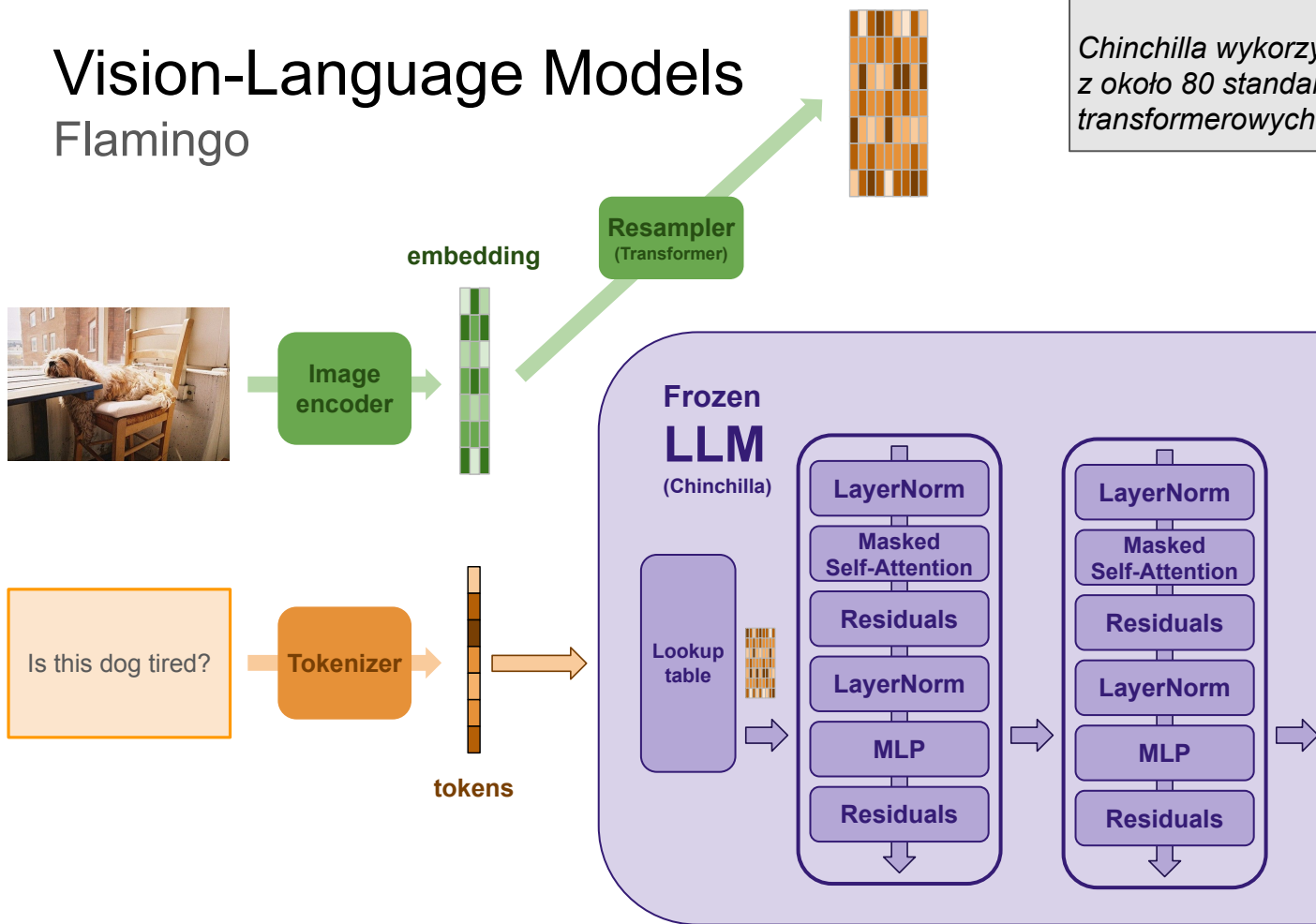


Komentarz:

Flamingo korzysta z resamplera, czyli prostego transformera. Resampler nie służy dopasowywaniu wymiarów, ale tworzeniu wizualnych stanów ukrytych, które mogą zostać wykorzystane w procesie uczenia VLMA.

Vision-Language Models

Flamingo

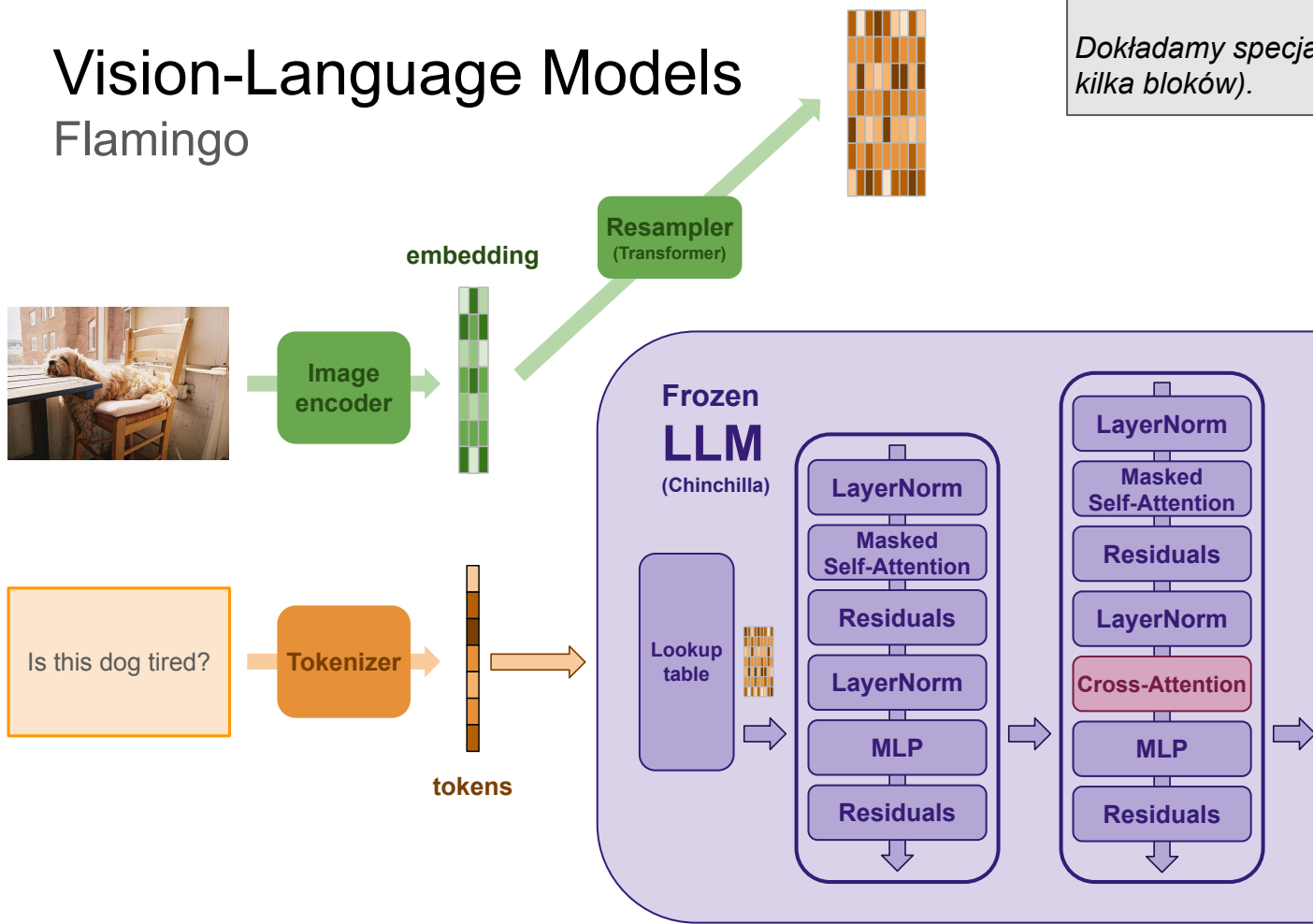


Komentarz:

Chinchilla wykorzystywana w Flamingo składa się z około 80 standardowych bloków transformerowych.

Vision-Language Models

Flamingo



Komentarz:

Dokładamy specjalną warstwę cross-atencji (co kilka bloków).

Vision-Language Models

Flamingo

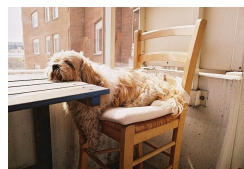


Image
encoder

embedding



Resampler
(Transformer)



Is this dog tired?

Tokenizer



tokens

Frozen
LLM
(Chinchilla)

Lookup
table



LayerNorm
Masked
Self-Attention
Residuals
LayerNorm
MLP
Residuals

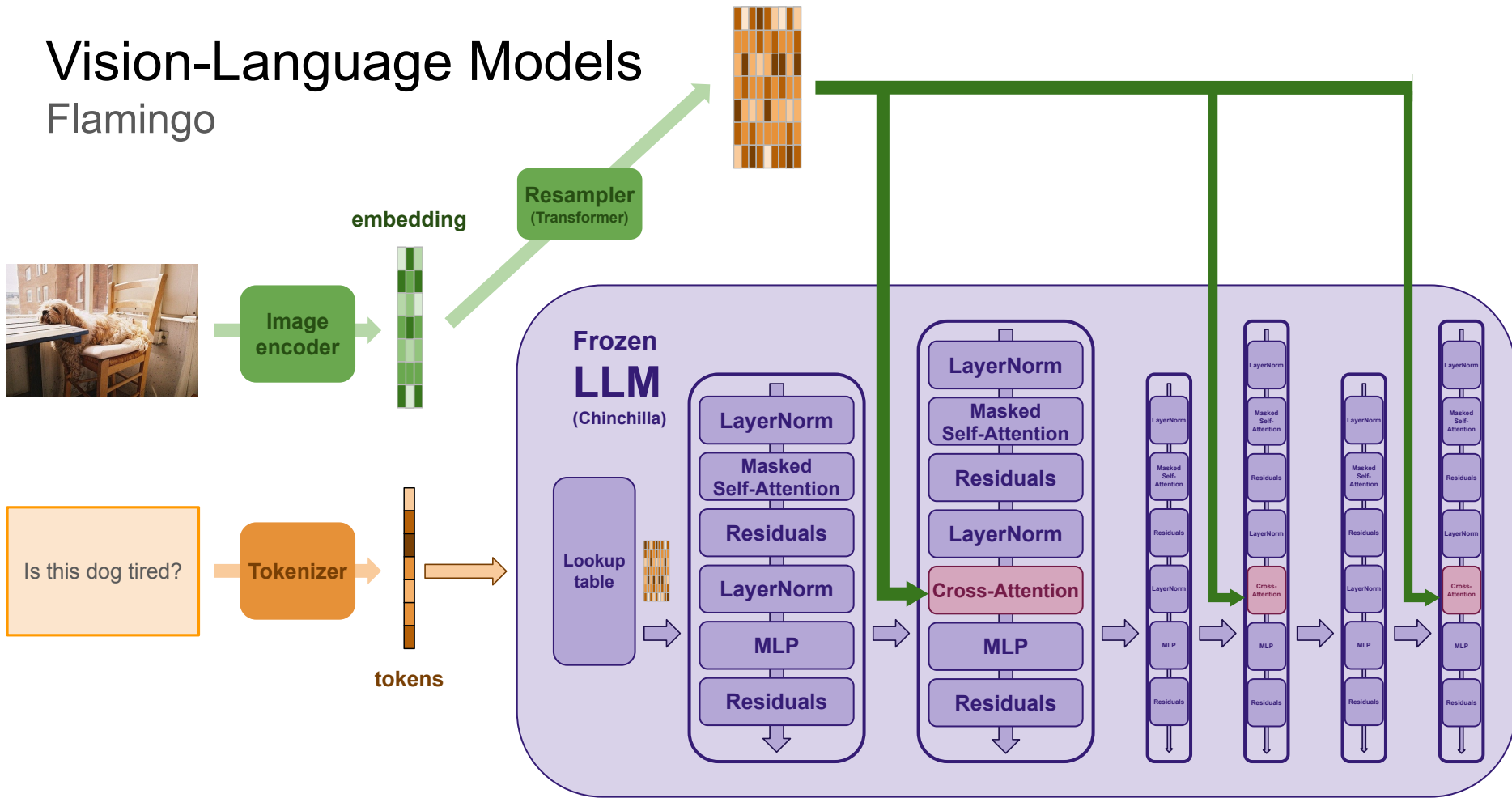
LayerNorm
Masked
Self-Attention
Residuals
LayerNorm
Cross-Attention
MLP
Residuals

Komentarz:

W tej warstwie będziemy przypominać modelowi,
aby rozumował w kontekście obrazu.

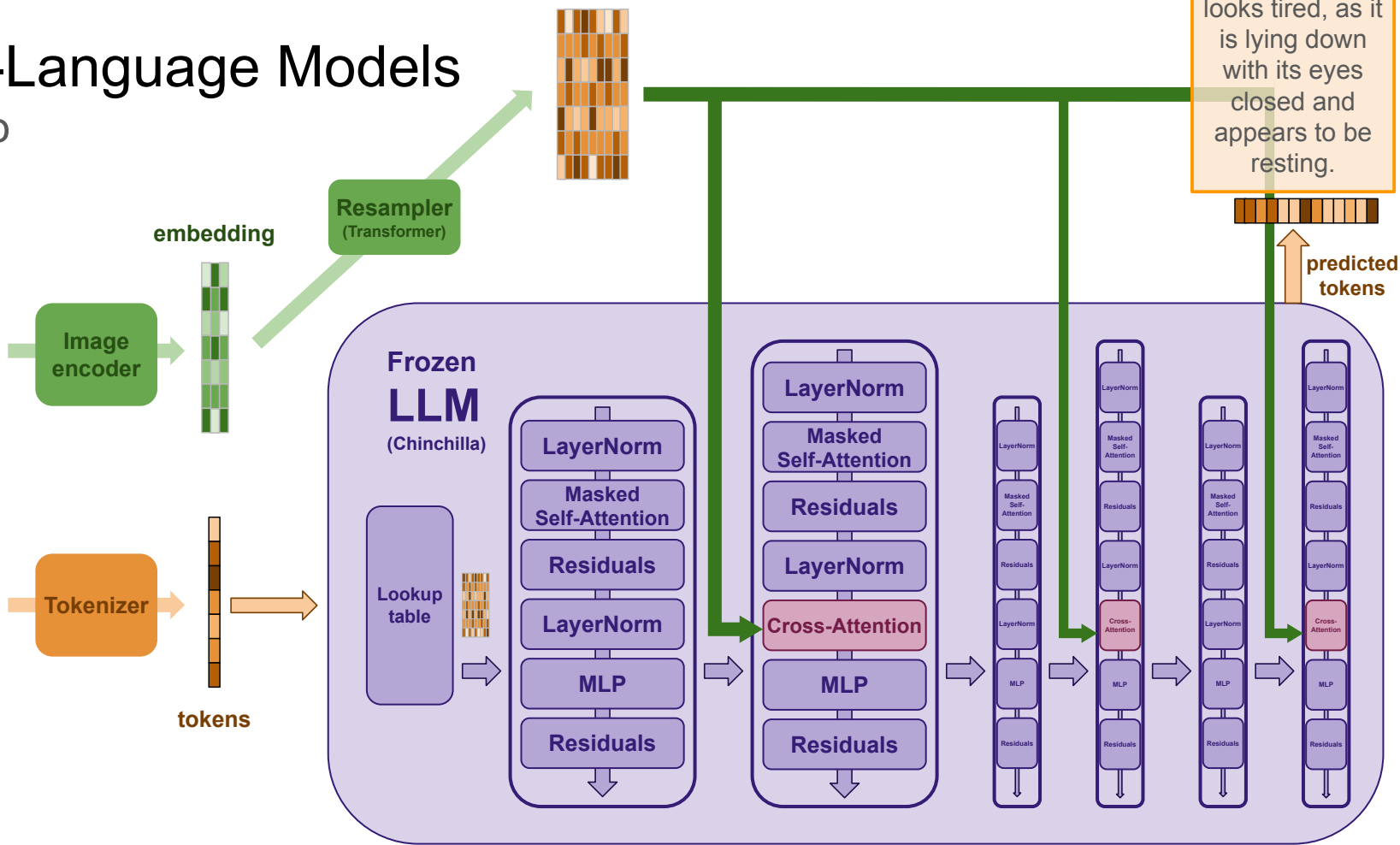
Vision-Language Models

Flamingo



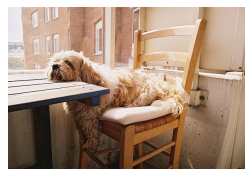
A fluffy, light-colored dog, possibly a Shetland Sheepdog, is sitting on a wooden chair with a white cushion. The dog's head is resting on the edge of a blue table. The background shows a brick building and a window, suggesting an indoor setting with a view of the outdoors.

Is this dog tired?

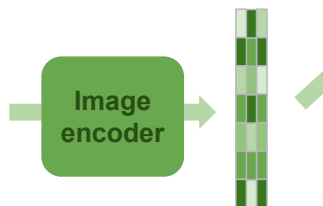
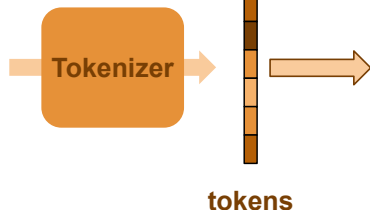


Vision-Language Models

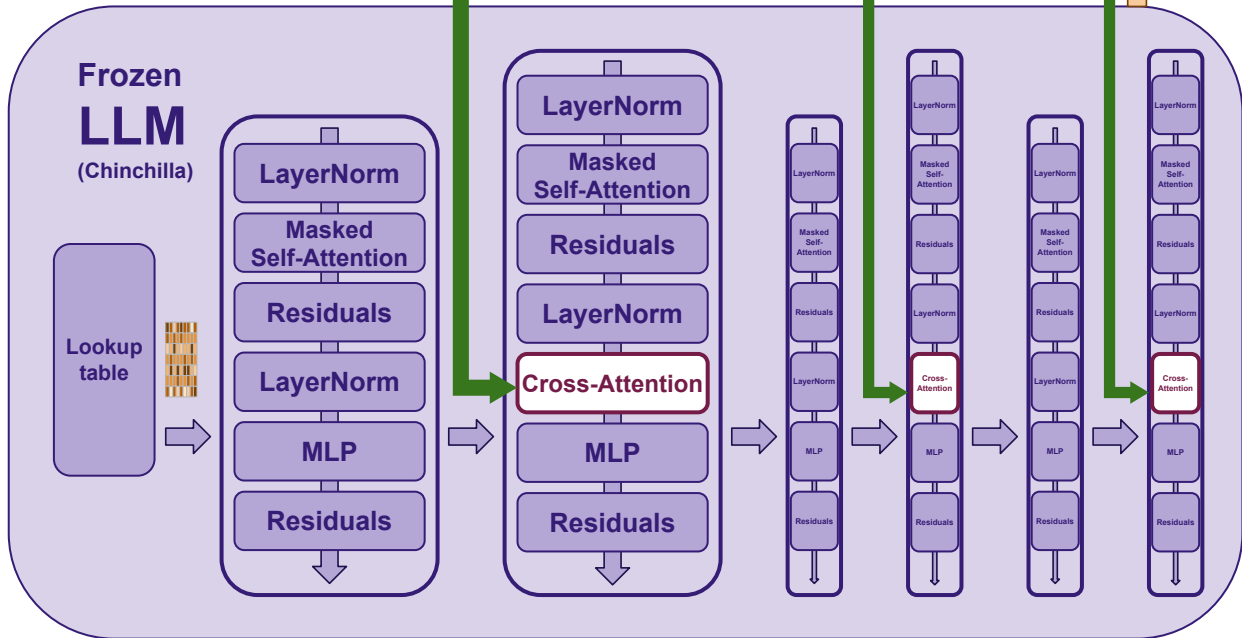
Flamingo



Is this dog tired?



Resampler
(Transformer)

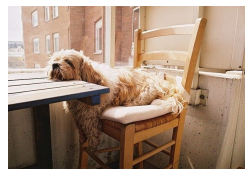


Komentarz:
Podczas propagacji wstecznej
uczymy parametry cross-atencji.

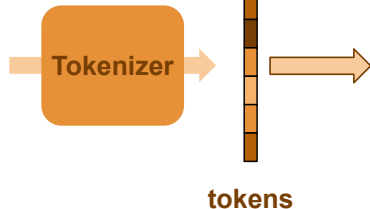
Yes, the dog looks tired, as it is lying down with its eyes closed and appears to be resting.

Vision-Language Models

Flamingo

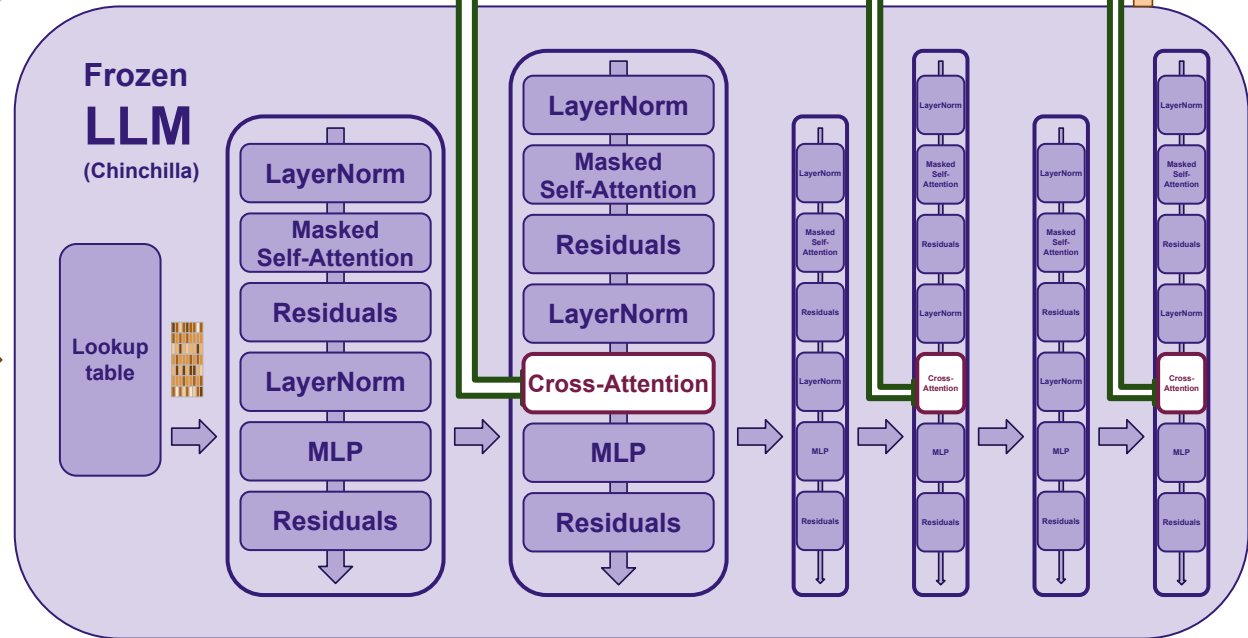


Is this dog tired?



embedding

Resampler
(Transformer)



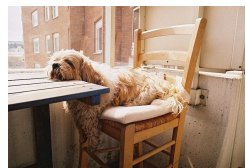
Komentarz:
Podczas propagacji wstecznej
uczymy parametry cross-atencji.

Yes, the dog looks tired, as it is lying down with its eyes closed and appears to be resting.

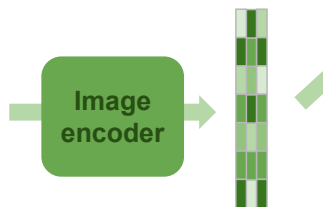
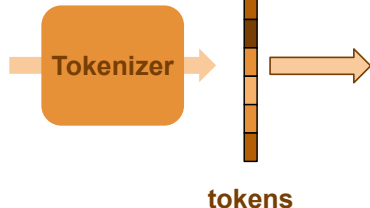
predicted tokens

Vision-Language Models

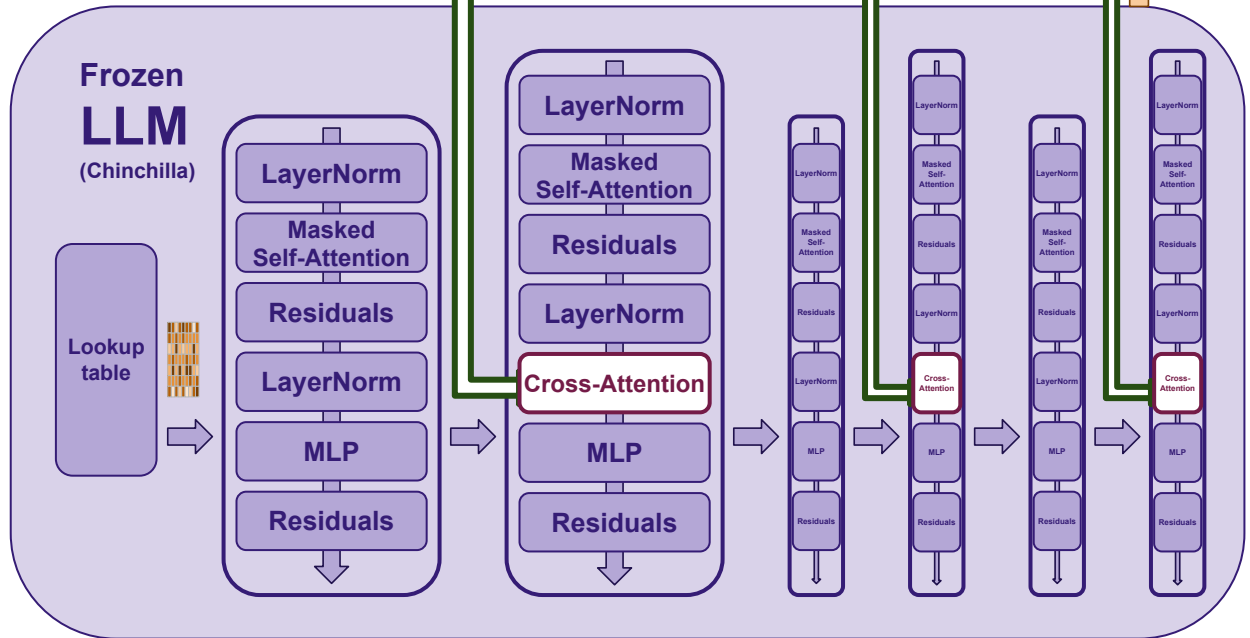
Flamingo



Is this dog tired?



Resampler
(Transformer)



*Komentarz:
Podczas propagacji wstecznej
uczymy resampler.*

Yes, the dog
looks tired, as it
is lying down
with its eyes
closed and
appears to be
resting.

Vision-Language Models

Flamingo

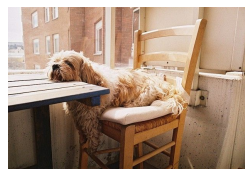
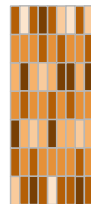


Image
encoder

embedding



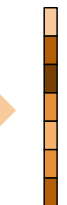
Resampler
(Transformer)



Komentarz:
Cały schemat Flamingo

Is this dog tired?

Tokenizer



tokens

Frozen
LLM
(Chinchilla)

Lookup
table



LayerNorm
Masked
Self-Attention
Residuals
LayerNorm
MLP
Residuals

LayerNorm
Masked
Self-Attention
Residuals
LayerNorm
Cross-Attention
MLP
Residuals

LayerNorm
Masked
Self-Attention
Residuals
LayerNorm
Cross-Attention
MLP
Residuals

LayerNorm
Masked
Self-Attention
Residuals
LayerNorm
Cross-Attention
MLP
Residuals

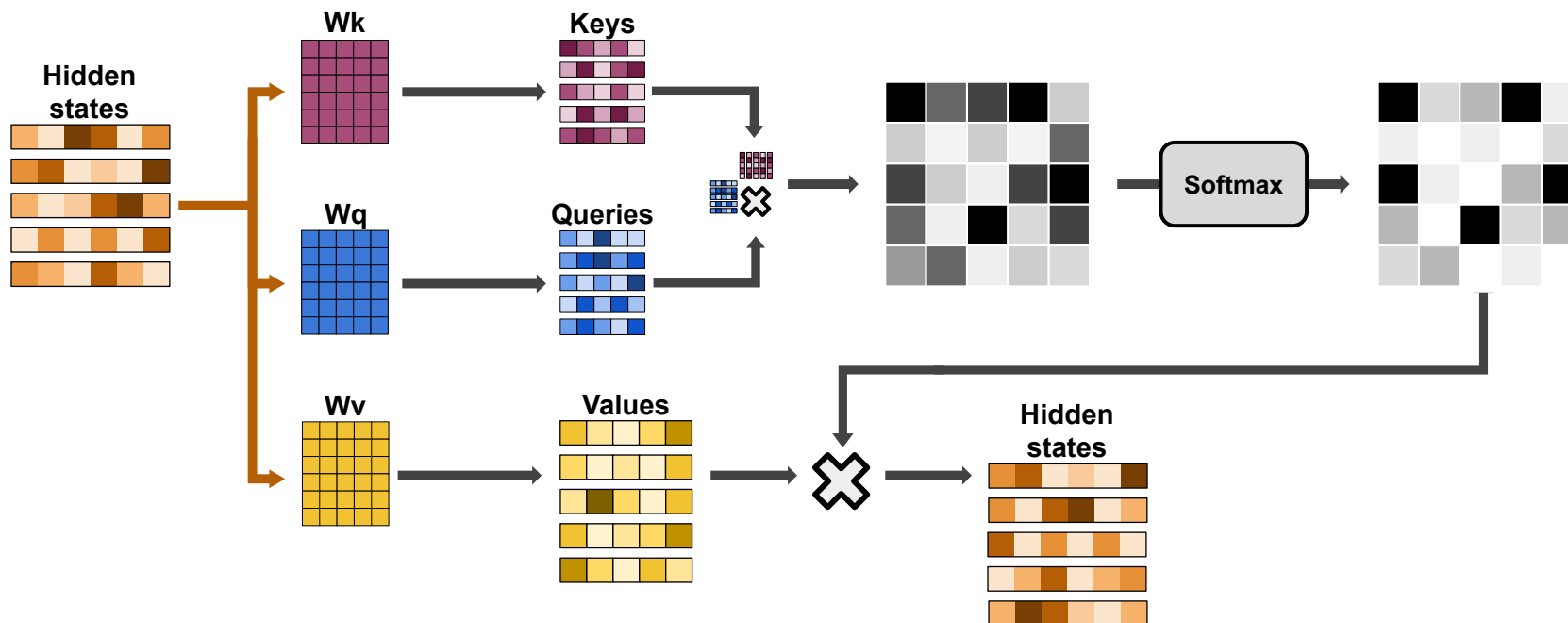
LayerNorm
Masked
Self-Attention
Residuals
LayerNorm
Cross-Attention
MLP
Residuals

LayerNorm
Masked
Self-Attention
Residuals
LayerNorm
Cross-Attention
MLP
Residuals

Komentarz:
Self-atencja - przypomnienie.

Vision-Language Models

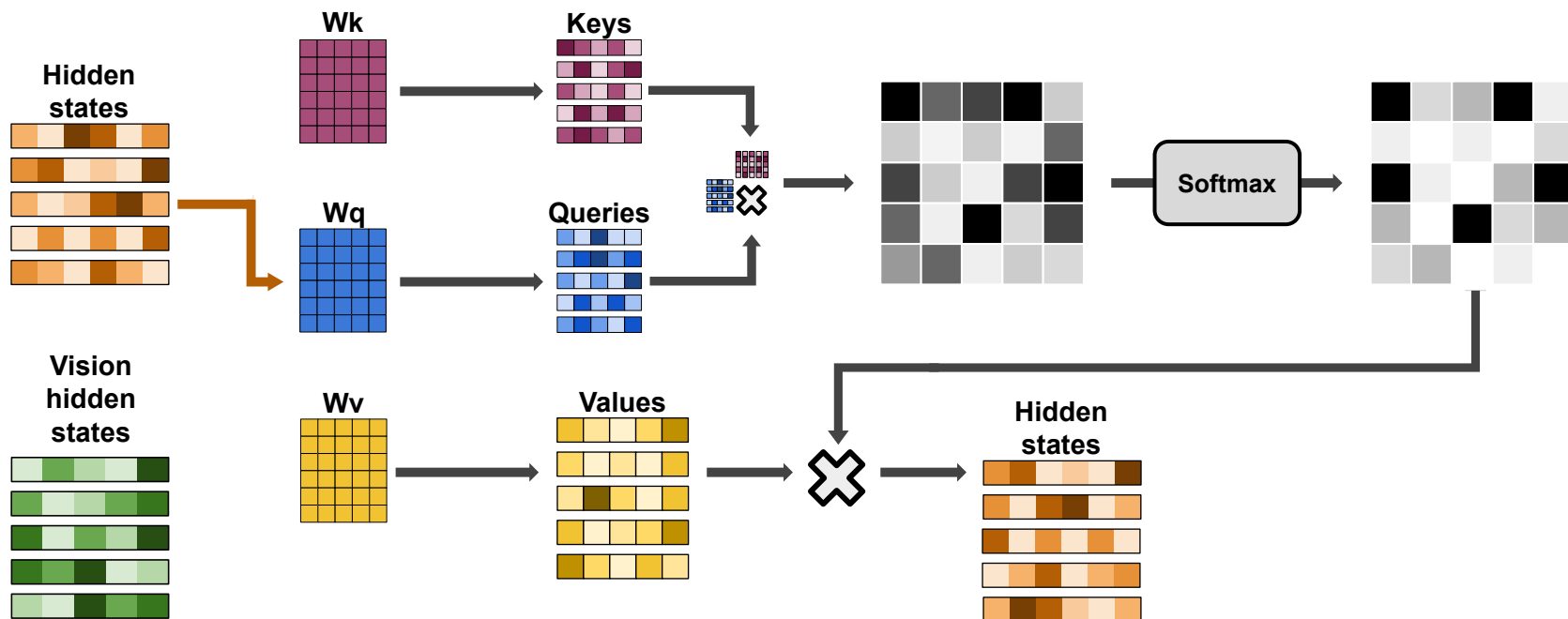
Self-Attention



Vision-Language Models

Cross-Attention

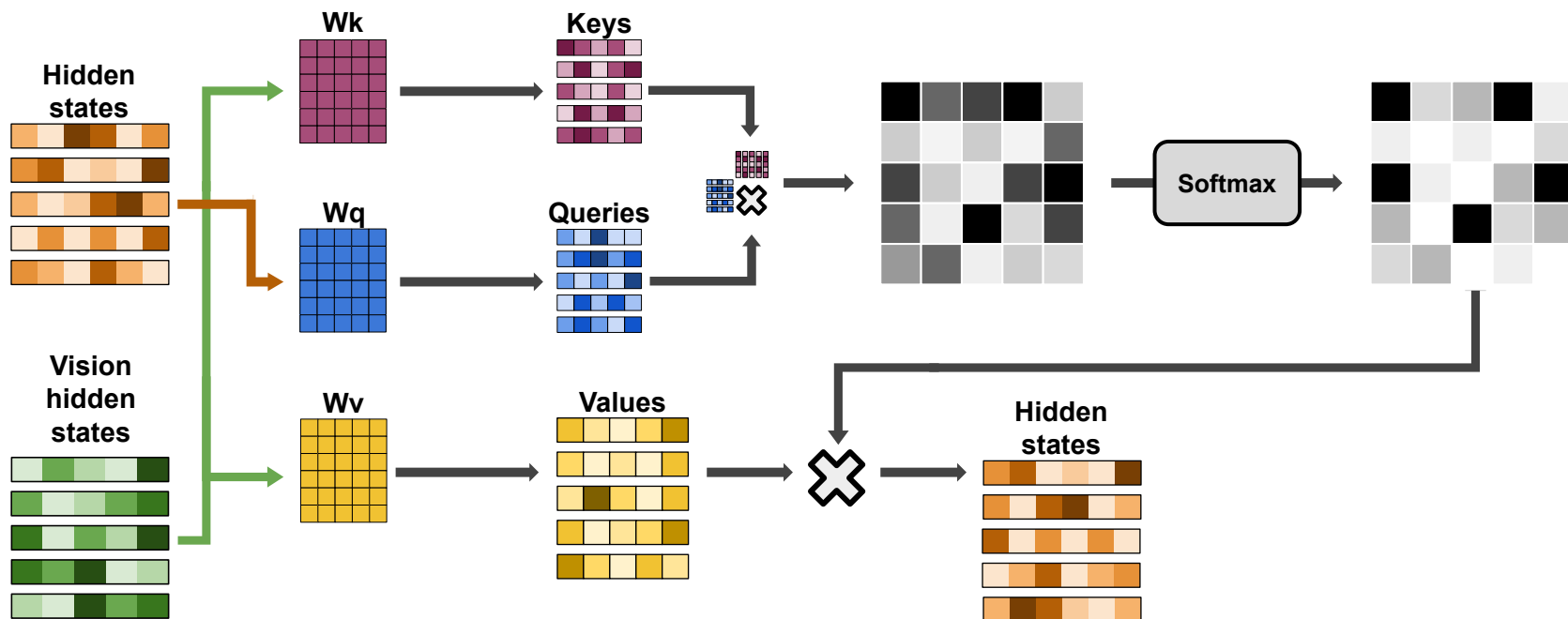
*Komentarz:
W cross-atencji z ukrytych stanów tekstowych
pochodzą wyłącznie wektory Queries.*



Vision-Language Models

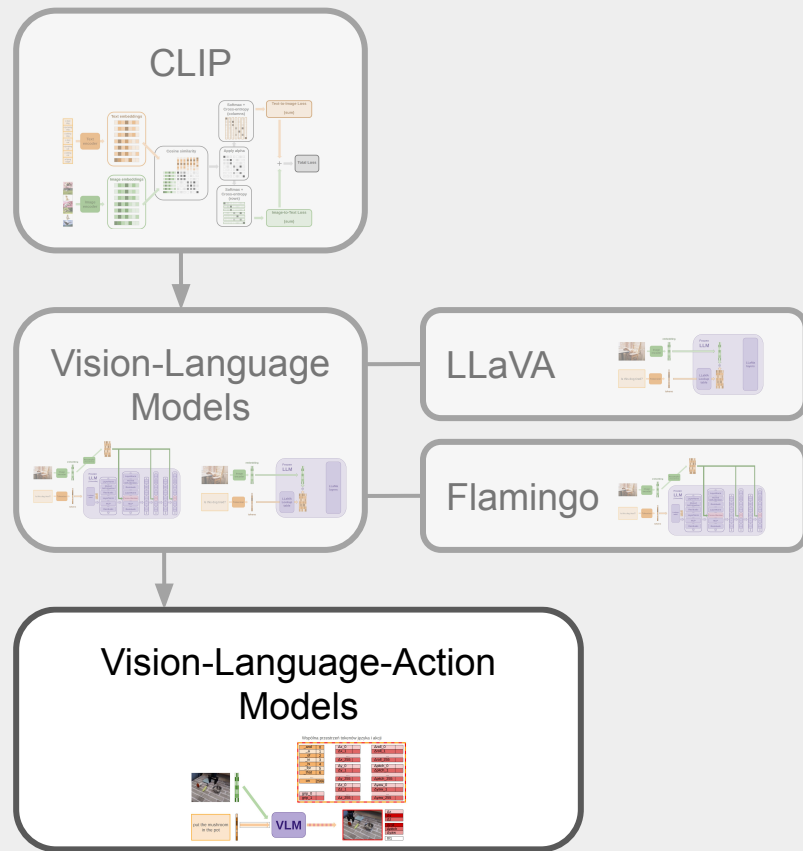
Cross-Attention

*Komentarz:
W cross-atencji z ukrytych stanów obrazkowych
pochodzą wektory Keys oraz Values.*



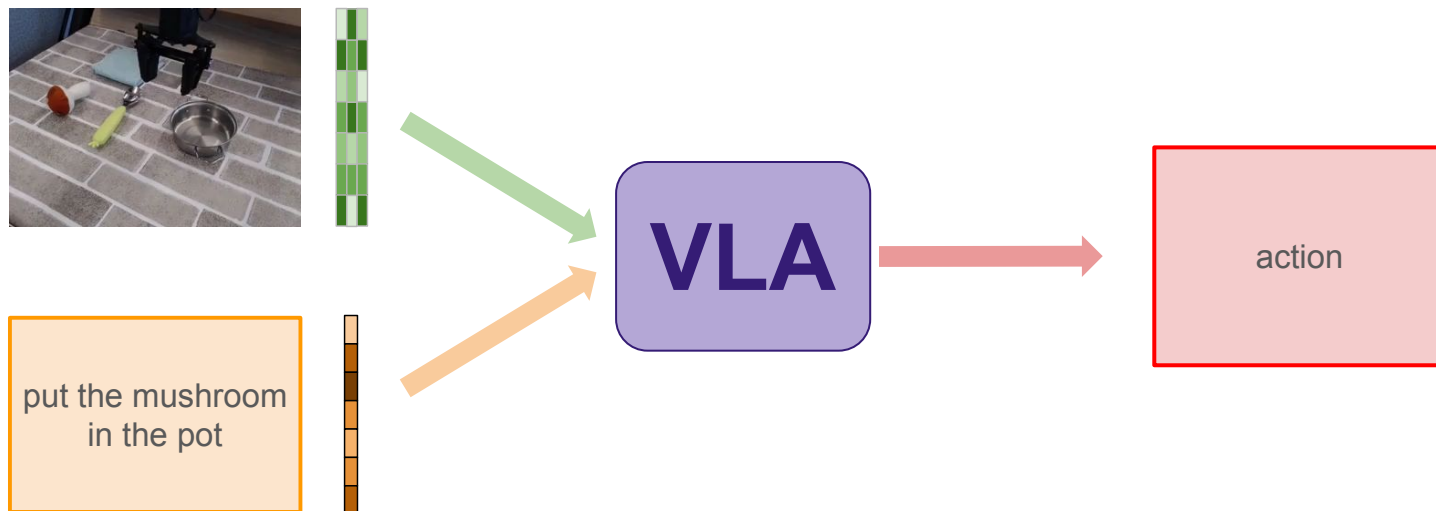
Vision-Language-Action Models

Czyli działanie



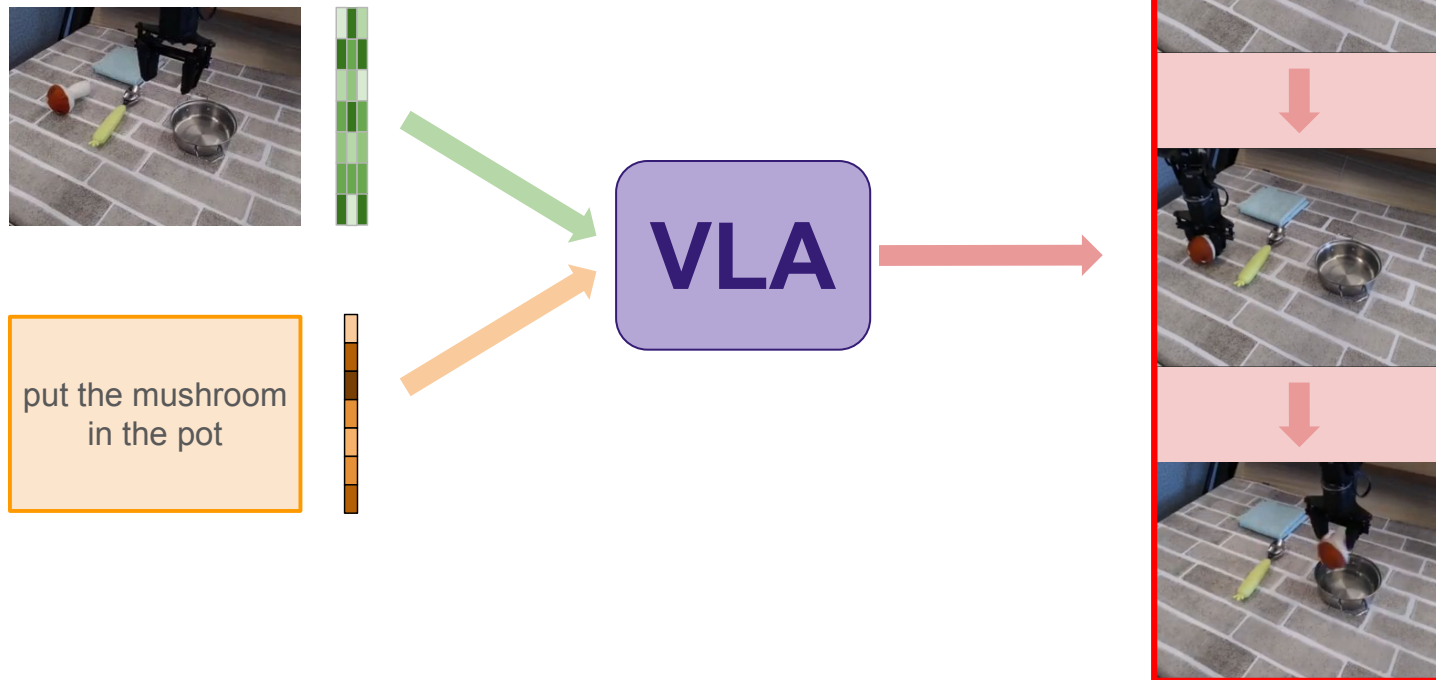
Vision-Language-Action Models

*Komentarz:
VLA - model który na podstawie wizji i tekstu
wykonuje akcję.*



Komentarz:
Oczekiwane całościowe działanie modelu.

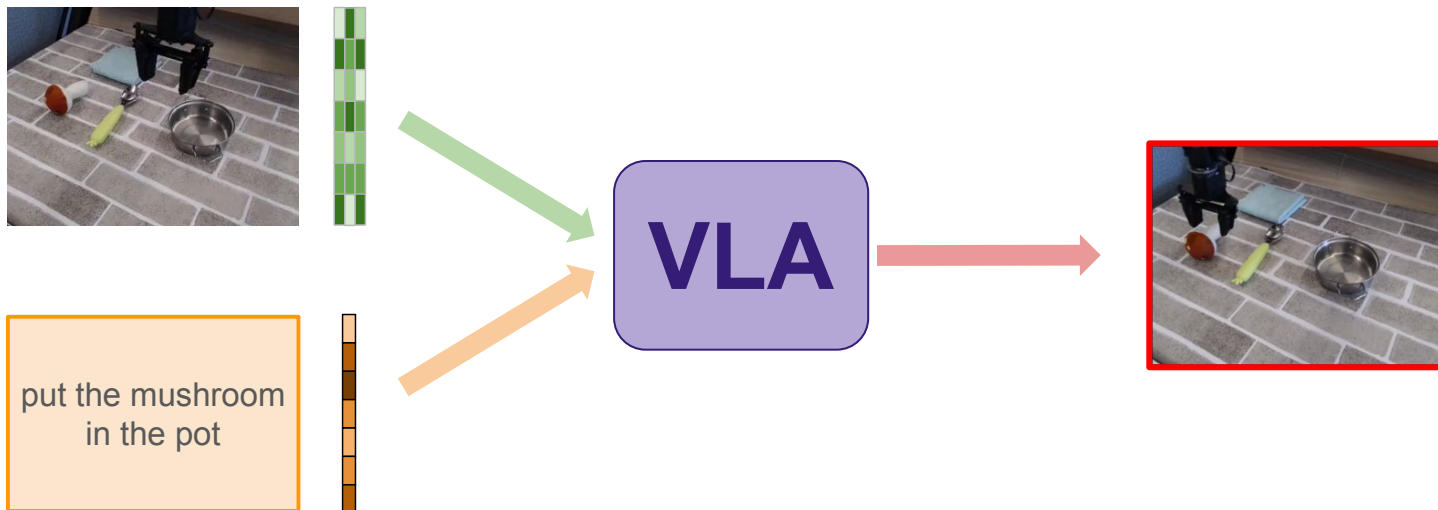
Vision-Language-Action Models



Vision-Language-Action Models

Komentarz:

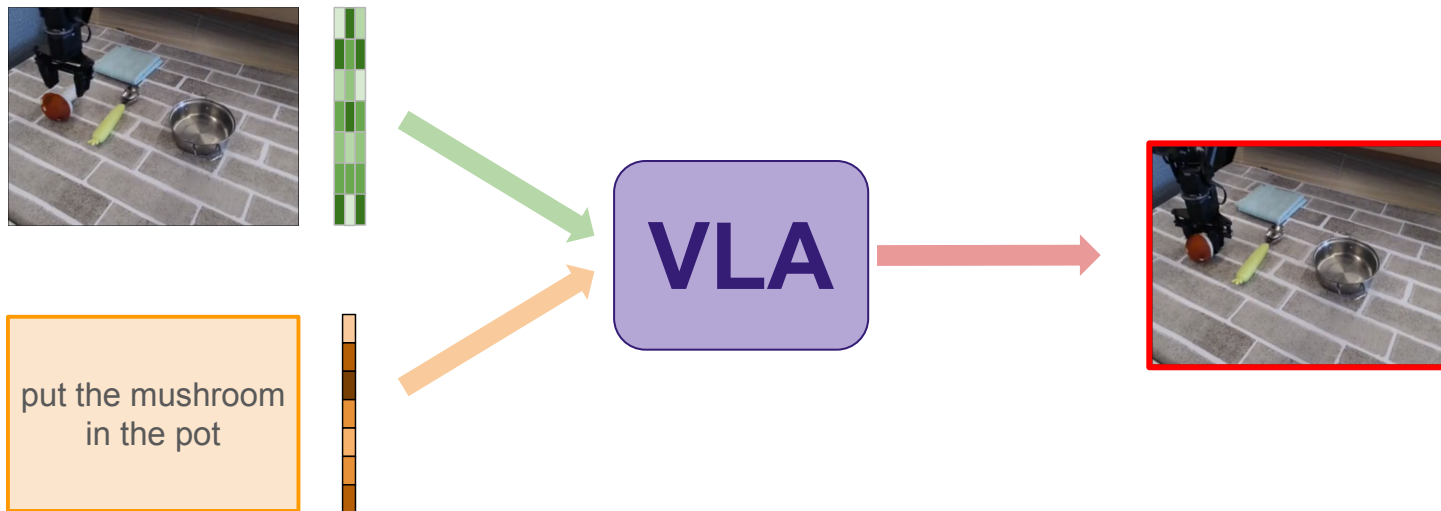
Model zwraca wyłącznie jedną akcję do wykonania, a nie cały ciąg akcji. Akcją może być pojedyncze przesunięcie ramienia i ewentualnie otworenie/zamknięcie chwytaka.



Vision-Language-Action Models

Komentarz:

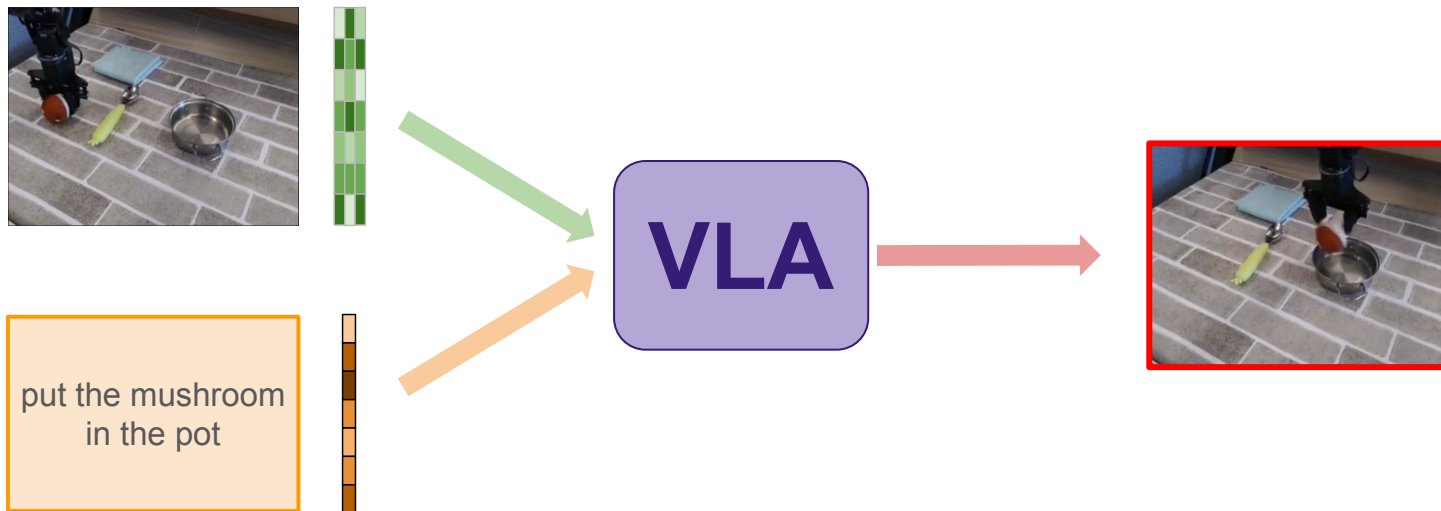
Po wykonaniu pierwszej akcji przekazujemy ten sam tekst oraz zaktualizowany obraz z kamaery umieszczonej na ramieniu robota.



Vision-Language-Action Models

Komentarz:

Po skutecznym zakończeniu zadania (gdy ostatnia akcja spowodowała osiągnięcie celu), bądź zbyt dużej liczbie akcji, model nie otrzymuje nowych informacji.



Vision-Language-Action Models

Przestrzeń akcji

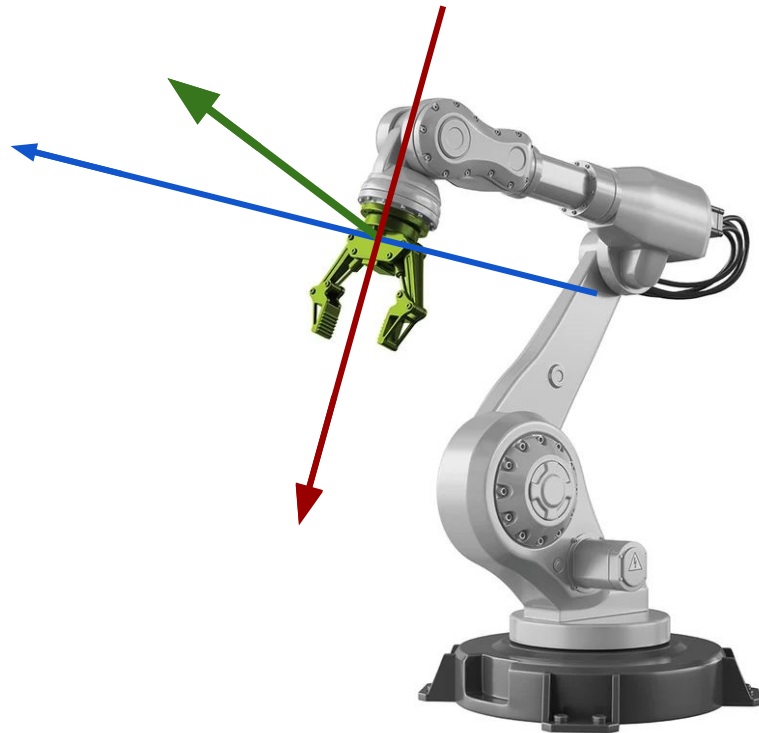


Vision-Language-Action Models

Przestrzeń akcji

Przesunięcie chwytaka:

- Δx
- Δy
- Δz



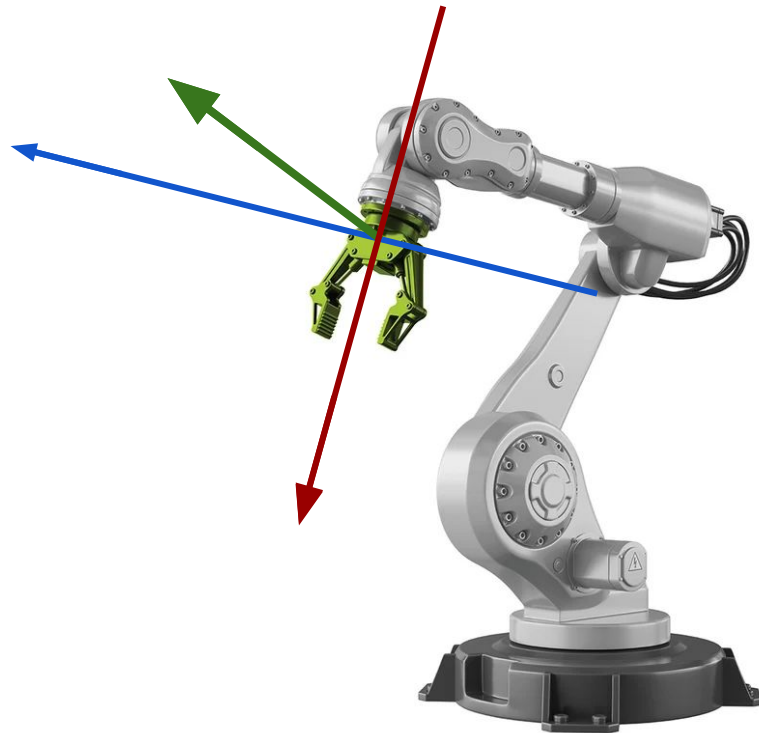
Vision-Language-Action Models

Przestrzeń akcji

Przesunięcie chwytaka:

- Δx [0-255]
- Δy [0-255]
- Δz [0-255]

*Komentarz:
Możliwe akcje umieszczamy w dyskretnej przestrzeni. Każda wartość przesunięcia może przyjąć jedną z 256 wartości.*



Vision-Language-Action Models

Przestrzeń akcji

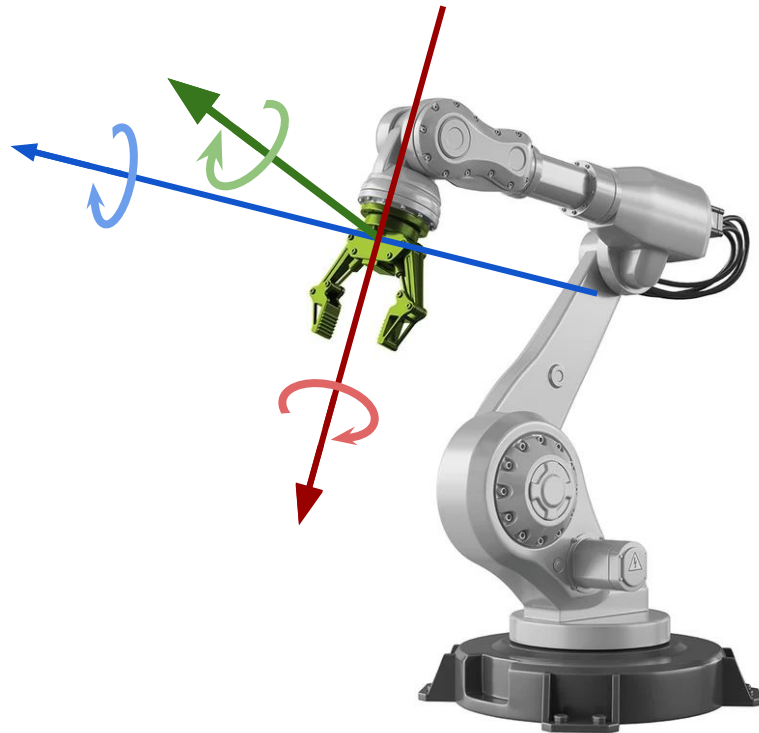
Przesunięcie chwytaka:

- Δx [0-255]
- Δy [0-255]
- Δz [0-255]

Rotacja chwytaka:

- $\Delta roll$ [0-255]
- $\Delta pitch$ [0-255]
- Δyaw [0-255]

*Komentarz:
Możliwe akcje umieszczamy w dyskretnej przestrzeni. Każda wartość obrotu może przyjąć jedną z 256 wartości.*



Vision-Language-Action Models

Przestrzeń akcji

Przesunięcie chwytaka:

- Δx [0-255]
- Δy [0-255]
- Δz [0-255]

Rotacja chwytaka:

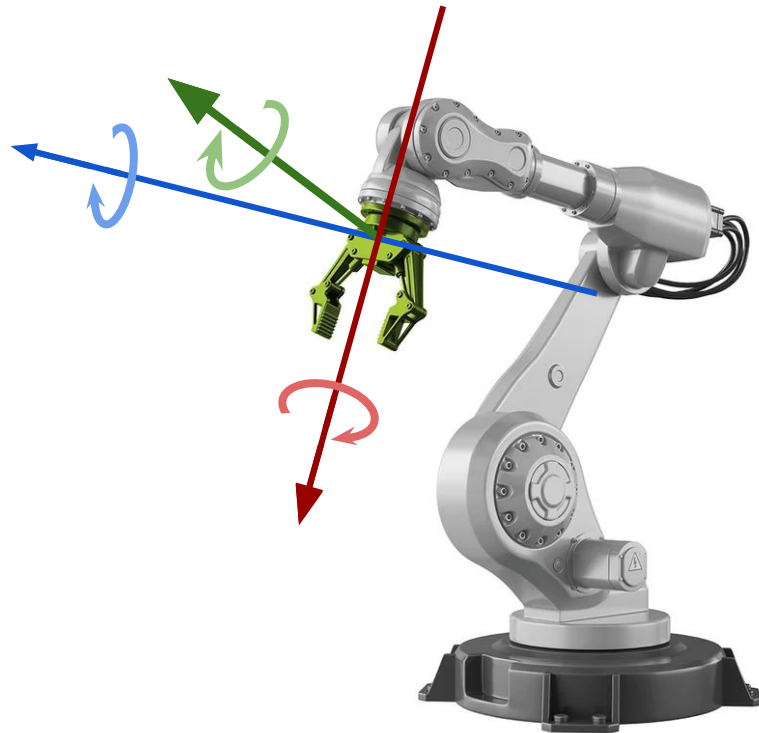
- $\Delta roll$ [0-255]
- $\Delta pitch$ [0-255]
- Δyaw [0-255]

Stan chwytaka:

- gripper_state {0, 1}

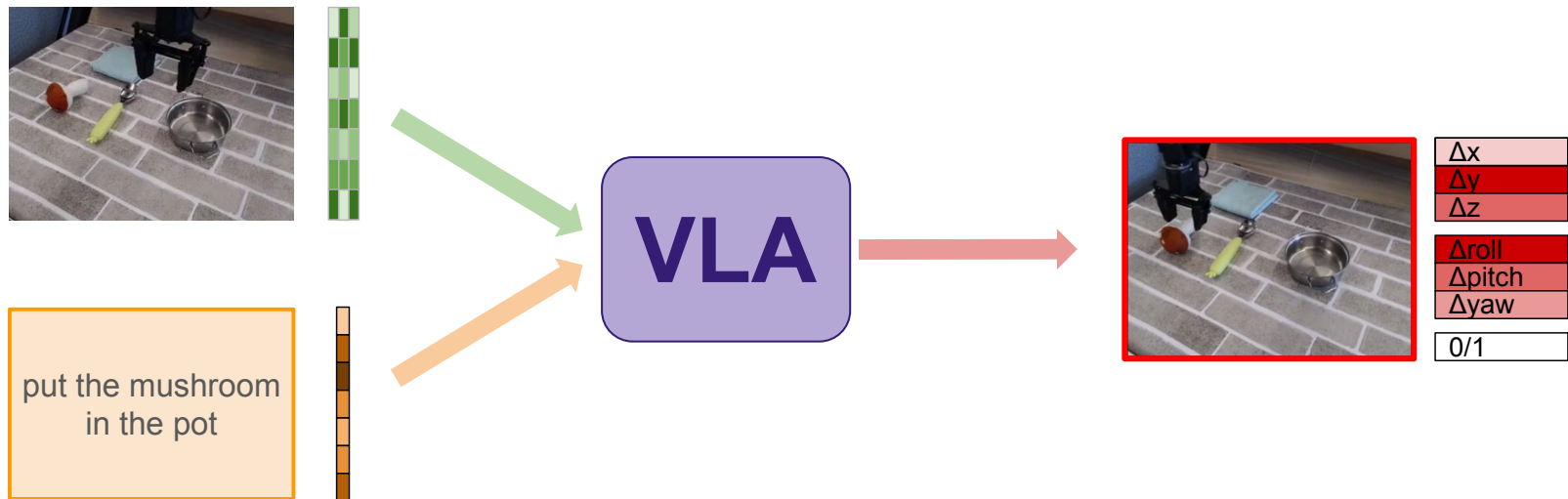
Komentarz:

Gripper_state informuje o tym, jaki mam być stan chwytaka po zakończeniu akcji (otwarty/zamknięty).



Vision-Language-Action Models

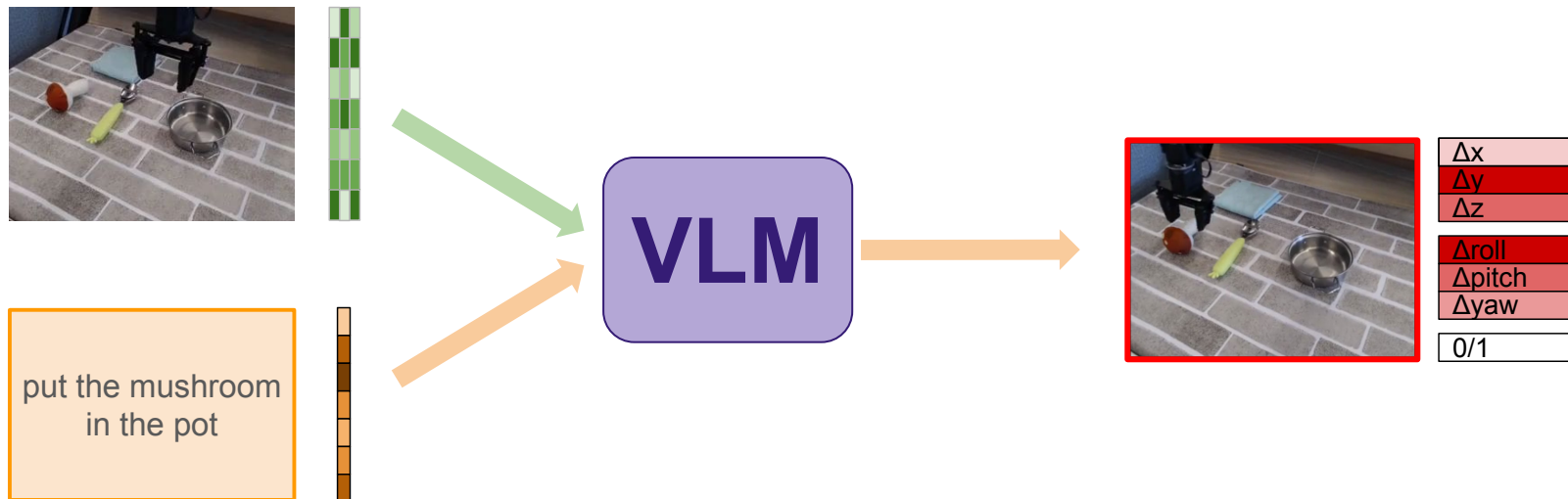
RT-2



Vision-Language-Action Models

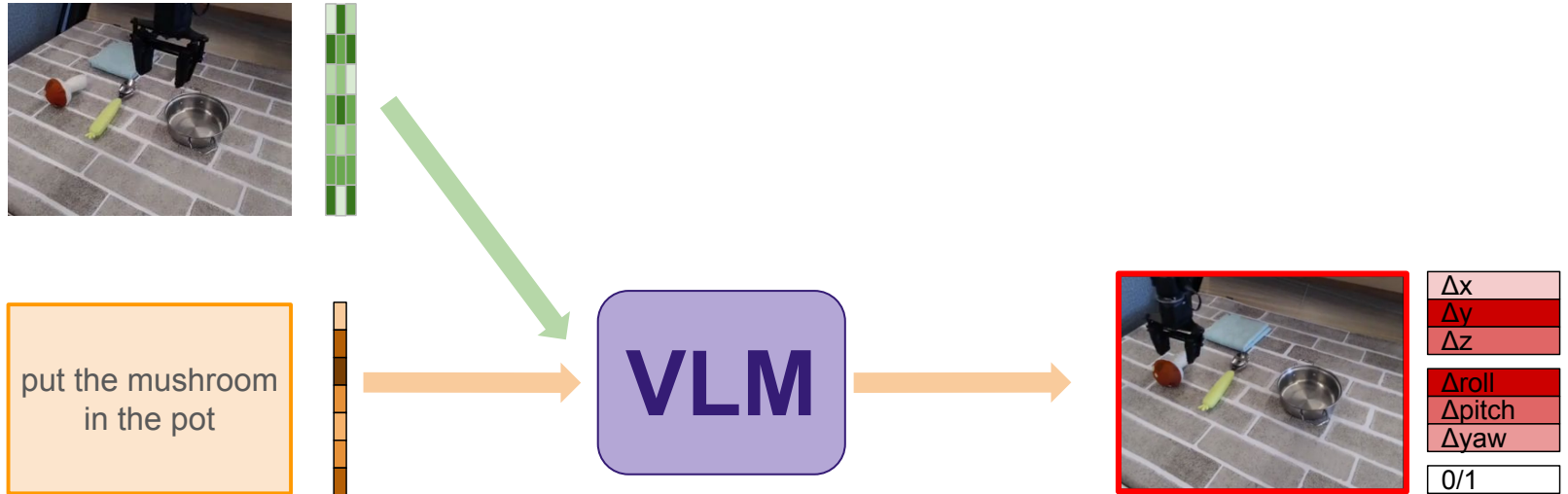
RT-2

*Komentarz:
Głównym modulem RT-2 (VLA) jest odpowiednio
wytrenowany VLM.*



Vision-Language-Action Models

RT-2

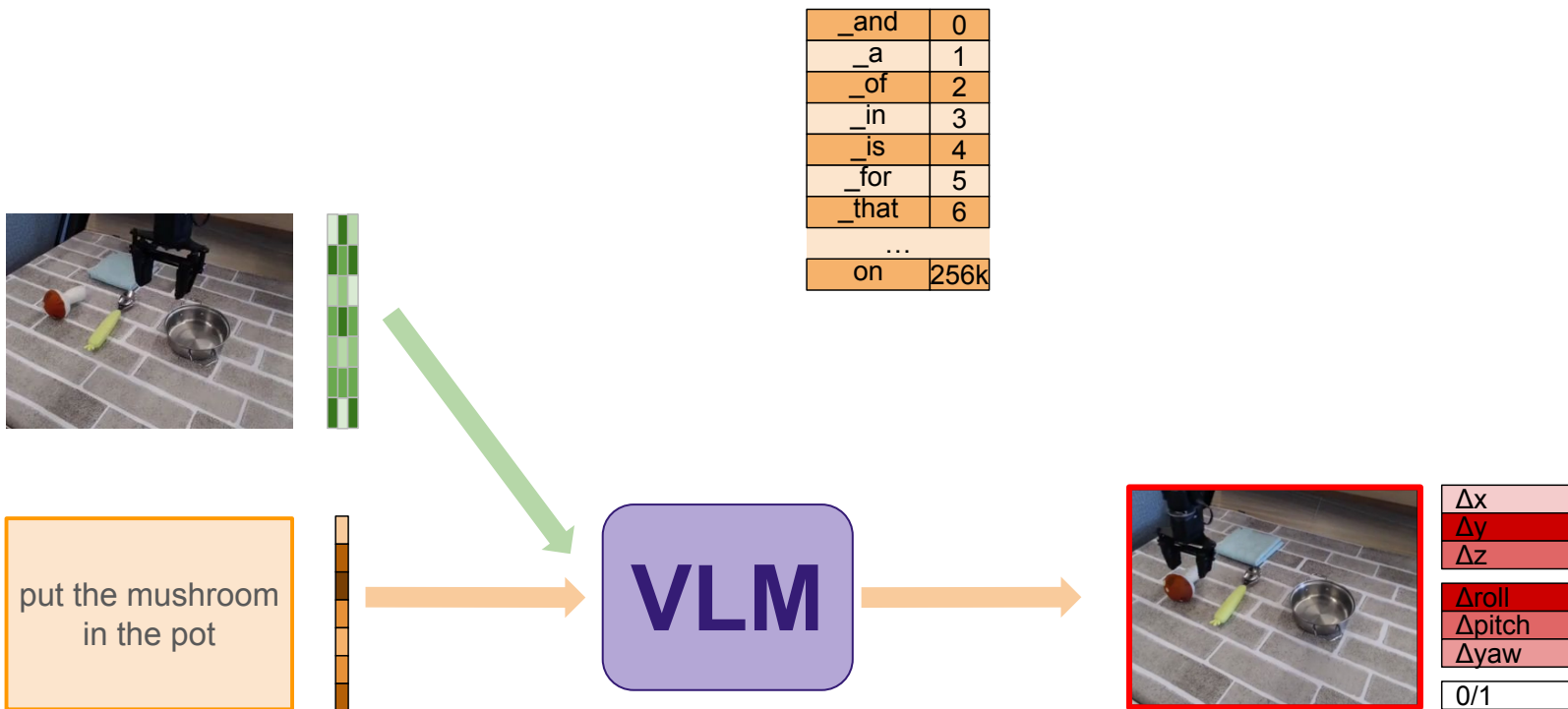


Vision-Language-Action Models

RT-2

Komentarz:

*Standardowo VLM zwraca token tekstowy.
Trudno zinterpretować takie tokeny jako akcje
roboty.*



_and	0
_a	1
_of	2
_in	3
_is	4
_for	5
_that	6
...	
on	256k

Vision-Language-Action Models

RT-2

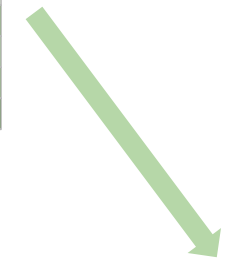
Komentarz:

Dokładamy 256 specjalnych tokenów, które model będzie zwracał, aby zasygnalizować odpowiednie przesunięcie w osi X.

Wspólna przestrzeń tokenów języka i akcji

_and	0
_a	1
_of	2
_in	3
_is	4
_for	5
_that	6
...	
on	256k

Δx_0	
Δx_1	
...	
Δx_{255}	



put the mushroom
in the pot



VLM



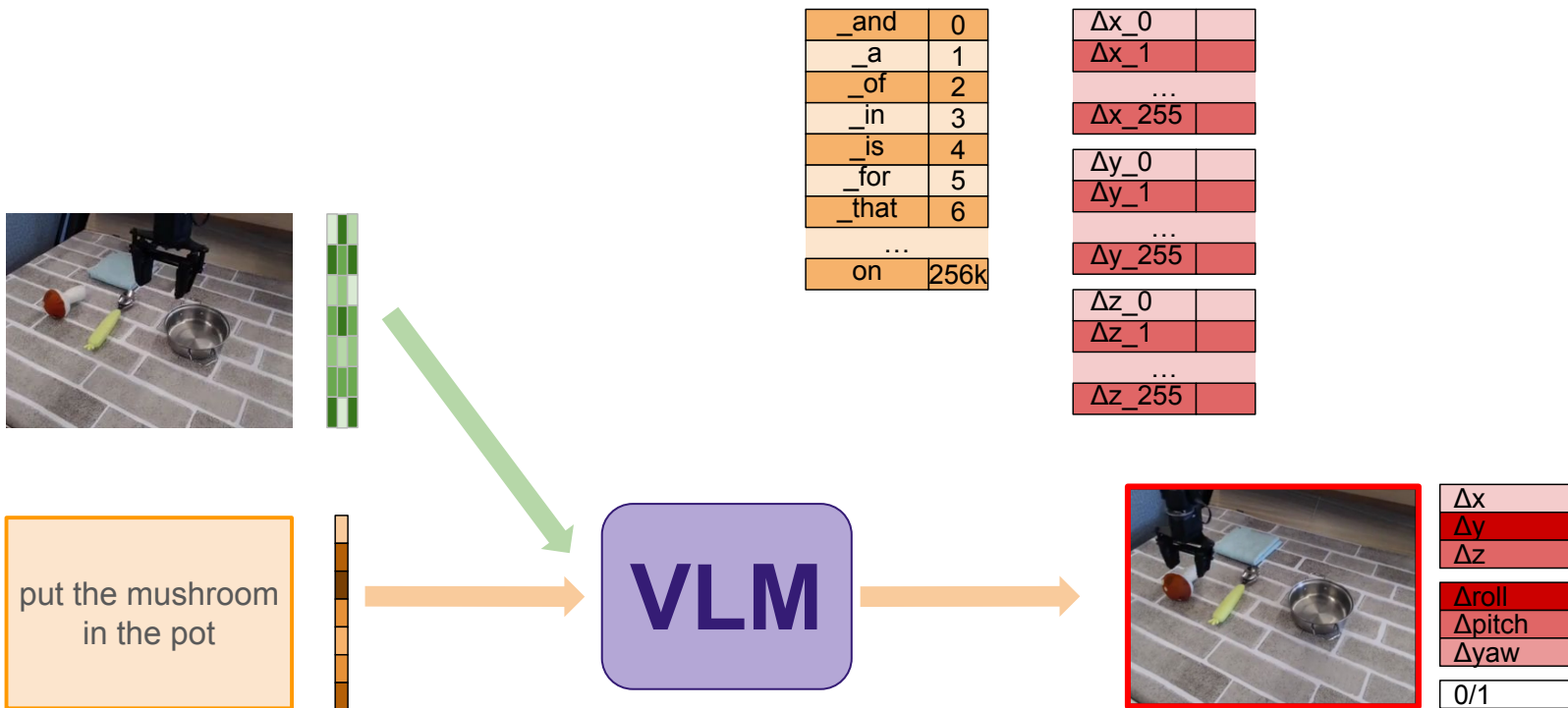
Δx
Δy
Δz
$\Delta roll$
$\Delta pitch$
Δyaw
0/1

Vision-Language-Action Models

RT-2

Komentarz:
Dokładamy 256 specjalnych tokenów dla przesunięcia w każdej z osi X, Y, Z.

Wspólna przestrzeń tokenów języka i akcji



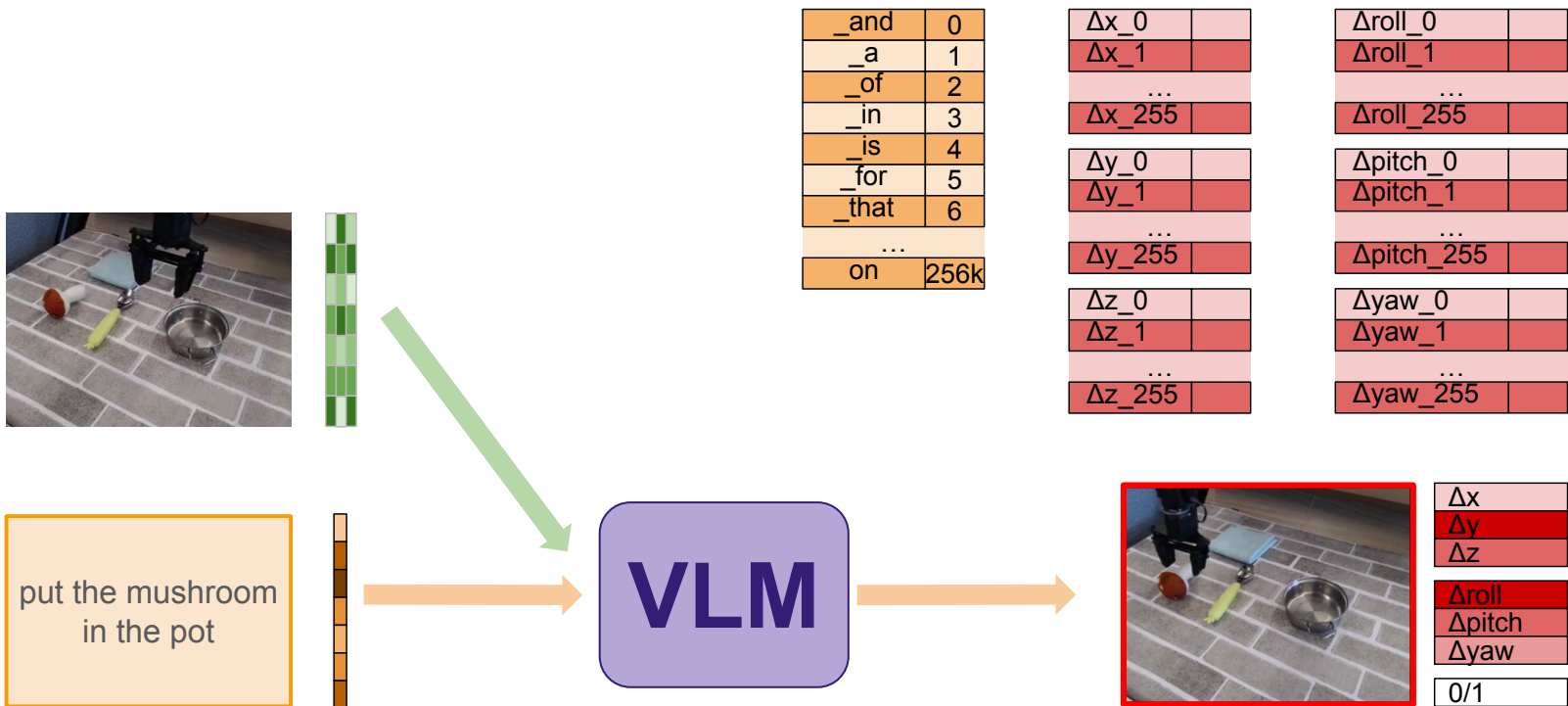
Vision-Language-Action Models

RT-2

Komentarz:

Dokładamy 256 specjalnych tokenów dla każdej z możliwych osi obrotu.

Wspólna przestrzeń tokenów języka i akcji

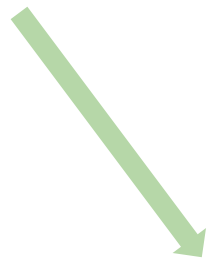


Vision-Language-Action Models

RT-2

*Komentarz:
Dokładamy 2 specjalne tokeny informujące o
stanie chwytaka po zakończeniu akcji.*

Wspólna przestrzeń tokenów języka i akcji



put the mushroom
in the pot



VLM



Δx	
Δy	
Δz	
$\Delta roll$	
$\Delta pitch$	
Δyaw	
0/1	

_and	0
_a	1
_of	2
_in	3
_is	4
_for	5
_that	6
...	
on	256k

grip_0	
grip_1	

Δx_0	
Δx_1	
...	
Δx_{255}	

Δy_0	
Δy_1	
...	
Δy_{255}	

Δz_0	
Δz_1	
...	
Δz_{255}	

$\Delta roll_0$	
$\Delta roll_1$	
...	
$\Delta roll_{255}$	

$\Delta pitch_0$	
$\Delta pitch_1$	
...	
$\Delta pitch_{255}$	

Δyaw_0	
Δyaw_1	
...	
Δyaw_{255}	

RT-2

Model będzie zwracał tokeny we wspólnej przestrzeni języka i akcji.

_and	0
_a	1
_of	2
_in	3
_is	4
_for	5
_that	6
...	
on	256k

Δx_0	
Δx_1	
...	
Δx_255	
Δy_0	
Δy_1	
...	
Δy_255	
Δz_0	
Δz_1	
...	
Δz_255	

Δroll_0	
Δroll_1	
...	
Δroll_255	
Δpitch_0	
Δpitch_1	
...	
Δpitch_255	
Δyaw_0	
Δyaw_1	
...	
Δyaw_255	



put the mushroom
in the pot



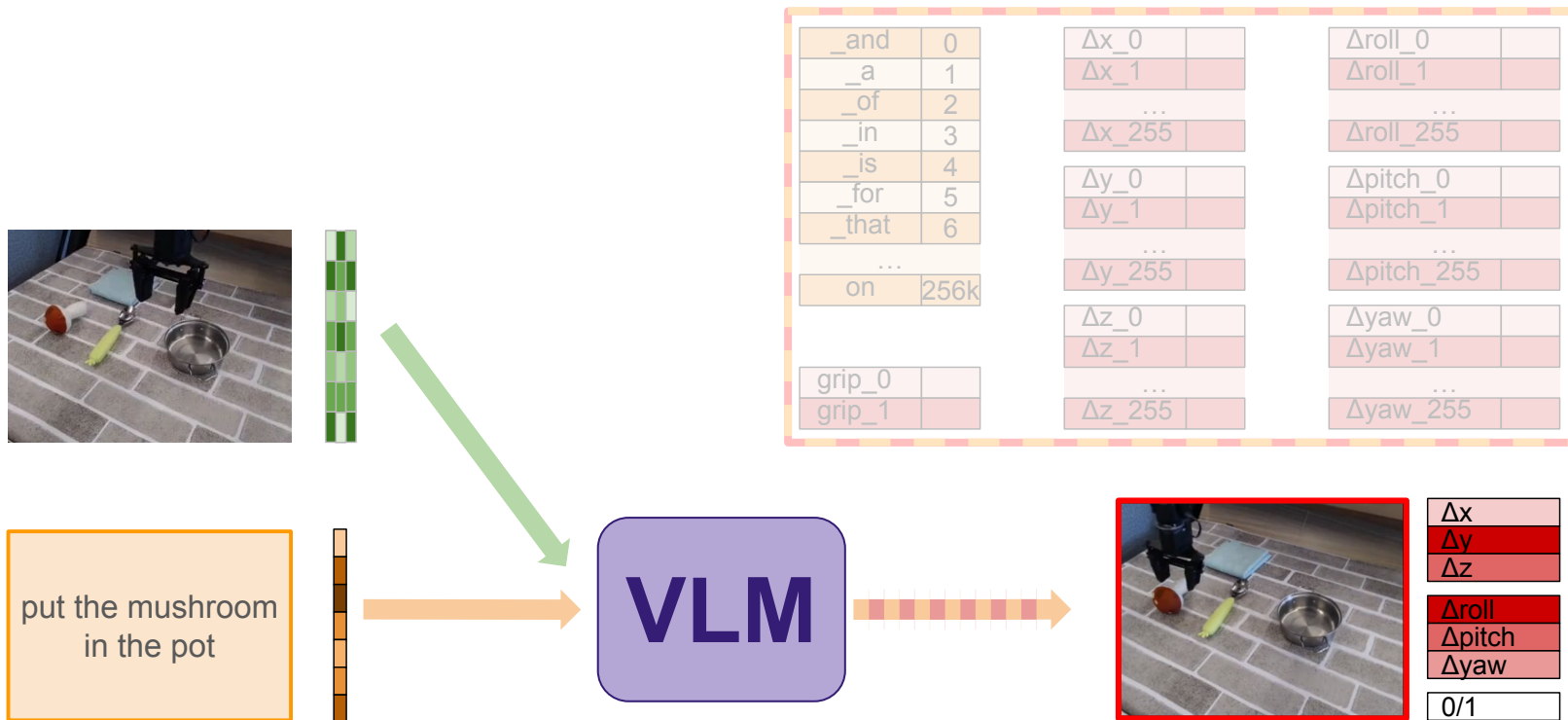
VLM



Δx
Δy
Δz
$\Delta roll$
$\Delta pitch$
Δyaw
0/1

RT-2

Wspólna przestrzeń tokenów języka i akcji



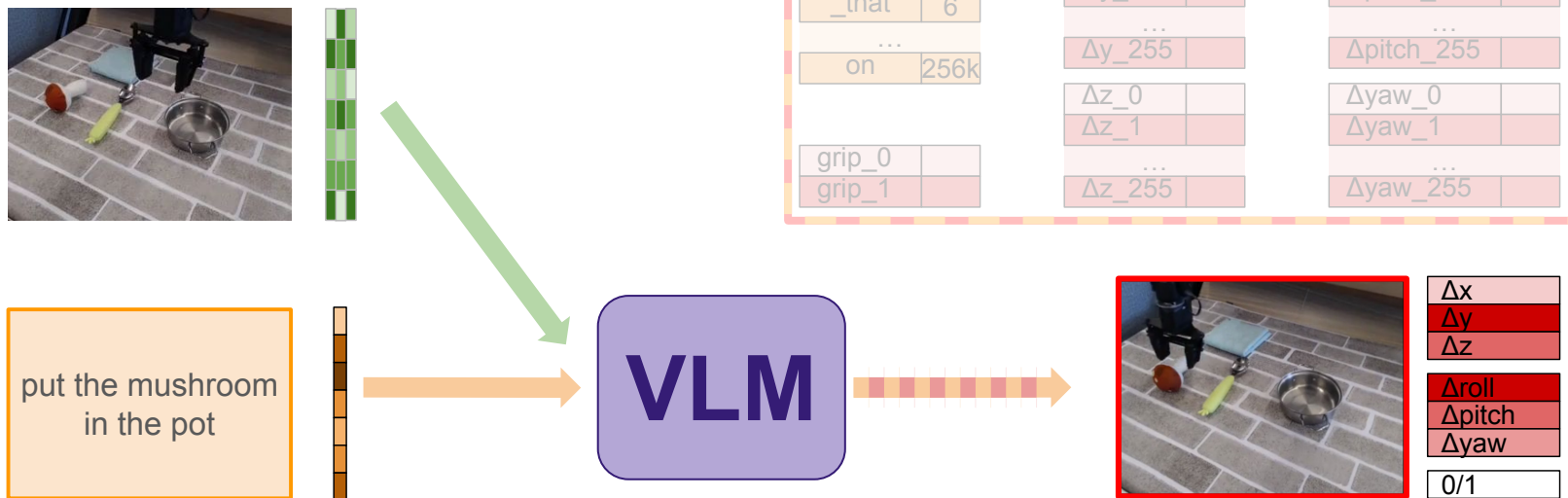
Vision-Language-Action Models

RT-2

Komentarz:

Drugą fazą treningu RT-2 był co-fine-tuning, czyli proces jednoczesnego uczenia na zadaniach wizyjno-tekstowych oraz zadaniach robotycznych

Wspólna przestrzeń tokenów języka i akcji



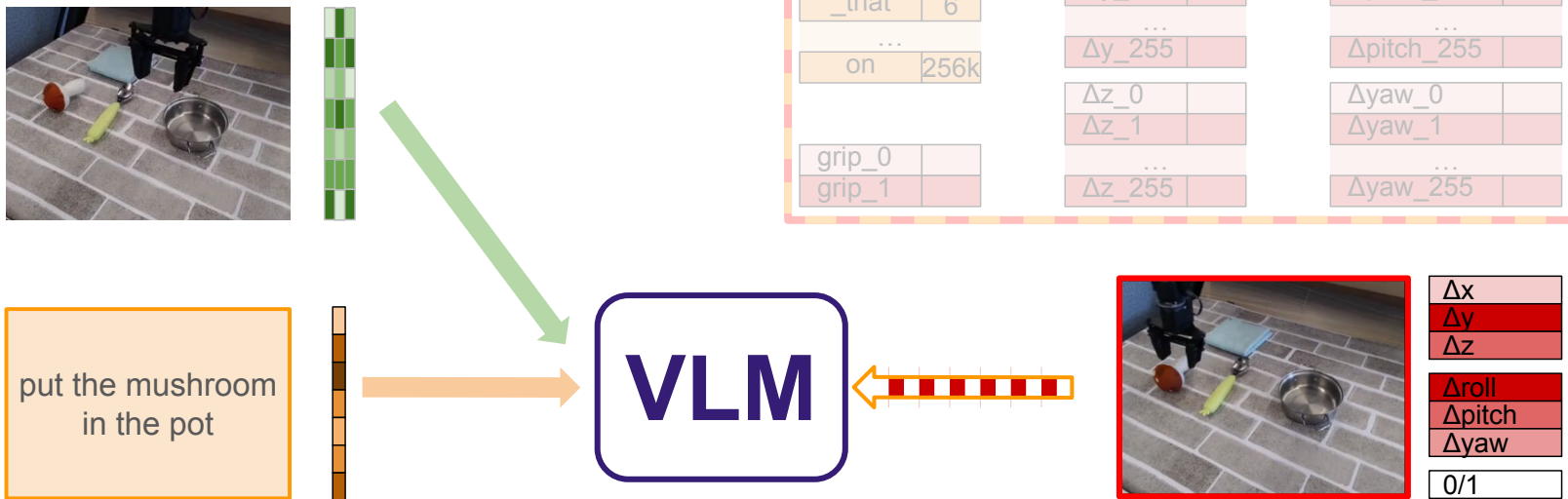
Vision-Language-Action Models

RT-2

Komentarz:

Na zmianę przekazywano batche z zadaniem dla robota (na podstawie którego uczył się zwracać tokeny akcji) oraz zadania wizyjno-tekstowe, dzięki którym nie zapominał semantyki obiektów.

Wspólna przestrzeń tokenów języka i akcji



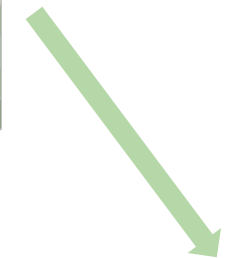
Vision-Language-Action Models

RT-2

Komentarz:

Na zmianę przekazywano batche z zadaniem dla robota (na podstawie którego uczył się zwracać tokeny akcji) oraz zadania wizyjno-tekstowe, dzięki którym nie zapominał semantyki obiektów.

Wspólna przestrzeń tokenów języka i akcji



Is this dog tired?



VLM



Yes, the dog looks tired, as it is lying down with its eyes closed and appears to be resting.



_and	0
_a	1
_of	2
_in	3
_is	4
_for	5
_that	6
...	
on	256k

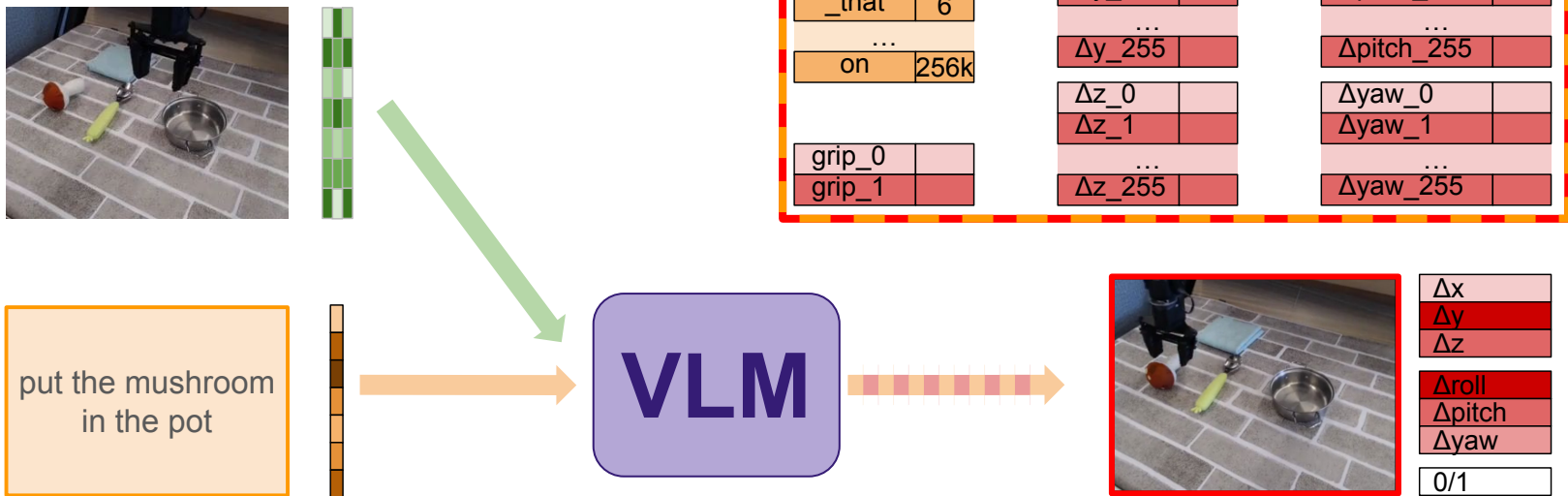
Δx_0	
Δx_1	
...	
Δx_{255}	
Δy_0	
Δy_1	
...	
Δy_{255}	
Δz_0	
Δz_1	
...	
Δz_{255}	

$\Delta roll_0$	
$\Delta roll_1$	
...	
$\Delta roll_{255}$	
$\Delta pitch_0$	
$\Delta pitch_1$	
...	
$\Delta pitch_{255}$	
Δyaw_0	
Δyaw_1	
...	
Δyaw_{255}	

Vision-Language-Action Models

RT-2

Wspólna przestrzeń tokenów języka i akcji



Źródła

Radford, A., Kim, J. W., & Hallacy, C., et al. (2021). *Learning Transferable Visual Models From Natural Language Supervision*. In **Proceedings of the 38th International Conference on Machine Learning (ICML 2021)**. arXiv:2103.00020. <https://arxiv.org/abs/2103.00020>

Liu, H., Li, C., & Wu, Q., et al. (2023). *Visual Instruction Tuning*. arXiv preprint arXiv:2304.08485. <https://arxiv.org/abs/2304.08485>

Hoffmann, J., Borgeaud, S., & Mensch, A., et al. (2022). *Training Compute-Optimal Large Language Models*. arXiv preprint arXiv:2203.15556. <https://arxiv.org/abs/2203.15556>

Alayrac, J.-B., Donahue, J., & Luc, P., et al. (2022). *Flamingo: a Visual Language Model for Few-Shot Learning*. In **Advances in Neural Information Processing Systems (NeurIPS 2022)**. arXiv:2204.14198. <https://arxiv.org/abs/2204.14198>

Brohan, A., Brown, N., & Carbajal, J., et al. (2023). *RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control*. In **Conference on Robot Learning (CoRL 2023)**. arXiv:2307.15818. <https://arxiv.org/abs/2307.15818>